

**Evaluación y comparación de plugins CNI para
Kubernetes:
un enfoque orientado a QoS, seguridad y eficiencia de
recursos en entornos de microservicios**

**Duwan Estiven Sierra Guerrero
Holman Audrey Alba Castro**

Director Gerardo Alberto Castang Montiel

Ingeniería Telemática
Telemática - Grupo de Investigación ORION
Universidad Distrital Francisco José de Caldas
Bogotá, D.C. - Colombia
Junio de 2026

Agradecimientos

GRACIAS

Resumen

Kubernetes no impone un plugin de red específico, lo que obliga a los equipos a seleccionar un *Container Network Interface* (CNI) sin métricas comparativas objetivas. Esta situación conduce a sobredimensionamiento de infraestructura (CPU/RAM) y a posturas de seguridad incompletas. El presente trabajo construye evidencia empírica reproducible —infraestructura como código, benchmarks automatizados y modelo de decisión multi-criterio (MCDA)— para guiar esa selección. Se evalúan Flannel, Calico, Cilium y Antrea en un clúster K3s en alta disponibilidad sobre DigitalOcean, midiendo *throughput* TCP, latencia de establecimiento, retransmisiones y overhead de CPU/RAM del agente CNI, así como el impacto de políticas de red Zero Trust. Los resultados alimentan un modelo MCDA parametrizable por perfil de uso (Fintech/seguridad, Streaming/rendimiento e IoT/recursos limitados) y un prototipo web que expone las recomendaciones. Los hallazgos principales muestran que Calico alcanzó el mayor *throughput* TCP (1804.40 Mbps) y la menor latencia de establecimiento; que Flannel —CNI predeterminado de K3s— acepta NetworkPolicies en la API de Kubernetes pero no las aplica en su plano de datos, configurando un falso positivo de seguridad; y que Cilium fue el único plugin capaz de aplicar políticas de capa 7 (HTTP). El modelo recomienda Cilium para el perfil Fintech, y Flannel para los perfiles Streaming e IoT por su estabilidad y menor huella de recursos.

Palabras clave: Kubernetes, CNI, NetworkPolicy, Zero Trust, MCDA, QoS, eBPF.

Abstract

Kubernetes does not impose a specific network plugin, forcing teams to select a *Container Network Interface* (CNI) without objective comparative metrics. This situation leads to infrastructure oversizing (CPU/RAM) and incomplete security postures. This thesis builds reproducible empirical evidence —infrastructure as code, automated benchmarks, and a multi-criteria decision analysis (MCDA) model— to guide this selection. Flannel, Calico, Cilium, and Antrea are evaluated on a high-availability K3s cluster on DigitalOcean, measuring TCP throughput, establishment latency, retransmissions, and CPU/RAM overhead of the CNI agent, as well as the impact of Zero Trust network policies. The results feed a parameterizable MCDA model by use profile (Fintech/security, Streaming/performance, and IoT/limited resources) and a web prototype that displays the recommendations. The main findings show that Calico achieved the highest TCP throughput (1804.40 Mbps) and the lowest connection-establishment latency; that Flannel —the default CNI in K3s— accepts NetworkPolicies through the Kubernetes API but does not enforce them in its data plane, constituting a security false positive; and that Cilium was the only plugin capable of enforcing layer-7 (HTTP) policies. The model recommends Cilium for the Fintech profile, and Flannel for the Streaming and IoT profiles due to its stability and smaller resource footprint.

Keywords: Kubernetes, CNI, NetworkPolicy, Zero Trust, MCDA, QoS, eBPF.

Tabla de contenidos

Resumen	II
Abstract	III
Tabla de Contenido	IX
Lista de tablas	XI
Lista de figuras	XII
1. Introducción	1
2. Planteamiento del Problema	3
3. Justificación	4
1. Pertinencia telemática	4
2. Impacto en las organizaciones	4
3. Aporte académico y entregables	5
4. Viabilidad	5
4. Objetivos	6
1. Objetivo General	6
2. Objetivos Específicos	6
5. Marco Teórico - Conceptual	7
1. El modelo de red de Kubernetes	7
1.1. Identidad de red del Pod	8
1.2. Nodos y reenvío de paquetes	8
1.3. Direccionamiento estable: Services y EndpointSlices	9
1.4. Ingreso de tráfico externo: Ingress y Gateway API	9
1.5. Aislamiento lógico: Namespaces y NetworkPolicies	9
1.6. Recorrido de una solicitud	11
2. La Interfaz de Red de Contenedores (CNI)	11
2.1. Especificación y funciones núcleo	11
2.2. Modelos de dataplane: redes superpuestas (Overlay) vs. enrutamiento nativo	11
2.3. Gestión de direcciones (IPAM)	12
2.4. MTU y fragmentación en redes con encapsulación	13
2.5. Observabilidad del plano de datos	13
2.6. Métricas de rendimiento y sobredimensionamiento	13
3. eBPF como mecanismo de aceleración en el kernel	14

4.	Arquitectura de Confianza Cero y control de red en Kubernetes	14
5.	Análisis de Decisión Multicriterio (MCDA)	15
6.	Estado del Arte	17
1.	Evaluación Comparativa de Rendimiento de CNIs en la Literatura	17
2.	Seguridad y Políticas de Red en la Literatura Académica	18
3.	Modelos de Decisión Multicriterio Aplicados a Infraestructura Cloud	19
3.1.	Brecha de Investigación	20
7.	Metodología del Proyecto	21
1.	Marco de trabajo Scrum	21
2.	Product Backlog y trazabilidad con los objetivos	22
3.	Planificación de Sprints (Sprint Planning)	22
8.	Diseño de la Solución	23
1.	Arquitectura de Red y Topología	23
1.1.	Red underlay	23
1.2.	Red overlay y CNI evaluados	24
2.	Sistema de Medición de Calidad de Servicio (QoS)	25
2.1.	Inyección de tráfico	25
2.2.	Variables telemáticas capturadas	25
2.3.	Pila de observabilidad: Prometheus y Grafana	26
3.	Modelo de Decisión Multicriterio (MCDA)	26
3.1.	Definición de criterios y métricas de evaluación	27
3.2.	Limpieza estadística con el Rango Inter cuartílico (IQR)	27
3.3.	Normalización continua Min-Max	28
3.4.	Factores de ponderación y restricciones no compensatorias	28
3.4.1.	Restricción no compensatoria de seguridad	29
3.5.	Modelo matemático de decisión multicriterio (MCDA)	29
4.	Seguridad y Micro-segmentación (Zero Trust)	30
4.1.	Alcance operativo declarado	30
4.2.	Requisitos verificables derivados del OE2	31
4.3.	Matriz extendida de casos de prueba TC-SEC	31
4.4.	Estrategia de Default Deny	31
4.5.	Automatización GitOps con ArgoCD	33
4.6.	Mapeo a marcos de referencia	33
5.	Flujo Integral del Sistema	33
6.	Arquitectura por Capas	34
7.	Trazabilidad y Requisitos de Diseño	34
8.	Prototipo Recomendador	36
9.	Alcance y Límites del Diseño	36

9. La Herramienta: Implementación y Despliegue	37
1. Aprovisionamiento de Infraestructura con Terraform	37
1.1. Estructura del Código Terraform (K8s-bootstrap/00.bootstrap) . . .	37
2. Configuración del Clúster K3s en Alta Disponibilidad	38
2.1. Base de Datos Externa para el Plano de Control	38
2.2. Instalación: clúster sin CNI para plugins externos y modo nativo para Flannel	38
3. Orquestación y Entrega Continua: GitOps con ArgoCD	39
3.1. El Patrón App of Apps	39
3.2. Configuración de selfHeal y prune	39
3.3. Overlays de Kustomize para los CNIs	40
4. Implementación del Stack de Observabilidad	40
4.1. Prometheus: El Recolector de Métricas	40
4.2. Grafana: Los Dashboards de Red	41
4.3. Resource Usage Exporter: Extracción Automatizada de Datos . . .	41
5. Implementación del Banco de Pruebas de QoS	41
5.1. Escenario de Rendimiento con iperf3	41
5.2. Cliente de Latencia Personalizado	42
6. Procesamiento Estadístico y Aplicación del Modelo MCDA	42
6.1. Limpieza con el Rango Intercuartílico (IQR)	43
6.2. Normalización y Cálculo del Score MCDA	43
7. Prototipo Recomendador: Aplicación Web SPA	44
7.1. Tecnologías de Implementación	44
7.2. Flujo de la Aplicación	44
8. Implementación de Network Policies y Seguridad Zero Trust	44
8.1. Políticas Default Deny	44
8.2. Micro-segmentación por Capas de Aplicación	45
8.3. Validación Automatizada de Políticas	45
9. Estructura de Repositorios y Entregables Técnicos	45
10. Desafíos de Implementación y Soluciones Adoptadas	45
11. Flujo completo de ejecución	47
10.Fase de Desarrollo por Sprints	48
1. Fase de preparación del laboratorio (Sprints 0–1)	48
2. Fase de evaluación de rendimiento de los CNI (Sprints 2, 4, 7 y 10)	48
3. Fase de seguridad y micro-segmentación (Sprints 3, 6 y 12)	49
4. Fase del modelo de decisión MCDA (Sprints 5 y 13)	49
5. Fase de la herramienta y la guía (Sprints 8 y 14)	49
6. Fase de observabilidad y <i>rightsizing</i> (Sprints 9 y 11)	49
7. Fase de integración y cierre (Sprints 15–16)	50

11.Resultados y Verificación de Objetivos	51
1. Metodología General de Pruebas	51
1.1. Control de Condiciones: el Escenario de Referencia	51
1.2. Automatización: de la Prueba Manual a la Prueba Orquestada	52
2. Pruebas de Rendimiento de Red — Objetivo Específico 1	52
2.1. Protocolo de Prueba de Throughput con iperf3	52
2.2. Resultados de Throughput TCP por CNI	53
2.3. Protocolo de Prueba de Latencia TCP	53
2.4. Resultados de Latencia TCP por CNI	54
3. Pruebas de Seguridad con Network Policies — Objetivo Específico 2	54
3.1. Matriz de Pruebas de Network Policies	54
3.2. Resultados de las Pruebas de Seguridad por CNI	54
3.3. Prueba de Overhead de Seguridad	56
4. Pruebas de Eficiencia Computacional — Consumo del Agente CNI	56
5. Validación del Modelo Matemático MCDA — Objetivo Específico 3	57
5.1. Validación del Algoritmo IQR	57
5.2. Validación de la Normalización Min-Max	58
5.3. Validación del Scoring MCDA por Perfil	58
6. El Prototipo Recomendador CNI como Herramienta de Decisión y Despliegue	59
6.1. Interfaz y Experiencia de Usuario	59
6.2. Asistencia de Despliegue e Instalación Guiada	61
6.3. Pruebas de Corrección Funcional	62
6.4. Prueba de Integridad de Datos de Extremo a Extremo	62
7. Análisis Consolidado de Resultados	63
7.1. Respuesta a las Preguntas de Investigación	63
7.2. Lecciones Aprendidas de las Pruebas	63
8. Tabla Resumen de Cumplimiento de Objetivos	64
12.Discusión	65
1. Impacto estructural del plano de datos	65
2. Falso positivo de seguridad en configuraciones predeterminadas	65
3. Eficiencia de la microsegmentación Zero Trust	66
4. Delimitación de la evaluación de seguridad	66
5. Limitaciones técnicas de la evaluación de seguridad	66
6. Limitaciones del modelo MCDA	66
13.Conclusiones	68
14.Recomendaciones	69
15.Trabajo Futuro	70
1. Implementación de Admission Control	70
2. Detección y Respuesta en Tiempo Real (Runtime Security)	70

3.	Integración de Identidad Criptográfica (Service Mesh)	70
4.	Pruebas de Estrés en Mapas eBPF L7	71
Referencias		73
Anexos		74
A. Manual de Usuario del Recomendador CNI		75
1.	Acceso a la herramienta	75
2.	Flujo de uso paso a paso	75
3.	Interpretación de los resultados	76
4.	Resolución de problemas frecuentes	76
B. Manual de Instalación y del Programador		77
1.	Requisitos previos	77
2.	Paso 1: Aprovisionamiento del testbed con Terraform	77
3.	Paso 2: Despliegue de aplicaciones con GitOps (ArgoCD)	78
4.	Paso 3: Ejecución de los benchmarks	78
5.	Paso 4: Procesamiento de datos y modelo MCDA	78
6.	Paso 5: Reconstrucción y publicación del recomendador (SPA)	79
7.	Estructura de repositorios	79
C. Anexo Técnico de Entregables y Repositorios		80
1.	Repositorio Principal del Proyecto	80
2.	Detalle Técnico y Entregables por Objetivo Específico	80
2.1.	Objetivo Específico 1: Aprovisionamiento y Automatización del Test- bed	80
2.1.1.	Repositorios del Testbed	81
2.1.2.	Pipeline de Aprovisionamiento en Cuatro Etapas	81
2.1.3.	Inventario de Recursos Aprovisionados	83
2.1.4.	Protocolo de Medición	83
2.1.5.	Flujo de Recolección y Persistencia de Datos	84
2.2.	Objetivo Específico 2: Políticas de Red y Microsegmentación Zero Trust	85
2.3.	Objetivo Específico 3: Implementación del Algoritmo MCDA	86
2.4.	Objetivo Específico 4: Prototipo Interactivo de Recomendación	86
3.	Guía de Reproducción y Verificación Científica	89
3.1.	Paso 0: Aprovisionamiento del Testbed	89
3.2.	Paso 1: Clonación del Repositorio y Reconstrucción de Datos	90
3.3.	Paso 2: Compilación y Despliegue de la SPA Recomendadora	90

D. Configuración y evaluación de las Network Policies por CNI	92
1. Adaptación específica por CNI (R2)	92
1.1. Fragmento de CiliumNetworkPolicy (L7)	92
1.2. Fragmento de ClusterNetworkPolicy de Antrea (Tier securityops)	93
1.3. Default Deny base	94
2. Reproducibilidad del OE2	94
E. Marco de Criterios y Modelo de Decisión CNI	95
1. Introducción y el dilema de la selección de CNI	95
2. Datos experimentales consolidados del testbed	96
3. Resultados MCDA y análisis de selección	96
3.1. Interpretación y desglose de resultados	96
4. Lista de verificación para la configuración del recomendador CNI	99
5. Guía de reproducción y verificación científica	102

Lista de tablas

7.1. Mapeo del Product Backlog con los objetivos específicos (OE).	22
7.2. Planificación maestra de sprints y su relación con los objetivos específicos.	22
8.1. CNI evaluados: tecnología base, versión desplegada y enfoque.	24
8.2. Variables telemáticas capturadas por el sistema de medición.	26
8.3. Matriz comparativa cualitativa de compromisos (trade-offs) en CNIs.	27
8.4. Estructura de criterios, preguntas de control y métricas del modelo de selección.	27
8.5. Matriz de factores de ponderación (pesos calculados) por perfil de la tesis.	28
8.6. Descomposición del OE2 en requisitos verificables y efectos esperados.	31
8.7. Matriz extendida TC-SEC: cada caso de prueba, amenaza modelada y requisito del OE2 al que aporta evidencia.	32
8.8. Casos de uso de seguridad implementados por CNI.	32
8.9. Mapeo conceptual de los casos TC-SEC a controles de marcos de referencia.	33
8.10. Trazabilidad entre problema de investigación, objetivo y decisión de diseño.	35
9.1. Archivos Terraform del módulo de aprovisionamiento y su función específica.	38
9.2. Instalación de CNIs: método y configuración clave en <code>user_data/ks3_server_init.sh</code>	39
9.3. Métricas de QoS capturadas por el banco de pruebas y su relevancia telemática.	42
9.4. Casos de prueba del pipeline de validación de Network Policies.	45
9.5. Estructura de los cuatro repositorios del proyecto y su relación con los objetivos específicos.	46
9.6. Desafíos técnicos encontrados durante la implementación y soluciones adoptadas.	46
11.1. Parámetros del escenario de referencia para todas las pruebas.	51
11.2. Resultados de rendimiento TCP por CNI (promedio de 5 corridas de 300 segundos, outliers IQR eliminados).	53
11.3. Resultados de latencia TCP por CNI (30 muestras $\times$ 5 corridas, outliers IQR eliminados). Todas las métricas: $\downarrow$ menor es mejor.	54
11.4. Matriz completa de pruebas de seguridad con Network Policies.	55
11.5. Resultados de pruebas de seguridad por CNI.	55

11.6. Overhead detallado de Network Policies por CNI y caso de uso, contextualizado con el baseline OE1.	56
11.7. Consumo computacional promedio del agente CNI bajo estrés de red (tráfico TCP intenso).	57
11.8. Validación del algoritmo IQR con outlier controlado inyectado artificialmente.	57
11.9. Validación de la normalización min-max de las seis métricas del modelo MCDA (Escala del Recommender: 1.00 = peor, 5.00 = mejor).	58
11.10 Puntajes MCDA finales por CNI y perfil (★ indica recomendación; escala 1–5, mayor es mejor).	59
11.11 Resultados de pruebas de corrección funcional del prototipo SPA.	62
11.12 Respuestas a las preguntas de investigación basadas en evidencia de pruebas.	63
11.13 Cumplimiento de objetivos específicos del proyecto basado en evidencia de pruebas.	64
12.1. Limitaciones técnicas de la evaluación de seguridad y su justificación de alcance.	67
C.1. Repositorios que conforman el entorno automatizado de experimentación.	81
C.2. Inventario de recursos de infraestructura aprovisionados por el testbed en cada corrida experimental.	83
C.3. Parámetros de los workloads de benchmark automatizados.	84
C.4. Responsabilidades de los componentes del recomendador SPA.	87
D.1. Capacidades específicas por CNI explotadas en las políticas de red.	93
E.1. Matriz comparativa cualitativa de compromisos (trade-offs) en CNIs.	95
E.2. Resultados experimentales consolidados del testbed con indicadores de optimización.	96
E.3. Puntuaciones consolidadas del modelo MCDA reales para cada perfil (escala 1–5).	97
E.4. Lista de verificación cualitativa para configurar los deslizadores del recomendador CNI.	101

Índice de figuras

5.1.	Topología de red de un clúster de Kubernetes (adaptado de [1]).	8
5.2.	Representación de política de red en Kubernetes (NetworkPolicy) con control de conexiones permitidas y denegadas. Fuente: elaboración propia. . .	10
5.3.	Diagrama de operación del CNI: interacciones entre Kubelet, Pod y plugins de red.	12
5.4.	Ajuste de MTU en presencia de encapsulación (p. ej., VXLAN) para evitar fragmentación y pérdida de rendimiento [2].	13
8.1.	Arquitectura por capas del testbed experimental.	35
11.1.	Interfaz de inicio y pantalla principal del prototipo Recomendador CNI. . .	60
11.2.	Guía interactiva de instalación y comandos recomendados para Cilium en el perfil Fintech.	61
11.3.	Guía interactiva de instalación y comandos recomendados para Flannel en el perfil IoT.	62
C.1.	Pipeline de aprovisionamiento del testbed: cuatro etapas Terraform con barreras de estado idempotentes (<code>K8s-bootstrap/00.bootstrap</code>).	82
C.2.	Flujo de recolección y persistencia de datos experimentales: desde los Cron-Jobs de Kubernetes hasta el dataset procesado que alimenta el modelo MCDA. . .	84
C.3.	Arquitectura de tres capas del recomendador SPA.	87
C.4.	Pantalla principal del recomendador SPA: <i>GuidedProfileBuilder</i> e indicador de fuente de datos activa.	89
C.5.	Salida accionable del recomendador para dos perfiles de uso.	90
E.1.	Configuración de sliders para el perfil Fintech / Seguridad en el Recomendador CNI.	98
E.2.	Resultados del ranking MCDA en el prototipo interactivo para el perfil Fintech / Seguridad (Ganador: Cilium con 3.631).	98
E.3.	Configuración de sliders para el perfil IoT / Recursos Limitados en el Recomendador CNI.	100
E.4.	Resultados del ranking MCDA en el prototipo interactivo para el perfil IoT / Recursos Limitados (Ganador: Flannel con 3.834).	100

Capítulo 1

Introducción

La adopción de Kubernetes como plataforma de orquestación de contenedores se ha consolidado en la industria tecnológica, pero su viabilidad operativa depende en gran medida del componente de red que conecta los servicios dentro del clúster. Kubernetes no incluye por defecto un mecanismo de conectividad entre contenedores: delega esa función en un complemento intercambiable denominado Container Network Interface (CNI), cuya selección recae en el equipo técnico de cada organización.

Los equipos deben escoger entre múltiples complementos —Calico, Cilium, Flannel y Antrea, entre otros— sin contar con criterios de comparación objetivos ni datos de rendimiento medidos en condiciones representativas. Esta carencia conduce a tres problemas interrelacionados: configuraciones de red inseguras por la elección de complementos que no aplican políticas de control de acceso en su plano de datos, sobredimensionamiento de recursos computacionales por desconocimiento del costo real de cada complemento en CPU y memoria, y una exposición silenciosa a vulnerabilidades que el equipo operativo no detecta hasta que afectan a los usuarios.

La pregunta de investigación que motiva este trabajo es la siguiente: ¿cómo pueden las organizaciones comparar complementos de red y automatizar políticas de comunicación segura para optimizar recursos sin degradar el rendimiento?

De esta pregunta principal se derivan cuatro sub-preguntas específicas que estructuran la evaluación empírica:

1. ¿Cuáles son las diferencias en velocidad de respuesta —throughput, latencia y re-transmisiones TCP— entre los plugins CNI evaluados bajo condiciones controladas de carga?
2. ¿Cómo impacta cada CNI en el consumo de recursos del nodo —CPU y memoria RAM— durante sesiones de red de alta intensidad?
3. ¿Qué tan efectivo es cada CNI para aplicar Network Policies de seguridad, y cuál es el overhead que introduce sobre el rendimiento?
4. ¿En qué medida la diferencia de eficiencia entre plugins permite reducir el sobredimensionamiento de infraestructura a escala de clúster?

Esta tesis tiene como objetivo desarrollar un modelo de evaluación, selección y validación de complementos CNI en clústeres Kubernetes.

La evaluación empírica de complementos de red permite fundamentar esas decisiones con datos medidos, no con suposiciones. Este trabajo proporciona esa evidencia para cuatro CNIs representativos, junto con una metodología replicable por cualquier equipo que necesite realizar la misma comparación.

Este trabajo evalúa Flannel, Calico, Cilium y Antrea en un clúster K3s hospedado en la nube de DigitalOcean. La evaluación mide la velocidad de transferencia, la latencia, el uso de CPU, de memoria y la efectividad de las políticas de seguridad. No obstante, esta investigación solo considera configuraciones de red superpuestas (overlay). El estudio excluye despliegues en servidores físicos (bare-metal) o enrutamiento nativo debido a las restricciones de la red virtual privada.

El capítulo 2 revisa la literatura sobre evaluación de CNIs, seguridad Zero Trust y modelos de decisión. El capítulo 3 formula el problema y el capítulo 4 presenta la justificación de la investigación. Los capítulos 5, 6 y 7 definen los objetivos, el marco teórico y la metodología. Los capítulos 8 al 12 documentan los resultados, la discusión, las conclusiones, las recomendaciones y el trabajo futuro.

Capítulo 2

Planteamiento del Problema

Kubernetes se ha convertido en el estándar de facto para desplegar aplicaciones modernas en contenedores, pero el rendimiento, la seguridad y la eficiencia de un clúster no dependen solo del orquestador. Dependen del complemento de red que conecta los servicios entre sí. Esta responsabilidad recae en los plugins CNI, que determinan cómo se enrutan los paquetes, qué tan rápido viajan y qué comunicaciones están permitidas o bloqueadas.

El problema es que no existe un criterio objetivo y ampliamente adoptado para elegir entre los distintos CNIs disponibles. Las decisiones de selección suelen apoyarse en documentación dispersa, recomendaciones informales o en el plugin configurado por defecto —en el caso de K3s, Flannel—, sin datos de rendimiento medidos en condiciones representativas. El resultado concreto es triple: configuraciones de red inseguras, porque algunos complementos no aplican en el tráfico real las políticas que el operador cree activas; clústeres sobredimensionados, porque sin conocer el costo real de CPU y RAM de cada CNI se agregan recursos a modo de margen de seguridad; y fallos que solo se detectan cuando ya afectan al usuario final, porque la red interna del clúster carece de observabilidad.

El problema se agrava porque Kubernetes provee los mecanismos necesarios —NetworkPolicies, autoescalado, observabilidad—, pero su aplicación efectiva requiere conocimientos especializados y pruebas sistemáticas que la mayoría de equipos no tiene capacidad de realizar. En entornos donde la disponibilidad es crítica, operar sobre supuestos en lugar de evidencias convierte la red en el punto débil del sistema.

Esta investigación parte de la necesidad de comparar CNIs con datos objetivos y de automatizar la validación de políticas de red, para que las organizaciones puedan garantizar seguridad, rendimiento y eficiencia sin incrementar la complejidad operativa. La brecha no es menor: mientras Kubernetes promete escalabilidad y confiabilidad, la elección no informada del complemento de red puede comprometer ambas.

Capítulo 3

Justificación

La adopción de Kubernetes como estándar de facto para el despliegue de aplicaciones en contenedores ha trasladado una decisión crítica al equipo técnico de cada organización: la selección del complemento de red (CNI) que conecta los servicios dentro del clúster. Esta decisión es de naturaleza telemática —determina cómo se enrutan los paquetes, con qué latencia y throughput viajan, qué comunicaciones se permiten y cuántos recursos consume el plano de red—, pero en la práctica se toma sin evidencia medida, apoyándose en documentación dispersa o en el complemento configurado por defecto [3]. Este trabajo se justifica precisamente por esa brecha: convierte una elección basada en intuición en una decisión fundamentada en datos reproducibles.

1. Pertinencia telemática

El aporte se ubica en el núcleo de la ingeniería telemática: la convergencia entre redes de datos y servicios distribuidos. Tres dimensiones telemáticas estructuran la solución. La primera es la **Calidad de Servicio (QoS)**: la medición controlada de throughput, latencia de establecimiento y retransmisiones TCP entre nodos cuantifica el comportamiento real de la red del clúster bajo carga. La segunda es la **observabilidad de red**: la instrumentación con Prometheus y Grafana sobre el agente CNI hace visible el tráfico este-oeste interno, un plano que las herramientas tradicionales de monitoreo no cubren. La tercera es la **micro-segmentación bajo Zero Trust**: la automatización y validación de NetworkPolicies traduce el principio de mínimo privilegio en reglas verificables sobre el plano de datos. En conjunto, el proyecto no evalúa un producto de software genérico, sino el comportamiento telemático de la infraestructura de comunicaciones de un clúster.

2. Impacto en las organizaciones

El beneficio para las organizaciones que sigan esta guía de implementación es directo y medible. Conocer el costo real en CPU y memoria de cada CNI por unidad de tráfico permite dimensionar la infraestructura con base en evidencia y reducir el *sobredimensionamiento*

preventivo —la práctica de adquirir capacidad “por si acaso” que genera gasto innecesario en infraestructura y energía [4, 5]. La validación automatizada de políticas de red cierra brechas de seguridad que suelen permanecer ocultas hasta que un incidente las revela, sin elevar la complejidad operativa. Y la entrega de un método replicable —infraestructura como código, *benchmarks* automatizados y un modelo de decisión— permite que equipos sin especialización profunda en redes de Kubernetes realicen la misma comparación en su propio entorno. El impacto se resume en tres ejes: decisiones de arquitectura basadas en datos, reducción de costos operativos y una postura de seguridad verificable.

3. Aporte académico y entregables

Frente a los estudios comparativos existentes, que suelen reportar mediciones aisladas de rendimiento, este trabajo integra la evaluación de QoS, la seguridad y la eficiencia de recursos en un único modelo de decisión multicriterio parametrizable por perfil de uso. El resultado tangible no es solo el documento, sino una **herramienta**: un testbed reproducible y un prototipo recomendador que expone, para cada perfil organizacional, qué CNI conviene y por qué. Esta orientación hacia un producto verificable —y no únicamente hacia un informe— es coherente con la modalidad de monografía orientada a la solución de un problema tecnológico mediante modelos, prototipos y productos [3].

4. Viabilidad

El proyecto es viable técnica, económica y legalmente. La totalidad del *stack* —Kubernetes (K3s), Calico, Cilium, Flannel, Antrea, Prometheus, Grafana, iperf3— se distribuye bajo licencias permisivas (Apache 2.0, MIT, BSD o GPL) que habilitan su uso académico sin restricciones. La infraestructura se aprovisiona bajo demanda en la nube pública con un costo acotado y controlado mediante automatización, y no se manipulan datos personales ni información sensible, por lo que no aplican consideraciones de protección de datos.

Capítulo 4

Objetivos

1. Objetivo General

Desarrollar un modelo de evaluación, selección y validación de plugins CNI en Kubernetes, compuesto por una guía metodológica, un prototipo de pruebas de laboratorio y una lista de verificación.

2. Objetivos Específicos

1. Evaluar cuantitativamente el rendimiento de los CNI disponibles para Kubernetes (Calico, Cilium, Flannel y Antrea) mediante pruebas controladas de latencia, velocidad de transferencia y uso de recursos, para identificar sus fortalezas y limitaciones en escenarios reales.
2. Evaluar cómo la configuración segura y automatizada de las políticas de red adaptadas a cada plugin CNI reduce la superficie de ataque y la probabilidad de incidentes de seguridad en clústeres Kubernetes, con el fin de determinar su capacidad de reducir vulnerabilidades y prevenir incidentes de seguridad en entornos de laboratorio representativos de producción.
3. Desarrollar, como parte del modelo propuesto, un conjunto de criterios y métricas que permitan la comparación objetiva entre diferentes CNI para Kubernetes mediante la definición de umbrales de rendimiento y factores de ponderación, para facilitar la toma de decisiones técnicas basadas en evidencia cuantificable.
4. Implementar un prototipo funcional que recomiende y configure el CNI más adecuado según las necesidades específicas de cada organización, para simplificar el proceso de implementación y apoyar la definición de configuraciones recomendadas en entornos de prueba.

Capítulo 5

Marco Teórico - Conceptual

1. El modelo de red de Kubernetes

Kubernetes (K8S) no implementa por sí mismo la conectividad entre cargas de trabajo; define un modelo de red y delega su realización en componentes externos. Comprender este modelo es necesario para interpretar las métricas del experimento, dado que el plano de red —y en particular el plugin que lo implementa— determina el rendimiento, la seguridad y la eficiencia en la utilización de recursos dentro del clúster [1].

El modelo de red de Kubernetes establece tres principios fundamentales:

1. Cada Pod recibe una dirección de Protocolo de Internet (IP, Internet Protocol) propia y única.
2. Los Pods se comunican entre sí sin traducción de direcciones de red (NAT, Network Address Translation).
3. Los agentes del host (como kubelet) alcanzan a todos los Pods ubicados en ese nodo.

Sobre estas garantías se construyen las abstracciones de descubrimiento, balanceo y control de acceso del clúster [1].

La siguiente analogía ayuda a visualizar cómo interactúan estos componentes:

- **Pods (Viviendas):** Constituyen las residencias particulares del clúster. Cada Pod cuenta con una dirección IP única y efímera que le permite comunicarse directamente con cualquier otra vivienda del entorno sin intermediarios.
- **Nodos (Barrios):** Alojan múltiples viviendas y proveen los recursos físicos y de cómputo (CPU, memoria y conectividad) indispensables para la subsistencia de los servicios.
- **Servicios (Central telefónica):** Operan como un conmutador estable con un número de contacto único e invariable. Esto permite canalizar el tráfico hacia las réplicas adecuadas, independientemente de los cambios internos en su ciclo de vida.

- **Ingress y Gateway API (Portería):** Representan el acceso administrado y seguro desde el exterior (Internet), asegurando el gobierno del tráfico entrante y la separación de responsabilidades funcionales.

El sustrato vial que hace posible la comunicación física entre las viviendas y barrios corresponde a la Interfaz de Red de Contenedores (CNI, Container Network Interface), la cual se detalla en la Figura 5.1 y define la infraestructura de red subyacente.

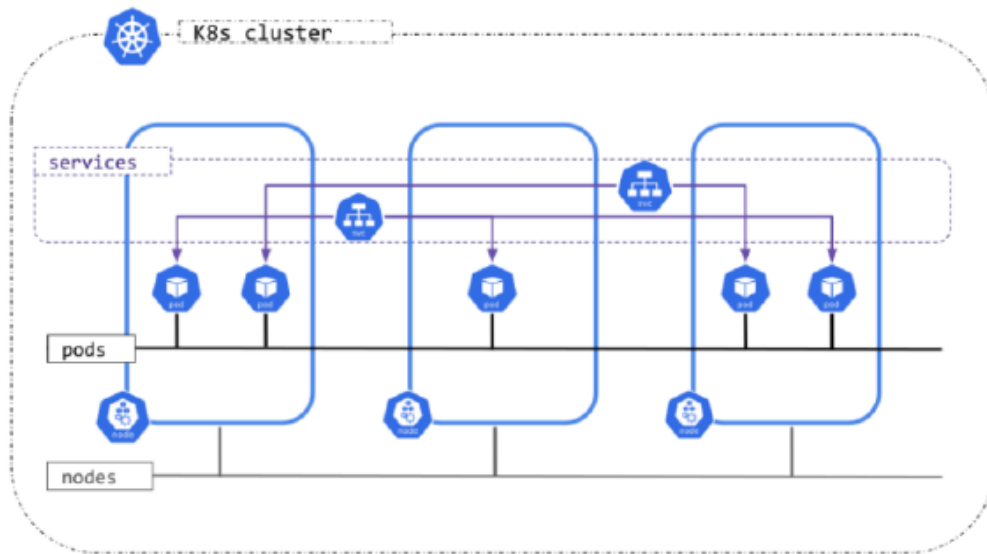


Figura 5.1: Topología de red de un clúster de Kubernetes (adaptado de [1]).

1.1. Identidad de red del Pod

Kubernetes adopta el modelo de direccionamiento IP por Pod, asignando una dirección única a cada unidad de ejecución. Este enfoque facilita que las cargas de trabajo interactúen de forma directa y nativa sin recurrir a NAT. Dado que la identidad del Pod es efímera —las réplicas se crean, destruyen y sustituyen de manera dinámica por los controladores—, la plataforma requiere abstracciones estables para asegurar el descubrimiento y direccionamiento consistente de los destinos [1].

1.2. Nodos y reenvío de paquetes

Cada nodo (Host o máquina virtual/física) hospeda múltiples Pods y aporta recursos de CPU, memoria y conectividad. En este nivel se instalan las rutas del sistema operativo y los mecanismos de reenvío local que garantizan la entrega de paquetes al Pod de destino, conforme al modelo de red del clúster: alcance directo entre Pods, ausencia de superposición de rangos de red y conectividad bidireccional este-oeste [1]. El agente del plugin de red se ejecuta precisamente en este nivel; por ello, su consumo de CPU y memoria por nodo Worker constituye una de las métricas centrales de la comparativa.

1.3. Direccionamiento estable: Services y EndpointSlices

Un Service expone un identificador estable —una dirección IP virtual (VIP, Virtual IP) conocida como ClusterIP— que representa a un conjunto de Pods equivalentes. El cliente se conecta a esa IP virtual y la plataforma balancea la solicitud hacia una de las réplicas disponibles, desacoplando a los consumidores de los cambios en el ciclo de vida de los Pods [6, 7].

Para mantener actualizada la relación de destinos, Kubernetes emplea *EndpointSlices*, los cuales mantienen un inventario actualizado de las direcciones IP y puertos de los Pods que respaldan cada Service. Este mecanismo reduce significativamente la sobrecarga de control frente a los antiguos Endpoint monolíticos en clústeres de gran escala [8]. Esta capa es relevante para interpretar la latencia medida: el tiempo en que una solicitud alcanza el Pod correcto comprende tanto la resolución del balanceador (iptables, IPVS o eBPF) como el reenvío realizado por el plugin de red.

1.4. Ingreso de tráfico externo: Ingress y Gateway API

El tráfico procedente de Internet no accede directamente a los Pods, sino a través de una capa de ingreso administrada. La *Gateway API* formaliza los roles asociados —infraestructura, operaciones y desarrollo— y las reglas de capa 7 (como hostnames, rutas y terminación TLS) que determinan el enrutamiento de cada solicitud hacia el Service correspondiente [9].

En despliegues de alta disponibilidad, esta capa suele estar precedida por un balanceador de carga externo de capa 3/4. Diseños avanzados de balanceo como *Maglev* demuestran cómo la combinación de hash consistente y seguimiento de flujos de red optimiza la distribución de carga antes de impactar los nodos del clúster [10].

1.5. Aislamiento lógico: Namespaces y NetworkPolicies

Un *Namespace* representa una delimitación lógica dentro del plano de control, orientada a organizar y estructurar recursos por proyectos, equipos o entornos (como desarrollo, pruebas y producción) [1]. Esta separación garantiza la unicidad de nombres DNS internos en la forma `<servicio>.<namespace>.svc.cluster.local`, lo cual previene ambigüedades en la resolución de nombres [1, 7].

Sin embargo, los Namespaces no restringen la comunicación física de red por defecto. La conectividad este-oeste permanece abierta a menos que se definan reglas explícitas de control de acceso mediante NetworkPolicies, aplicando principios de Confianza Cero (*Zero Trust*) para limitar el tráfico exclusivamente a lo autorizado [11–14].

Las NetworkPolicies funcionan como firewalls lógicos distribuidos de capa 3/4 que permiten o restringen conexiones basándose en selectores de etiquetas (*labels*), puertos y protocolos, como se ilustra en la Figura 5.2.

Su aplicación efectiva depende de que el CNI implemente el plano de control y de datos correspondiente. Si el plugin CNI seleccionado carece de soporte para NetworkPolicies, la

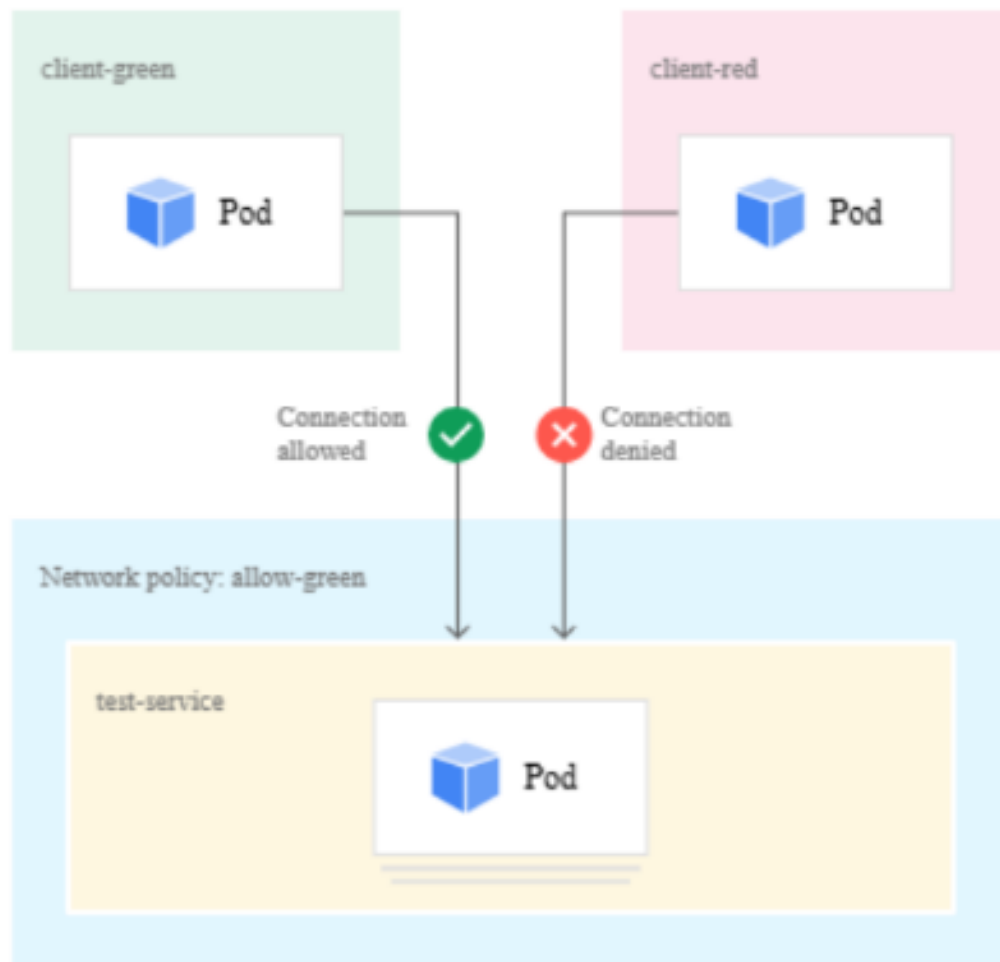


Figura 5.2: Representación de política de red en Kubernetes (NetworkPolicy) con control de conexiones permitidas y denegadas. Fuente: elaboración propia.

API de Kubernetes aceptará el recurso, pero no se aplicará ninguna regla de seguridad en los nodos, dejando el clúster expuesto [11, 15].

1.6. Recorrido de una solicitud

El ciclo completo del tráfico responde a la siguiente secuencia operativa:

1. El cliente externo envía una solicitud que llega al balanceador y se dirige a la portería lógica (Ingress o Gateway API) [9].
2. La portería inspecciona el paquete en capa 7 y lo reenvía al Service adecuado [6, 7].
3. El Service distribuye la petición consultando los EndpointSlices activos para elegir un Pod de destino [8].
4. El plugin de red encamina el tráfico hasta el Pod de destino en su nodo correspondiente [1, 11, 15].

2. La Interfaz de Red de Contenedores (CNI)

2.1. Especificación y funciones núcleo

La especificación Container Network Interface (CNI) define el estándar mediante el cual el runtime de contenedores interactúa con diversos plugins para automatizar la configuración de red. La especificación se fundamenta en operaciones simples y encadenables como la adición (ADD) y la eliminación (DEL) de interfaces de red de los contenedores [15–18], un proceso ilustrado esquemáticamente en la Figura 5.3.

Las funciones básicas que implementa cualquier plugin CNI son:

- **Identidad de Red:** Asignación de IPs del clúster que viabilizan el modelo IP-por-Pod y aseguran la comunicación este-oeste [1].
- **Enlace e Interfaces:** Creación de veth-pairs para conectar el espacio de nombres de red (*network namespace*) del Pod con la pila de red del host [11, 15].
- **Gestión de Direcciones (IPAM):** Reserva y liberación sistemática de direcciones de red a través de componentes como host-local [15, 16].

2.2. Modelos de dataplane: redes superpuestas (Overlay) vs. enrutamiento nativo

El plano de datos del CNI se suele clasificar en dos estrategias principales:

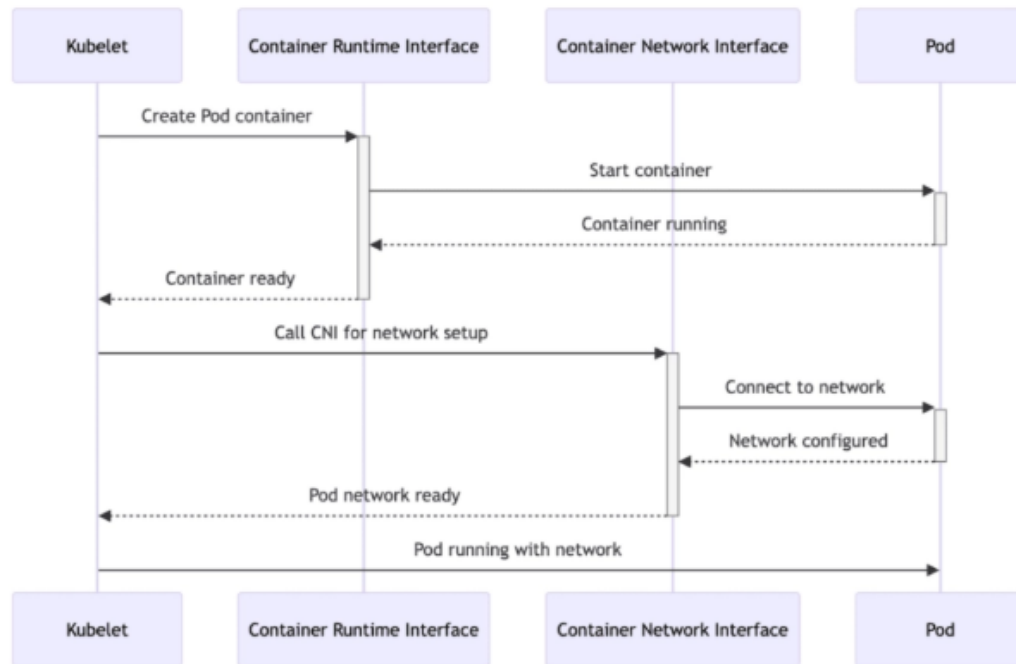


Figura 5.3: Diagrama de operación del CNI: interacciones entre Kubelet, Pod y plugins de red.

- **Redes Superpuestas (Overlay):** Encapsulan los paquetes de los Pods dentro de protocolos de transporte como VXLAN o Geneve sobre UDP. Esto permite aislar el direccionamiento del clúster de la infraestructura física. Su principal ventaja radica en la simplicidad operativa y el desacoplamiento; a cambio, introduce cabeceras adicionales, posibles ajustes de MTU y cierta carga de CPU por encapsulación/descapsulación.
- **Enrutamiento Nativo (Directo):** Prescinde de túneles lógicos y expone las IPs de los Pods directamente a la red física mediante enrutamiento L3 convencional. Aunque mejora el rendimiento de red y reduce el consumo de CPU, demanda un diseño de direccionamiento rígido y una estrecha integración con la red corporativa.

Estudios previos muestran diferencias significativas en el desempeño según el dataplane adoptado, condicionando la viabilidad técnica en entornos con recursos limitados como edge computing o nubes públicas [4, 5]. En contraste, la aceleración mediante planos de datos avanzados como Open vSwitch (OVS) o programas eBPF (extended Berkeley Packet Filter) incide sustancialmente en la latencia de procesamiento [19, 20].

2.3. Gestión de direcciones (IPAM)

La asignación eficiente de direccionamiento mediante IPAM (localizado en archivos locales por nodo con host-local o centralizado en la base de datos del clúster) es indispensable

para mitigar colisiones de red, facilitar despliegues dual-stack y garantizar la escalabilidad dinámica ante la creación o destrucción masiva de réplicas de Pods [15, 16].

2.4. MTU y fragmentación en redes con encapsulación

En redes superpuestas, el overhead de la encapsulación disminuye el tamaño máximo de segmento de datos útil. Se debe ajustar la Unidad Máxima de Transmisión (MTU, Maximum Transmission Unit) efectiva restando el tamaño de las cabeceras adicionales para prevenir la fragmentación de paquetes, caídas de rendimiento y pérdidas de conexión bajo carga [2], un proceso ilustrado en la Figura 5.4.

Network MTU	Calico MTU	Calico MTU with IP-in-IP (IPv4)	Calico MTU with VXLAN (IPv4)	Calico MTU with VXLAN (IPv6)	Calico MTU with WireGuard (IPv4)	Calico MTU with WireGuard (IPv6)
1500	1500	1480	1450	1430	1440	1420
9000	9000	8980	8950	8930	8940	8920
1500 (AKS)	1500	1480	1450	1430	1340	1320
1460 (GCE)	1460	1440	1410	1390	1400	1380
9001 (AWS Jumbo)	9001	8981	8951	8931	8941	8921
1450 (OpenStack VXLAN)	1450	1430	1400	1380	1390	1370

Figura 5.4: Ajuste de MTU en presencia de encapsulación (p. ej., VXLAN) para evitar fragmentación y pérdida de rendimiento [2].

2.5. Observabilidad del plano de datos

La operación confiable de microservicios requiere visibilidad sobre los flujos de red. Soluciones de monitoreo de flujos integradas en el CNI permiten agregar telemetría detallada de origen y destino correlacionándola directamente con las identidades nativas del clúster (Namespaces y etiquetas) mediante estándares como IP Flow Information Export (IPFIX) [21, 22].

2.6. Métricas de rendimiento y sobredimensionamiento

La comparación entre CNIs descansa en tres métricas principales: la latencia media y de percentiles críticos (P95 y P99), el throughput TCP/UDP sostenido y el uso de recursos (CPU y RAM) por paquete procesado [4, 5]. Estas mediciones son determinantes para mitigar el sobredimensionamiento preventivo de hardware en la nube, optimizando la relación costo-rendimiento de la infraestructura [2, 4, 5, 12, 13, 22].

3. eBPf como mecanismo de aceleración en el kernel

El filtro de paquetes de Berkeley extendido (eBPf, *extended Berkeley Packet Filter*) es una tecnología del kernel de Linux que permite ejecutar programas en espacio privilegiado de forma segura, sin modificar el código fuente del kernel ni cargar módulos adicionales [23]. Originalmente concebido para el filtrado eficiente de paquetes de red, evolucionó en un entorno de ejecución de propósito general que hoy sustenta funcionalidades de red, seguridad y observabilidad dentro del sistema operativo.

Para entender por qué eBPf importa en este contexto, conviene compararlo con iptables, el mecanismo tradicional de filtrado de paquetes en Linux. iptables funciona como una lista de reglas que el kernel revisa una por una cada vez que llega un paquete: cuantas más reglas haya —producto de más servicios y más políticas de red—, mayor es el tiempo que tarda cada decisión de enrutamiento. eBPf reemplaza ese mecanismo por tablas hash donde la respuesta se obtiene en tiempo constante, sin importar cuántas reglas existan. En un clúster con cientos de servicios y políticas de seguridad, esta diferencia se traduce en latencias medibles.

Técnicamente, un programa eBPf se compila a un bytecode que el kernel verifica antes de ejecutarlo, comprobando que no acceda a memoria no autorizada ni genere bucles infinitos. Una vez aprobado, se ejecuta directamente en el kernel mediante un compilador JIT (*just-in-time*), sin necesidad de copiar datos al espacio de usuario [23]. Esta arquitectura elimina dos fuentes principales de latencia en el procesamiento de red tradicional: las copias de memoria entre kernel y espacio de usuario, y el recorrido lineal de las cadenas de reglas de iptables.

En el contexto de los CNI evaluados en este trabajo, Cilium utiliza eBPf tanto para el reenvío de paquetes entre Pods como para la aplicación de NetworkPolicies, incluidas reglas de capa 7 sobre el protocolo HTTP. Calico también ofrece un modo eBPf opcional como alternativa a su dataplane basado en iptables [24].

4. Arquitectura de Confianza Cero y control de red en Kubernetes

La Arquitectura de Confianza Cero (Zero Trust Architecture, ZTA) es un modelo de seguridad que parte de una premisa opuesta al perímetro de red tradicional: ninguna carga de trabajo es confiable por defecto, independientemente de si se encuentra dentro o fuera de la red corporativa. Rose et al. [12] formalizaron este paradigma en el NIST SP 800-207, definiendo tres principios operativos fundamentales: verificar siempre la identidad del solicitante, aplicar el menor privilegio posible en cada acceso, y asumir que la red puede estar comprometida en cualquier momento.

Una forma de imaginar Zero Trust es pensar en un edificio donde todas las puertas están cerradas con llave por defecto. Para que alguien entre a una oficina no basta con que esté dentro del edificio; necesita una llave específica para esa puerta. En Kubernetes, cada llave es una regla de NetworkPolicy.

Llevado al entorno de Kubernetes, Zero Trust se operacionaliza mediante NetworkPolicies: reglas declarativas que el CNI aplica en el plano de datos de cada nodo. El punto de partida es una política de *Default Deny* —denegar todo el tráfico por defecto— sobre la cual se habilitan únicamente las comunicaciones explícitamente necesarias. Esta estrategia reduce la superficie de ataque porque, ante el compromiso de un Pod, el atacante no puede alcanzar otros servicios del clúster sin una regla que lo permita [12, 13].

Un aspecto crítico, y directamente relevante para este trabajo, es que la NetworkPolicy es un objeto de la API de Kubernetes, pero su aplicación efectiva depende del CNI. Si el plugin no implementa las políticas en su plano de datos, el objeto existe en la base de datos del clúster y el operador puede crearlo activo, pero no produce ningún efecto real sobre el tráfico [11]. Esta distinción entre la declaración de una política y su *enforcement* efectivo es una de las variables centrales del experimento.

5. Análisis de Decisión Multicriterio (MCDA)

El Análisis de Decisión Multicriterio (MCDA) es un conjunto de métodos formales para seleccionar entre alternativas cuando existen múltiples criterios de evaluación con distinta importancia relativa [25]. Es el enfoque adecuado cuando ninguna alternativa es la mejor en todas las dimensiones al mismo tiempo: en este trabajo, ningún CNI lidera simultáneamente en throughput, latencia, consumo de CPU, consumo de RAM y capacidades de seguridad. Ignorar ese compromiso y elegir únicamente por throughput equivale a comprar un automóvil mirando solo la velocidad máxima.

El modelo implementado en este trabajo utiliza el método de **suma ponderada** —conocido en la literatura como *Simple Additive Weighting* (SAW) o *Weighted Sum Model* (WSM)—, formalizado por Fishburn como agregación aditiva de utilidades [26] y reconocido como el método MCDA más difundido por su transparencia interpretativa [25]. La puntuación de cada alternativa se obtiene mediante la [ecuación \(8.4\)](#):

$$P_{CNI} = \sum_{i=1}^n (V_i \times W_i) \quad (5.1)$$

donde P_{CNI} es el puntaje final del plugin CNI, V_i es el valor normalizado de la métrica i y W_i es el peso asignado a esa métrica según el perfil de usuario evaluado, con la condición $\sum_{i=1}^n W_i = 1$.

Para que métricas con unidades distintas (Gbps, milisegundos, millicores) sean comparables en esa fórmula, se aplica **normalización min-max**, una de las técnicas estándar de normalización en MCDA por preservar las distancias relativas entre alternativas [27]. En este trabajo se re-escala al intervalo [1, 5] mediante la [ecuación \(8.1\)](#):

$$V_i = 1 + 4 \cdot \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (5.2)$$

donde $x_{\min}$ y $x_{\max}$ son los valores mínimo y máximo observados entre los CNIs para esa métrica. En métricas donde un valor mayor es mejor (throughput), la normalización es

directa; en métricas donde un valor menor es mejor (latencia, CPU, RAM), se invierte el cociente —usando $x_{\text{máx}} - x$ en el numerador— de modo que $V_i = 5$ representa siempre la mejor alternativa y $V_i = 1$ la peor. La elección del intervalo $[1, 5]$ —en lugar del clásico $[0, 1]$ — responde a una escala de utilidad acotada de cinco niveles, equivalente a una escala Likert, que facilita la lectura por parte de equipos no especializados sin alterar el ordenamiento de las alternativas [27]. Con $\sum_{i=1}^n W_i = 1$, el puntaje P_{CNI} también queda en $[1, 5]$, lo que permite interpretar el resultado final en la misma escala.

Los pesos W_i son los parámetros que capturan las prioridades de cada organización. En este trabajo se definen tres perfiles: Fintech, que asigna mayor peso a la seguridad; Streaming, orientado al rendimiento de red; e IoT, centrado en el menor consumo de recursos por nodo. La parametrización explícita de los pesos hace que el modelo sea ajustable ante nuevos perfiles sin modificar el procedimiento de medición.

El método SAW es *compensatorio*: un valor alto en una métrica puede compensar un valor bajo en otra. Para impedir que esa compensación opere sobre la seguridad —de modo que un CNI sin soporte de NetworkPolicies obtenga un buen puntaje únicamente por su eficiencia computacional—, el modelo incorpora una **restricción no compensatoria**, en la línea de los modelos conjuntivos de decisión multiatributo [28]. Cuando el perfil exige un nivel de seguridad alto, todo CNI que no implemente NetworkPolicies en su plano de datos recibe una penalización multiplicativa sobre su puntaje final, lo que lo descarta como opción viable con independencia de su rendimiento. La formulación y el valor de este factor de penalización se detallan en la metodología (capítulo 8).

Capítulo 6

Estado del Arte

El diseño y la optimización del plano de datos en la orquestación de contenedores constituye un área de activa investigación. En este capítulo se analiza y contrasta la literatura científica y técnica reciente referente a la evaluación de complementos de red en Kubernetes, la aplicación de políticas de seguridad bajo esquemas de confianza cero y el uso de modelos de decisión para la selección de infraestructura.

1. Evaluación Comparativa de Rendimiento de CNIs en la Literatura

La evaluación experimental de complementos CNI representa un campo recurrente debido al impacto directo de la red en la eficiencia de las aplicaciones. Diversos autores han examinado el comportamiento de complementos populares como Calico, Cilium, Flannel y Antrea bajo distintos escenarios.

Un estudio pionero desarrollado por Kang et al. [4] evaluó el rendimiento de múltiples plugins CNI en entornos de edge computing con recursos limitados. Los autores analizaron variables como el throughput TCP, la latencia de red y el uso de CPU, concluyendo que Flannel destaca por su baja sobrecarga en escenarios con limitaciones de cómputo. Sin embargo, esta investigación no contempló el impacto del cifrado de red o de las políticas de seguridad en la degradación de las métricas.

En contraste, Koukis et al. [5] extendieron la comparación de rendimiento a entornos multi-nodo utilizando herramientas de benchmarking automatizadas. Sus resultados mostraron que los plugins basados en enrutamiento nativo superan a las soluciones superpuestas (overlay) en throughput. No obstante, reconocen que el aislamiento lógico que proveen los overlays simplifica la operación en nubes públicas, lo que establece un compromiso entre rendimiento y simplicidad operativa.

Complementariamente, Ducastel [29] publicó una comparativa de rendimiento sobre redes de 40 Gbps en servidores físicos (*bare-metal*), confirmando que los plugins con enrutamiento nativo obtienen ventajas sustanciales sobre los overlays VXLAN en entornos de alta velocidad. Sus condiciones experimentales difieren del escenario adoptado en este

trabajo —evaluación en nube pública con red VPC—, lo que refuerza la conclusión de que el desempeño relativo de cada CNI depende críticamente de la infraestructura subyacente.

Por su parte, el trabajo de Pfaff et al. [20] sobre Open vSwitch (OVS) demostró que las arquitecturas de conmutación basadas en software de alto rendimiento disminuyen la sobrecarga de procesamiento en comparación con soluciones basadas en el filtrado tradicional del kernel. Esta fundamentación justifica la arquitectura adoptada por Antrea, que utiliza OVS para optimizar la conmutación interna del clúster. Tu et al. [19] revisitaron el dataplane de OVS diez años después de su diseño original, confirmando que su arquitectura basada en software sigue siendo competitiva frente a implementaciones más recientes, con la ventaja de una madurez operativa probada.

Adicionalmente, investigaciones recientes sobre eBPF (extended Berkeley Packet Filter) en el kernel de Linux, como las presentadas por Zhang et al. [23], evidencian que compilar filtros y políticas directamente en instrucciones del kernel mitiga drásticamente el uso de CPU. A diferencia de iptables, cuya complejidad de búsqueda es lineal respecto al número de reglas, el uso de mapas hash en eBPF mantiene un tiempo de procesamiento constante. Esto confiere a Cilium y Calico (en su modo eBPF) una ventaja competitiva significativa en clústeres de gran escala. El informe de FH Joanneum [24] analiza específicamente el despliegue de Cilium desde la perspectiva de seguridad, documentando cómo las capacidades eBPF permiten implementar políticas de red de capa 7 sin necesidad de un proxy adicional, un diferenciador que los estudios de rendimiento puro suelen ignorar.

La literatura reporta una divergencia clara en los resultados de rendimiento según el dataplane empleado, donde eBPF representa la tendencia predominante para clústeres de gran volumen, mientras que Flannel sigue siendo relevante en escenarios de escasos recursos. Una limitación compartida por estos estudios es que ninguno combina la evaluación de rendimiento con la de seguridad y el consumo de recursos en un mismo experimento controlado.

2. Seguridad y Políticas de Red en la Literatura Académica

El control de acceso de red en entornos nativos de la nube ha evolucionado desde perímetros estáticos hacia modelos dinámicos orientados a la identidad del servicio. La Arquitectura de Confianza Cero (Zero Trust Architecture, ZTA), formalizada por Rose et al. [12] en el NIST SP 800-207, proporciona el marco conceptual bajo el que se evalúan las políticas de red de este trabajo; sus principios se desarrollan en el Marco Teórico de este documento.

La implementación de estas políticas distribuidas introduce un overhead que los autores clásicos suelen omitir. Chandramouli et al. [14] analizaron la seguridad en microservicios basada en arquitecturas de malla de servicios (Service Mesh). Aunque confirmaron que el uso de mTLS y proxies L7 garantiza un control exhaustivo, señalaron que la introducción de proxies en espacio de usuario conlleva un costo en latencia y consumo de CPU, limitación

frecuente para organizaciones que buscan proteger su tráfico sin sobredimensionar sus nodos. Kotenko et al. [30] extienden este análisis identificando las mejores prácticas para asegurar arquitecturas de microservicios, señalando que la granularidad del control de acceso y el rendimiento son objetivos frecuentemente contrapuestos en la operación real; su trabajo resalta que la elección del mecanismo de enforcement —proxy, iptables o eBPF— determina en qué medida ese compromiso puede reducirse.

En el ámbito del monitoreo de red, Gomes et al. [21] realizaron un mapeo sistemático de la observabilidad en microservicios. Los autores identificaron que la exportación de telemetría de flujos correlacionada con namespaces y etiquetas lógicas es fundamental para la auditoría de seguridad. No obstante, alertaron que el volumen de datos recolectados mediante protocolos como IPFIX genera una sobrecarga de almacenamiento y procesamiento que debe ser evaluada empíricamente [22].

Las investigaciones previas coinciden en que la seguridad Zero Trust es necesaria, pero su coste en rendimiento y consumo de recursos sigue siendo una variable crítica insuficientemente cuantificada en la literatura actual [12, 14].

3. Modelos de Decisión Multicriterio Aplicados a Infraestructura Cloud

La selección de componentes de infraestructura en la nube implica decisiones con múltiples criterios contrapuestos que los enfoques monocriterio —elegir el CNI con mayor throughput sin considerar seguridad ni consumo de recursos— no pueden resolver adecuadamente. Los modelos de Análisis de Decisión Multicriterio (MCDA) constituyen un marco formal consolidado para este tipo de problemas [25]. En el ámbito específico de los CNIs, el estudio de Dakić et al. [31] evalúa el rendimiento y la latencia de múltiples plugins bajo perfiles ajustados de hardware y concluye que ciertos CNIs se adaptan mejor a casos de uso específicos; sin embargo, no formaliza esa adecuación en un modelo de decisión cuantitativo. De manera similar, otros estudios comparativos sobre arquitecturas de red en Kubernetes [32, 33] evalúan alternativas de forma cualitativa —describiendo ventajas y desventajas de cada opción—, sin llegar a formalizar un modelo cuantitativo que integre simultáneamente métricas de QoS, consumo de recursos y capacidades de seguridad en un único puntaje.

A pesar de la madurez de los modelos MCDA en ingeniería de sistemas, su aplicación al ecosistema de complementos de red de Kubernetes es escasa. La mayoría de las decisiones de selección de CNI en la industria se basan en recomendaciones empíricas o heurísticas simples, ignorando el compromiso que existe entre rendimiento, seguridad y huella de recursos del clúster [4, 5].

3.1. Brecha de Investigación

Al integrar las tres vertientes de la literatura examinadas, se evidencia una brecha de investigación. A pesar de los valiosos estudios comparativos de rendimiento de CNI [4,5] y del análisis de la seguridad Zero Trust [12,14], la literatura actual carece de experimentos que midan de forma simultánea el rendimiento de red (QoS), el consumo de recursos del agente CNI y el coste derivado de la aplicación de políticas de seguridad automatizadas en nubes públicas.

Esta carencia de datos integrales representa una limitación metodológica crítica que obliga a los equipos de ingeniería a tomar decisiones de infraestructura subóptimas o a sobredimensionar preventivamente los clústeres. Por consiguiente, esta tesis cubre dicho vacío al proveer un modelo de evaluación cuantitativo y un marco de decisión multicriterio estructurado para la selección informada de complementos CNI bajo políticas Zero Trust.

Capítulo 7

Metodología del Proyecto

El proyecto se desarrolló bajo el marco de trabajo **Scrum**, una metodología ágil que organiza el trabajo en ciclos cortos e iterativos llamados *sprints*. La elección de Scrum responde a la naturaleza del problema: al comparar cuatro plugins CNI bajo condiciones de carga, no se conoce de antemano qué métrica resultará determinante, por lo que el equipo necesita medir, analizar y reorientar el plan de forma continua. El ciclo natural de Scrum —*prueba* → *análisis* → *mejora*— se ajusta a esta incertidumbre técnica: cada sprint entrega un incremento funcional (un *script*, un *benchmark* o un panel comparativo) y cierra con una evaluación de datos reales que sirve de base para planificar el siguiente.

1. Marco de trabajo Scrum

Scrum estructura el trabajo en torno a tres roles, un conjunto de ceremonias y artefactos ligeros. En este proyecto los roles se asignaron de la siguiente forma:

- **Product Owner:** el director del trabajo de grado, con perfil académico, responsable de priorizar el *Product Backlog* y validar que cada incremento aporte valor al objetivo del proyecto.
- **Scrum Master:** los tutores técnicos, encargados de facilitar el proceso, remover impedimentos y velar por la aplicación de buenas prácticas.
- **Equipo de Desarrollo:** los dos autores, responsables de construir el producto —infraestructura, *benchmarks*, modelo de decisión y prototipo.

Dado el tamaño reducido del equipo, las ceremonias se mantuvieron breves: una reunión diaria de sincronización, una planificación al inicio de cada sprint, una revisión del incremento al final y una *retrospectiva* para registrar aprendizajes. Esta cadencia permitió mantener informados a los asesores sin sobrecargar al equipo con documentación innecesaria.

2. Product Backlog y trazabilidad con los objetivos

El *Product Backlog* se derivó directamente de los cuatro objetivos específicos del trabajo, de modo que cada conjunto de tareas contribuye de forma trazable a un objetivo. La [tabla 7.1](#) resume esta correspondencia.

Cuadro 7.1: Mapeo del Product Backlog con los objetivos específicos (OE).

Objetivo	Líneas de trabajo del backlog
OE1	Testbed automatizado (IaC), <i>baselines</i> de rendimiento de los cuatro CNI, rightsizing de recursos.
OE2	Biblioteca de Network Policies, validación automatizada, observabilidad y alertas de red.
OE3	Matriz de criterios y métricas, normalización, ponderaciones y modelo MCDA.
OE4	Guía metodológica, prototipo recomendador (SPA) y empaquetado de entregables.

3. Planificación de Sprints (Sprint Planning)

El proyecto se planificó en **17 sprints** (Sprint 0 a Sprint 16) de dos semanas cada uno, entre el 1.º de julio de 2025 y el 19 de febrero de 2026, según el cronograma del anteproyecto [3]. La [tabla 7.2](#) presenta el *Sprint Planning* maestro, donde cada sprint se asocia a su entregable principal y al objetivo que atiende.

Cuadro 7.2: Planificación maestra de sprints y su relación con los objetivos específicos.

Sprint	Foco / Entregable	Fechas	OE
0	Kickoff, entorno y tablero	01–10/07/2025	—
1	Kit de laboratorio v0.1	11–24/07/2025	OE1
2	CNI #1 <i>baseline</i>	25/07–07/08/2025	OE1
3	Biblioteca de Network Policies v0.1	08–21/08/2025	OE2
4	CNI #2 <i>baseline</i>	22/08–04/09/2025	OE1
5	Matriz de criterios v0.5 + checklist v0.1	05–18/09/2025	OE3
6	Validación automatizada de Network Policies	19/09–02/10/2025	OE2
7	CNI #3 <i>baseline</i>	03–16/10/2025	OE1
8	Guía metodológica v0.6	17–30/10/2025	OE3/OE4
9	Observabilidad y alertas de red	31/10–13/11/2025	OE1/OE2
10	CNI #4 <i>baseline</i> o variante	14–27/11/2025	OE1
11	<i>Rightsizing</i> v0.1	28/11–11/12/2025	OE1
12	Biblioteca de NP v1.0 + <i>snippets</i>	12–25/12/2025	OE2
13	Matriz v1.0 + checklist v1.0	26/12/2025–08/01/2026	OE3
14	Guía metodológica v1.0 + empaquetado	09–22/01/2026	OE4
15	Integración final y reporte	23/01–05/02/2026	—
16	Pulido final y entrega v1.0	06–19/02/2026	—

El detalle del desarrollo iterativo, agrupado por fases temáticas y con las retrospectivas de cada agrupación, se documenta en el [capítulo 10](#). El diseño técnico que materializa estos sprints se presenta en los [capítulos 8](#) and [9](#).

Capítulo 8

Diseño de la Solución

A partir de la planificación Scrum ([capítulo 7](#)), este capítulo presenta el diseño de la solución telemática. El diseño responde directamente a las causas del árbol de problemas y se estructura en cuatro pilares: una arquitectura de red controlada, un sistema de medición de Calidad de Servicio (QoS), un modelo de decisión multicriterio (MCDA) y una estrategia de seguridad automatizada bajo el principio de Zero Trust. El detalle de implementación y despliegue de cada componente se documenta en el [capítulo 9](#).

El diseño responde directamente a las causas identificadas en el árbol de problemas: la ausencia de criterios objetivos para seleccionar el componente de red (CNI), la falta de visibilidad sobre el tráfico interno del clúster y el sobredimensionamiento de recursos que resulta de operar sin datos reales [3]. La solución se estructura en cuatro pilares: una arquitectura de red controlada, un sistema de medición de Calidad de Servicio (QoS), un modelo de decisión multicriterio (MCDA) y una estrategia de seguridad automatizada.

1. Arquitectura de Red y Topología

El entorno de pruebas se diseñó para que el CNI sea la única variable entre ciclos de medición; el hardware, el sistema operativo, la topología y la duración de las pruebas permanecen constantes en todos los escenarios.

1.1. Red underlay

La red underlay constituye la infraestructura física sobre la que opera el clúster. Se optó por la nube pública de DigitalOcean en lugar de servidores físicos propios, ya que la homogeneidad del entorno elimina el ruido de hardware local —variabilidad de NIC, temperatura, interferencias— y garantiza que cualquier diferencia medida en throughput, latencia o CPU sea atribuible al CNI y no al sustrato. El aprovisionamiento se realizó mediante Terraform, herramienta de Infraestructura como Código (IaC): la configuración completa de los servidores queda descrita en archivos versionados, lo que permite reproducir el entorno de forma idéntica. Esta reproducibilidad es una condición necesaria para

la validez del experimento, pues sin ella la comparación entre CNIs pierde valor probatorio [4].

Se utilizó K3s —distribución liviana de Kubernetes— con un nodo Master, datastore PostgreSQL gestionado por DigitalOcean y dos nodos Worker dedicados. El menor *foot-print* de K3s frente a Kubernetes estándar deja una mayor proporción de CPU y RAM disponible para el CNI, lo que aumenta la sensibilidad de las mediciones; el datastore externo elimina el punto único de fallo del plano de control sin la complejidad operativa de un clúster etcd. La topología incluye una red privada VPC para el tráfico interno entre nodos, de manera que la latencia medida dependa exclusivamente de la eficiencia del CNI y no de la variabilidad de Internet público [5].

1.2. Red overlay y CNI evaluados

La red overlay, implementada por el plugin CNI, provee la conectividad entre cargas de trabajo distribuidas en distintos nodos. Kubernetes no incluye esta funcionalidad por defecto y delega la elección del CNI en cada organización [11, 15]; esa libertad de elección, ejercida sin criterios objetivos basados en mediciones, es el problema central que aborda este trabajo.

El núcleo del diseño es la comparativa de cuatro CNI con filosofías de implementación distintas (Tabla 8.1).

Cuadro 8.1: CNI evaluados: tecnología base, versión desplegada y enfoque.

CNI	Tecnología Base	Versión	Enfoque
Flannel	VXLAN (L2)	nativo K3s	Overlay sencillo de instalar; sin soporte nativo de NetworkPolicies.
Calico	VXLAN (DO VPC)	3.29.1 Tigera Operator	Tigera Operator con encapsulation: VXLAN ; políticas avanzadas L3/L4.
Cilium	eBPF + VXLAN tunnel	1.16.5	routingMode=tunnel , tunnelProtocol=vxlan ; opera en el kernel con eBPF y filtrado L7 HTTP nativo.
Antrea	OVS (SDN)	2.2.0	Open vSwitch con sistema jerárquico de Tiers (Emergency > SecurityOps > NetworkOps > Platform > Application).

Las tecnologías base de estos CNI sustentan dataplanes diferentes. **VXLAN** crea redes virtuales sobre redes físicas existentes encapsulando los paquetes originales dentro de paquetes UDP, lo que permite transportar tráfico de capa 2 [34] sobre redes de capa 3; el costo es un *overhead* de 50 bytes por paquete, que puede reducir el throughput efectivo en transferencias de paquetes pequeños [2]. **eBPF** permite ejecutar programas de forma segura dentro del kernel de Linux; aplicado a redes, posibilita que Cilium intercepte y redirija paquetes sin copiarlos al espacio de usuario, eliminando una fuente principal de latencia [23]. **Open vSwitch (OVS)** es un switch virtual orientado a SDN [20] que Antrea

emplea para implementar conectividad y políticas mediante un sistema de Tiers jerárquico, capaz de expresar reglas corporativas con mayor prioridad que las NetworkPolicies estándar de Kubernetes.

La selección de estos cuatro CNI no es arbitraria: cada uno aporta un punto de comparación específico y un riesgo característico a medir. Flannel actúa como línea base de conectividad sencilla, con el riesgo de carecer de *enforcement* nativo de NetworkPolicies. Calico aporta madurez operativa y políticas de red en producción, con un posible costo por la evaluación de reglas. Cilium representa el uso de eBPF como mecanismo moderno de aceleración, visibilidad y políticas L7, a cambio de mayor complejidad conceptual y mayor consumo de RAM. Antrea aporta una aproximación basada en OVS y conceptos de SDN, con dependencia de componentes adicionales. En conjunto, cubren el espectro de opciones que enfrenta un equipo de operaciones real, desde la simplicidad operativa hasta las capacidades avanzadas de observabilidad y seguridad.

2. Sistema de Medición de Calidad de Servicio (QoS)

La Calidad de Servicio (QoS) cuantifica el comportamiento real de la red desde la perspectiva de las aplicaciones. El sistema de medición sigue los principios de la metrología de redes —control de variables, reproducibilidad y representatividad estadística— para que las diferencias observadas sean atribuibles al CNI y no al procedimiento de medición.

2.1. Inyección de tráfico

El tráfico se genera con iperf3, herramienta de referencia para pruebas de rendimiento de redes, que opera con un modelo cliente-servidor en el que un Pod emisor envía tráfico a un Pod receptor y ambos registran estadísticas detalladas de la transferencia. Para asegurar tráfico inter-nodo, el cliente y el servidor se separan mediante `podAntiAffinity` suave (`preferredDuringSchedulingIgnoredDuringExecution`, weight 100): con dos Workers disponibles, el scheduler los ubica en nodos distintos en la práctica, y la modalidad `preferred` —en lugar de `required`— mantiene la resiliencia ante reinicios de nodo. La ejecución se automatiza mediante CronJobs de Kubernetes, con ráfagas de tráfico TCP de 300 segundos cada 30 minutos, frecuencia que captura el comportamiento de la red en distintos momentos y compone un perfil estadístico representativo [4].

2.2. Variables telemáticas capturadas

El sistema de medición parte del principio de que una métrica aislada no describe la calidad de una red. La Tabla 8.2 resume las variables capturadas, que combinan rendimiento de red, estabilidad y costo computacional; la latencia se registra en sus valores mínimo, promedio, máximo y de variabilidad (*jitter*), de modo que los picos no queden ocultos por el promedio.

Cuadro 8.2: Variables telemáticas capturadas por el sistema de medición.

Variable QoS	Unidad de medida	Relevancia
Throughput (ancho de banda efectivo)	Gbps / Mbps	Capacidad efectiva de transferencia entre Pods. Herramienta: iperf3 -J, 300 s TCP.
Latencia de establecimiento TCP	ms (min/avg/max/mdev)	Costo de iniciar una comunicación entre servicios. Herramienta: script Bash con <code>nc -z -w 2</code> , 30 muestras por corrida.
Retransmisiones TCP	número de paquetes	Indicador de inestabilidad en el canal de encapsulamiento del CNI.
Costo computacional del CNI	% CPU / MB RAM	Recursos que consume el propio CNI para procesar el tráfico; determina el impacto en el sobredimensionamiento.

2.3. Pila de observabilidad: Prometheus y Grafana

Prometheus recolecta métricas mediante un modelo de *scraping* periódico sobre cada nodo, registrando estadísticas de CPU, memoria, red y estado del CNI. El almacenamiento en series temporales permite correlacionar el consumo de recursos del agente CNI con los instantes exactos de las pruebas de throughput, verificando que las métricas de eficiencia reflejen el comportamiento bajo carga y no en reposo [21]. **Grafana** constituye la capa de visualización: presenta en dashboards interactivos el throughput, la latencia, el consumo de CPU del agente y las retransmisiones TCP. Esta visibilidad permitió detectar anomalías durante las sesiones —picos de CPU o reinicios inesperados del agente— antes de que comprometieran la integridad de las corridas [21].

3. Modelo de Decisión Multicriterio (MCDA)

Para resolver la selección objetiva del CNI se diseñó un modelo de Análisis de Decisión Multicriterio (MCDA), enfoque empleado en ingeniería y economía cuando una decisión involucra múltiples factores con distinta importancia [25]. A diferencia de una comparación basada en un único criterio —por ejemplo, el throughput máximo—, el MCDA captura el *trade-off* entre velocidad, eficiencia de recursos y capacidades de seguridad, y produce una recomendación que refleja las prioridades de cada tipo de organización.

El reto principal de elegir un CNI es que no existe una única solución óptima en todas las dimensiones simultáneamente. El rendimiento base de velocidad de red (*throughput*) es solo una dimensión del problema. Por ejemplo, una pasarela de transacciones bancarias exige seguridad estricta y aislamiento lógico, mientras que un clúster IoT en nodos con recursos de cómputo limitados prioriza un bajo consumo de memoria RAM y CPU. La Tabla E.1 muestra a nivel conceptual este compromiso de rendimiento (*trade-off*) cualitativo entre los plugins evaluados, sirviendo de introducción visual al dilema de selección.

Cuadro 8.3: Matriz comparativa cualitativa de compromisos (trade-offs) en CNIs.

Dimensión	Flannel	Calico	Cilium	Antrea
Latencia y jitter	Medio	Excelente	Bueno	Aceptable
Throughput	Bueno	Excelente	Aceptable	Bueno
Consumo de CPU	Excelente	Deficiente	Aceptable	Aceptable
Consumo de RAM	Excelente	Deficiente	Aceptable	Aceptable
Network Policies L4	Nulo (No soporta)	Excelente	Excelente	Excelente
Network Policies L7	Nulo (No soporta)	Nulo (No soporta)	Excelente	Nulo (No soporta)

3.1. Definición de criterios y métricas de evaluación

Para comparar los plugins de manera objetiva, el modelo agrupa los indicadores en tres criterios principales (C1, C2 y C3), resumidos en la Tabla 8.4.

Cuadro 8.4: Estructura de criterios, preguntas de control y métricas del modelo de selección.

Criterio Principal	Pregunta de Control (UI)	Métricas y Variables Asociadas
C1: Rendimiento de Red (QoS)	¿Qué tan rápido tiene que responder?	Latencia base promedio (ms), Latencia máxima (ms)
	¿Qué tan estable debe ser la conexión?	Jitter (MDEV) (ms), Retransmisiones TCP (paquetes/sesión)
	¿Cuántos datos se van a mover al mismo tiempo?	Throughput TCP promedio (Mbps)
C2: Eficiencia de Recursos	¿Los servidores donde va a funcionar son pequeños?	Consumo de CPU promedio (% del nodo), Consumo de RAM promedio (MB)
C3: Impacto de Seguridad	¿Qué tan importante es la seguridad y el control?	Soporte nativo de Network Policies, Sobrecarga (overhead) de latencia y throughput (%)

- **C1 — Rendimiento de red (QoS):** Evalúa latencia promedio/máxima, inestabilidad (*jitter*), velocidad de transferencia (*throughput*) y retransmisiones (pérdida de paquetes).
- **C2 — Eficiencia de recursos:** Mide el consumo de CPU y memoria RAM que el agente CNI requiere en cada nodo worker para funcionar.
- **C3 — Impacto de seguridad:** Mide la sobrecarga (*overhead*) de latencia y throughput al activar Network Policies en el clúster.

3.2. Limpieza estadística con el Rango Intercuartílico (IQR)

Antes de aplicar el modelo, los datos recolectados pasan por una limpieza estadística mediante el Rango Intercuartílico (IQR), que elimina *outliers* causados por eventos externos como mantenimientos del proveedor o picos de uso compartido. El criterio de corte

emplea las *vallas* de Tukey ($Q_1 - 1,5 \cdot \text{IQR}$, $Q_3 + 1,5 \cdot \text{IQR}$), regla clásica del análisis exploratorio de datos para identificar valores atípicos [35]. Este procesamiento evita que la recomendación dependa de valores atípicos o de la magnitud numérica de una métrica, y se implementa en el script `docs/procesador.js`.

3.3. Normalización continua Min-Max

Para estandarizar las métricas crudas y permitir la suma ponderada del modelo MCDA, el algoritmo procesador aplica una función de normalización *min-max* continua que proyecta los valores a una escala uniforme de 1,00 a 5,00. A diferencia de los modelos basados en umbrales discretos escalonados, esta aproximación refleja proporcional y exactamente las brechas de rendimiento entre los plugins evaluados.

La transformación de las métricas empíricas se realiza mediante el método de escalamiento lineal Min-Max, de acuerdo con la formulación analizada por [27] y revisada en los surveys de técnicas de normalización en MCDA [36]. La Ecuación (8.1) define la transformación directa para las métricas donde un mayor valor representa un mejor desempeño (ej. *throughput*), mientras que la Ecuación (8.2) describe la transformación para aquellas dimensiones donde un menor valor es deseable (ej. latencia, retransmisiones, consumo de CPU y RAM).

$$V_{norm} = 1 + \left(\frac{V_{crudo} - Min}{Max - Min} \right) \times 4 \quad (8.1)$$

$$V_{norm} = 1 + \left(\frac{Max - V_{crudo}}{Max - Min} \right) \times 4 \quad (8.2)$$

Los resultados normalizados garantizan que el CNI con el mejor desempeño en una dimensión obtenga exactamente 5,00 y el de peor desempeño obtenga exactamente 1,00.

3.4. Factores de ponderación y restricciones no compensatorias

El modelo aplica factores de ponderación (pesos porcentuales) que suman el 100 % (1,0), asignando mayor peso a las métricas más críticas según el caso de uso. La distribución de pesos calculados dinámicamente por la herramienta para los tres perfiles base se muestra en la Tabla 8.5.

Cuadro 8.5: Matriz de factores de ponderación (pesos calculados) por perfil de la tesis.

Perfil / Caso de Uso	Latencia	Estabilidad	Throughput	Recursos	Seguridad
Fintech / Seguridad crítica	20.8 %	24.8 %	21.0 %	8.1 %	25.3 %
Streaming / Alto rendimiento	27.7 %	22.7 %	25.1 %	12.3 %	12.3 %
IoT / Recursos limitados	20.8 %	19.0 %	18.5 %	28.0 %	13.7 %

3.4.1. Restricción no compensatoria de seguridad

Para evitar que un CNI inseguro gane por pura eficiencia computacional, el modelo combina el método compensatorio SAW con una restricción conjuntiva, siguiendo los modelos no compensatorios de decisión multiatributo [28]. Si el perfil exige un nivel de seguridad *Alto* o *Muy Alto* (valor ≥ 4 en el recomendador, como ocurre en el perfil Fintech), cualquier CNI que carezca de soporte nativo para Network Policies (como Flannel) sufre una penalización. Esto reduce su puntaje general al 40 % del valor ponderado calculado:

$$P_{\text{final}} = \begin{cases} P_{\text{MCDA}} \times 0,40 & \text{si la seguridad es prioritaria y el CNI carece de políticas} \\ P_{\text{MCDA}} & \text{en caso normal} \end{cases} \quad (8.3)$$

Esta restricción matemática asegura que los plugins con falencias críticas en el plano de seguridad sean descartados automáticamente en entornos regulados, invalidando victorias engañosas logradas puramente mediante ventajas en la eficiencia de recursos computacionales. El factor $\alpha = 0,40$ (reducción del 60 %) es un umbral de diseño calibrado para que la penalización sea severa sin anular por completo el puntaje, lo que permite que el recomendador conserve el ordenamiento relativo del resto de dimensiones para fines de diagnóstico.

3.5. Modelo matemático de decisión multicriterio (MCDA)

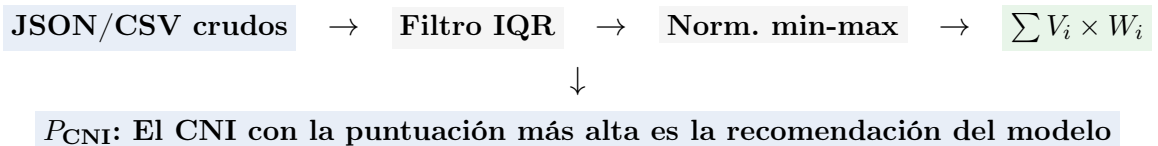
Para la recomendación formal del CNI, el algoritmo aplica el modelo MCDA de sumatoria ponderada clásica o SAW (Simple Additive Weighting), fundamentado en la teoría clásica de decisiones multicriterio [25, 37]:

$$P_{\text{CNI}} = \sum_{i=1}^n V_i \times W_i \quad (8.4)$$

Donde P_{CNI} es el puntaje del plugin para el perfil; V_i es la puntuación normalizada asignada según la métrica i ; W_i es el peso porcentual de la métrica en el perfil; y n es el número de métricas evaluadas.

La escala final de los puntajes del modelo MCDA está normalizada en la escala de 1,0 a 5,0, donde una puntuación de 5,0 representa un CNI con comportamiento ideal (cumplimiento excelente de todos los requisitos prioritarios) y 1,0 representa un cumplimiento totalmente deficiente del perfil evaluado.

El flujo de procesamiento del recomendador interactivo se resume a continuación:



4. Seguridad y Micro-segmentación (Zero Trust)

En su configuración inicial, cualquier aplicación dentro del clúster puede comunicarse con cualquier otra. Las Network Policies permiten implementar el principio de *Zero Trust*: conceder solo lo necesario y verificar explícitamente cada acceso [12, 13]. Esto reduce la superficie de ataque, ya que ante el compromiso de un Pod las políticas limitan el movimiento lateral a las rutas explícitamente autorizadas. La aplicación efectiva de estas políticas depende del CNI: si el plugin no las implementa en su plano de datos, la política existe como objeto declarativo sin producir efecto [11, 15].

4.1. Alcance operativo declarado

El enunciado original del OE2 introduce dos términos —*superficie de ataque y probabilidad de incidentes de seguridad*— que pertenecen al vocabulario de la gestión de riesgos pero que no fueron medidos en este trabajo con métricas cuantitativas estándar de la industria (recuento de CVE, técnicas MITRE ATT&CK bloqueadas, modelos formales de amenazas). Para sostener la evaluación con la evidencia disponible, se establecen las siguientes definiciones operativas, que se aplican a lo largo del manuscrito.

- **Superficie de ataque** se define operativamente como el *conjunto de flujos de red autorizados entre Pods o namespaces del clúster*. Su reducción se observa cuando el número de pares (origen, destino, puerto) permitidos por la política declarativa disminuye respecto al estado por defecto del clúster, donde toda comunicación interna está permitida. El bloqueo efectivo de un flujo no autorizado en una prueba de conectividad —por ejemplo, el rechazo del tráfico Frontend→Database en TC-SEC-04— constituye evidencia directa de esta reducción.
- **Probabilidad de incidentes de seguridad** se define como una *aproximación cualitativa* de la capacidad del CNI para bloquear los movimientos laterales y los egresos prohibidos declarativamente en los tres casos de uso ejecutados (`zero_trust`, `multi_tier` y `egress_block`). No se trata de una probabilidad estadística calculada a partir de una frecuencia base de incidentes; es una proxy observacional que permite comparar capacidades de *enforcement* entre CNIs.
- **Vulnerabilidades a reducir** se entienden como *configuraciones inseguras por defecto* —comunicación inter-Pod abierta, ausencia de *default-deny*, egress externo no controlado, ausencia de segmentación entre tiers de aplicación—, no como CVE conocidos en el código del CNI o de Kubernetes.

Esta delimitación es requerida para evaluar la *configuración* de políticas de red, no la resistencia del clúster frente a herramientas ofensivas concretas. La distinción se retoma con mayor detalle en la §4 de la Discusión.

4.2. Requisitos verificables derivados del OE2

El enunciado del OE2 se descompone en cuatro requisitos verificables (R1–R4), que imponen obligaciones contrastables con evidencia, y dos efectos esperados (E1–E2), cuya validez se sostiene a partir de los resultados de esos requisitos. La Tabla 8.6 resume la descomposición y señala, para cada elemento, la sección del manuscrito que aporta la evidencia correspondiente.

Cuadro 8.6: Descomposición del OE2 en requisitos verificables y efectos esperados.

Id	Requisito	Obligación a demostrar	Anclaje en el manuscrito
R1	Configurar de forma segura y automatizada	Políticas de red declarativas, versionadas y aplicadas sin intervención manual sobre el clúster.	§1 y TC-SEC-08 (§4.3).
R2	Adaptar las políticas a cada plugin CNI	Uso de capacidades específicas del CNI, no solo <code>NetworkPolicy</code> estándar de Kubernetes.	§1, fragmentos de configuración por CNI.
R3	Evaluar la reducción de la superficie de ataque	Procedimiento que compare el antes y el después, con y sin políticas, y entre CNIs.	§4.3 y §3.3.
R4	Determinar la capacidad de reducir vulnerabilidades	Conclusión técnica trazable a partir de la evidencia de R3.	§4.6 y Tabla 11.13.
E1	Reducir la superficie de ataque	Efecto medible operativamente según §4.1.	Evidencia: TC-SEC-04, TC-SEC-06, TC-SEC-07.
E2	Prevenir incidentes de seguridad	Efecto medible operativamente según §4.1.	Evidencia: <code>zero_trust</code> , <code>multi_tier</code> , <code>egress_block</code> .

4.3. Matriz extendida de casos de prueba TC-SEC

La Tabla 8.7 reescribe la matriz TC-SEC del cuerpo principal (Tabla 11.4) añadiendo, para cada caso, la amenaza modelada y el requisito del OE2 al que aporta evidencia. Con ello, cada afirmación del objetivo queda trazada hasta la prueba concreta que la respalda y hasta el criterio operativo con el que se verificó.

La columna “Requisito” funciona como traza directa entre cada prueba y el enunciado del OE2. Así, el efecto de *prevenir incidentes* (E2) se respalda en los casos etiquetados con E2 —TC-SEC-04, 06 y 07—, mientras que la *reducción de la superficie de ataque* (E1) se apoya en TC-SEC-01, 02, 04 y 07; el resto de requisitos se lee de forma análoga.

4.4. Estrategia de Default Deny

El diseño implementa Zero Trust en dos niveles: primero, una política de *Default Deny* que bloquea todo el tráfico entrante y saliente de cada namespace; después, una habilitación selectiva que abre únicamente las comunicaciones necesarias. Sobre esta base se definieron tres casos de uso (Tabla 8.8), ejecutados sobre cada CNI que soporta Network Policies

Cuadro 8.7: Matriz extendida TC-SEC: cada caso de prueba, amenaza modelada y requisito del OE2 al que aporta evidencia.

TC-SEC	Amenaza modelada	Verificación operativa	Requisito
01 Default Deny Ingress	Comunicación entrante no autorizada hacia el namespace.	<code>nc -zv <pod-ip>8080</code> ; éxito = timeout en < 5 s.	R3, E1
02 Default Deny Egress	Conexiones salientes no autorizadas desde Pods de producción.	<code>curl -max-time 3</code> ; éxito = timeout.	R3, E1
03 Frontend→Backend permitido	Funcionalidad legítima preservada bajo política activa.	<code>nc -zv backend 8080</code> ; éxito = <i>Connection succeeded</i> .	R3
04 Bloqueo Frontend→Database	Movimiento lateral directo desde la capa de presentación a la capa de datos.	<code>nc -zv database 5432</code> ; éxito = <i>Connection refused</i> o timeout.	R3, E1, E2
05 Backend→Database permitido	Funcionalidad legítima preservada bajo política activa.	<code>nc -zv database 5432</code> ; éxito = <i>Connection succeeded</i> .	R3
06 Database no inicia conexiones salientes	Exfiltración o comportamiento anómalo desde la capa de datos.	Egress denegado por política; éxito = bloqueo confirmado.	R3, E2
07 Bloqueo entre namespaces	Movimiento lateral cruzando límites de namespace.	Pod de staging no alcanza Pods de production ; éxito = bloqueo confirmado.	R3, E1, E2
08 Auto-restauración ArgoCD	Persistencia del estado seguro frente a desviaciones operativas.	Borrado manual; éxito = restauración automática en < 2 min.	R1

—Calico, Cilium y Antrea—; Flannel se incluye como referencia negativa para confirmar la ausencia de *enforcement* en su plano de datos.

Cuadro 8.8: Casos de uso de seguridad implementados por CNI.

Caso	Descripción
zero_trust	Default Deny: bloquea todo ingress y egress por defecto; permite solo el tráfico benchmark explícito.
multi_tier	Reglas explícitas frontend → backend → database mediante label selectors. Cilium agrega CiliumNetworkPolicy L7 HTTP (solo GET/HEAD desde frontend).
egress_block	Bloqueo de egress externo con DNS permitido; tráfico inter-pod interno habilitado.

Dos CNI aportan diferenciadores propios. Cilium implementa filtrado HTTP nativo por método mediante CiliumNetworkPolicy L7 (GET y HEAD permitidos de frontend a backend; POST, PUT y DELETE bloqueados), directamente en eBPF y sin *service mesh*. Antrea, mediante ClusterNetworkPolicy en el Tier **securityops**, impone reglas con prioridad sobre cualquier NetworkPolicy de namespace, modelando políticas corporativas obligatorias —del tipo ISO 27001 o SOC 2— que los desarrolladores no pueden sobrescribir.

4.5. Automatización GitOps con ArgoCD

Para preservar la efectividad de las políticas tras cambios en el sistema, se diseñó un flujo GitOps con ArgoCD. Si una política de red se elimina del clúster, ArgoCD detecta la diferencia entre el estado actual y el definido en Git y restaura el recurso automáticamente (`selfHeal: true`); los recursos eliminados del repositorio también se eliminan del clúster (`prune: true`). Esta automatización garantiza la integridad del experimento: durante las rotaciones de CNI, las políticas se restauran a su estado esperado sin intervención manual, lo que elimina una fuente potencial de sesgo en los resultados [13].

4.6. Mapeo a marcos de referencia

La Tabla 8.9 alinea los casos de prueba TC-SEC con los marcos conceptuales que sustentan la decisión de qué controles evaluar. El mapeo es *conceptual* —no se afirma cumplimiento certificado de ningún marco— y permite contextualizar el alcance del OE2 dentro del vocabulario habitual de la industria.

Cuadro 8.9: Mapeo conceptual de los casos TC-SEC a controles de marcos de referencia.

TC-SEC	Principio Zero Trust (NIST SP 800-207)	Patrón de seguridad evaluado
01, 02	Verificación explícita por flujo; denegación por defecto.	Segmentación por namespace y <i>default deny</i> .
03, 05	Acceso de mínimo privilegio aplicado a comunicaciones legítimas.	Habilitación selectiva por etiqueta de aplicación.
04, 07	Reducción de movimiento lateral mediante microsegmentación.	Aislamiento entre tiers y entre namespaces.
06	Asumir compromiso: el componente más sensible (DB) no inicia conexiones salientes.	Egress restringido en la capa de datos.
08	Postura de seguridad mantenida continuamente, no establecida una sola vez.	Reconciliación declarativa por GitOps.

El alcance del OE2 se limita al *enforcement* de red dentro del clúster. Quedan explícitamente fuera del alcance: el *admission control* (validación previa al despliegue de las políticas con herramientas como OPA/Gatekeeper o Kyverno), la detección y respuesta en *runtime* (Falco, Tetragon), el *service mesh* con mTLS para garantizar identidad criptográfica entre servicios [14], la seguridad de la cadena de suministro y la verificación de imágenes. Estas dimensiones complementarias se discuten en la §4 de la Discusión.

5. Flujo Integral del Sistema

El flujo completo de datos se organiza en cinco etapas:

1. **Generación de tráfico:** un CronJob activa iperf3-client cada 30 minutos (minutos 0 y 30); la anti-afinidad ubica cliente y servidor en nodos distintos.
2. **Captura de métricas:** iperf3 registra throughput, latencia TCP y retransmisiones (formato JSON, 300 s); un CronJob de latencia ejecuta 30 muestras con `nc -z -w 2` (minutos 10 y 40); Prometheus recolecta CPU y RAM del agente CNI.

3. **Extracción de recursos:** el CronJob del Resource Usage Exporter (minutos 25 y 55) consulta la API HTTP de Prometheus (`/api/v1/query_range`) y genera archivos raw JSON, CSV, PNG y `cni_resource_summary.json`.
4. **Procesamiento y scoring MCDA:** `procesador.js` aplica IQR, normalización min-max y cálculo de puntajes, y genera `cni-data.json`.
5. **Visualización y recomendación:** una SPA en React consume `cni-data.json` y muestra el ranking MCDA por perfil.

6. Arquitectura por Capas

La solución se organizó en siete capas para separar responsabilidades: la **capa 1** es la infraestructura underlay (DigitalOcean); la **capa 2**, el clúster Kubernetes K3s en alta disponibilidad; la **capa 3**, el plano CNI (Flannel, Calico, Cilium o Antrea); la **capa 4**, el plano de inyección de tráfico (Pods `iperf3` y CronJobs); la **capa 5**, observabilidad y captura (Prometheus, Grafana y exporter de recursos); la **capa 6**, procesamiento y decisión (`procesador.js`: IQR, normalización y MCDA); y la **capa 7**, presentación y recomendación (SPA React). Transversalmente opera la estrategia de seguridad Zero Trust —Network Policies con Default Deny, micro-segmentación y automatización GitOps con ArgoCD—, que asegura que tanto el entorno de prueba como las recomendaciones incorporen consideraciones de seguridad [30]. La Figura 8.1 ilustra esta estructura.

7. Trazabilidad y Requisitos de Diseño

El diseño no se planteó como una arquitectura aislada, sino como respuesta directa al problema formulado en el anteproyecto: las organizaciones adoptan Kubernetes como plataforma de orquestación, pero la calidad final del despliegue depende del plano de red, de la selección del CNI y del control de las comunicaciones internas del clúster [3]. La Tabla 8.10 hace explícita la correspondencia entre cada necesidad, su objetivo asociado y la decisión de diseño que la atiende.

De este planteamiento se derivan cinco requisitos de diseño. La **reproducibilidad** se obtiene mediante infraestructura como código, despliegue declarativo y automatización de experimentos. El **aislamiento de variables** mantiene constante todo elemento distinto del CNI —proveedor, tipo de instancia, versión de Kubernetes, topología y duración—. La **medición integral** evalúa el CNI por rendimiento de red, eficiencia de recursos e impacto de seguridad, y no solo por throughput. La **trazabilidad de la evidencia** exige que cada dato conserve su origen. La **compatibilidad con la operación real** se logra empleando objetos y patrones nativos de Kubernetes —Deployments, CronJobs, ArgoCD— cercanos a la práctica de los equipos de operaciones.

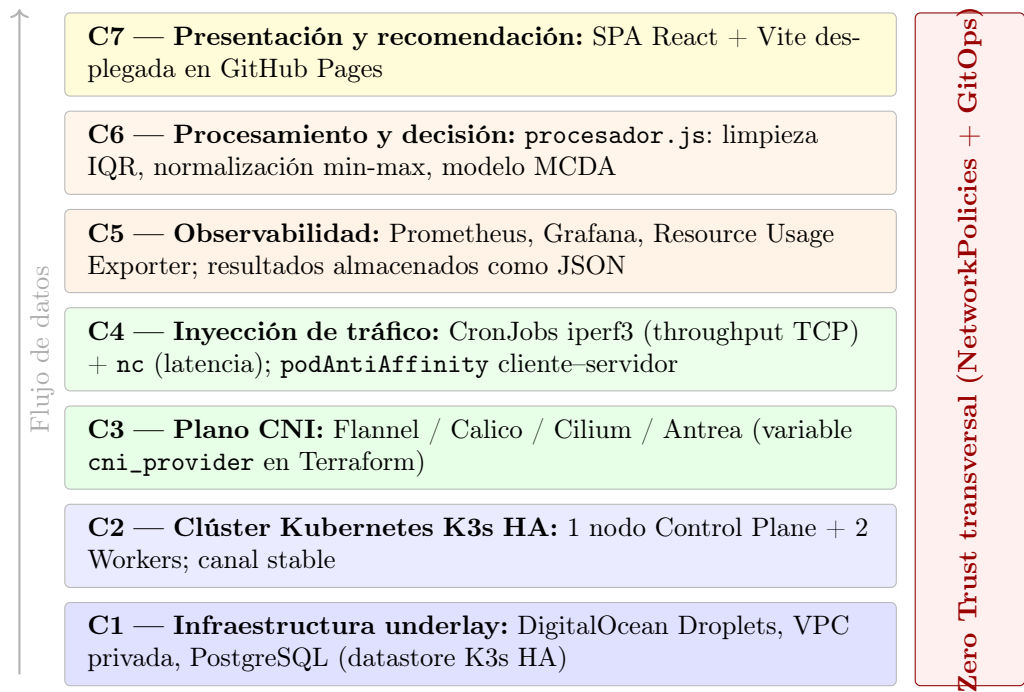


Figura 8.1: Arquitectura por capas del testbed experimental.

Cuadro 8.10: Trazabilidad entre problema de investigación, objetivo y decisión de diseño.

Necesidad identificada	Objetivo asociado	Respuesta de diseño
Comparar CNIs sin depender de recomendaciones informales.	Evaluar cuantitativamente Calico, Cilium, Flannel y Antrea en escenarios controlados.	Testbed reproducible sobre Kubernetes, ejecución automatizada y consolidación de métricas por CNI.
Controlar el riesgo de comunicaciones internas abiertas.	Evaluar políticas de red seguras y automatizadas.	NetworkPolicies bajo enfoque Zero Trust, casos de denegación por defecto y segmentación por capas.
Convertir resultados técnicos heterogéneos en una decisión comprensible.	Definir criterios, umbrales y ponderaciones para comparación objetiva.	Modelo MCDA con criterios de rendimiento, eficiencia de recursos e impacto de seguridad.
Apoyar a una organización que necesita seleccionar y configurar un CNI.	Implementar un prototipo funcional de recomendación.	Prototipo web que consume datos procesados, aplica perfiles de decisión y explica la recomendación.
Evitar sobredimensionamiento de infraestructura.	Relacionar desempeño de red con consumo de CPU y memoria.	Captura de recursos del CNI mediante Prometheus y cálculo de eficiencia relativa por perfil.

8. Prototipo Recomendador

El prototipo recomendador es la capa de interacción entre el modelo matemático y el usuario final. Su salida comprende tres niveles: la recomendación directa (qué CNI se sugiere), la justificación (qué métricas la explican) y la orientación de configuración (qué consideraciones debe atender el usuario al desplegar ese CNI), estructura que responde al objetivo específico 4 del anteproyecto [3]. Se implementó como SPA con React 19, Vite 7 y Tailwind CSS 3; el *build* de producción se despliega en GitHub Pages, y la actualización de datos encadena `procesador.js` —que genera `cni-data.json`— con la inyección de esos datos en el *bundle*.

9. Alcance y Límites del Diseño

El diseño se limita a evaluar CNIs en un testbed controlado sobre Kubernetes y no pretende generalizar automáticamente los resultados a cualquier proveedor, versión o hardware. Emplea tráfico TCP controlado como señal principal y no asume que una mayor complejidad implique un mejor resultado. La solución se concibió para sostener una mejora continua: a medida que se incorporen nuevas corridas o versiones de CNIs, el pipeline puede recalcular los puntajes. En lugar de entregar una decisión congelada, ofrece un método para decidir con evidencia actualizable: técnicamente, reemplaza opiniones por datos medidos en condiciones reproducibles; económicamente, conocer el consumo de CPU de cada CNI por unidad de tráfico permite dimensionar la infraestructura sin márgenes precautorios excesivos [3].

Capítulo 9

La Herramienta: Implementación y Despliegue

El diseño del capítulo anterior se materializa en una herramienta concreta, automatizada y reproducible: un *testbed* de evaluación desplegado como infraestructura como código, un motor de decisión multicriterio y un prototipo recomendador que expone las recomendaciones a las organizaciones. Este es el producto central del trabajo de grado. La infraestructura está codificada en Terraform, la configuración en manifiestos YAML versionados en Git y las pruebas automatizadas en CronJobs de Kubernetes —condición necesaria para que los resultados sean reproducibles y atribuibles al CNI. El manual de usuario y el manual de instalación se incluyen como anexos.

La infraestructura está codificada en Terraform, la configuración en manifiestos YAML versionados en Git y las pruebas automatizadas en CronJobs de Kubernetes —condición necesaria para que los resultados sean reproducibles y atribuibles al CNI.

1. Aprovisionamiento de Infraestructura con Terraform

El aprovisionamiento de servidores y redes se realizó mediante Terraform (IaC). Al describir la infraestructura en archivos versionados, Terraform la recrea de manera idéntica en cada ejecución y elimina la posibilidad de que variaciones manuales introduzcan diferencias entre los entornos de cada CNI.

1.1. Estructura del Código Terraform (K8s-bootstrap/00.bootstrap)

El proceso de aprovisionamiento tiene tres etapas: `terraform plan` (compara estado actual vs. código), `terraform apply` (crea Droplets, VPC y firewall) y `terraform output` (extrae IPs y kubeconfig). Terraform mantiene un archivo `terraform.tfstate` que permite destruir y recrear toda la infraestructura de manera controlada, garantizando la repetibilidad científica: cada CNI se evaluó en condiciones idénticas de arranque.

Cuadro 9.1: Archivos Terraform del módulo de aprovisionamiento y su función específica.

Archivo Terraform	Responsabilidad en la Infraestructura
<code>main.tf</code>	Define los recursos principales: Droplets de DigitalOcean y red VPC privada.
<code>variables.tf</code>	Centraliza todos los parámetros configurables: número de nodos (<code>agent_count = 2</code>), tamaño de instancias, región y proveedor CNI.
<code>ssh_keys.tf</code>	Automatiza la gestión de llaves SSH para el acceso seguro entre nodos.
<code>firewall.tf</code>	Configura las reglas de firewall a nivel de nube: primera línea de seguridad perimetral.
<code>outputs.tf</code>	Genera automáticamente el archivo kubeconfig con las credenciales del clúster K3s.

2. Configuración del Clúster K3s en Alta Disponibilidad

Una vez aprovisionados los servidores, se instaló y configuró K3s, una distribución liviana de Kubernetes, con topología de 1 nodo Master + 2 nodos Worker dedicados.

2.1. Base de Datos Externa para el Plano de Control

El plano de control se configuró con una base de datos PostgreSQL gestionada de DigitalOcean (también provisionada via Terraform) como datastore externo. Esto elimina la complejidad de mantener un clúster etcd y garantiza un plano de control sin punto único de fallo (SPOF).

2.2. Instalación: clúster sin CNI para plugins externos y modo nativo para Flannel

La implementación establece una distinción metodológica deliberada: Flannel se evalúa en su modo de despliegue más habitual —integrado nativamente en K3s con `flannel-iface: eth1`—, mientras que Calico, Cilium y Antrea se instalan como plugins externos sobre un clúster sin CNI. La asimetría refleja las condiciones reales de uso: Flannel es el CNI predefinido de K3s y la mayoría de equipos lo adopta sin configuración adicional, en tanto que los otros tres requieren instalación explícitamente y de forma declarativa. Evaluar cada CNI en su escenario de despliegue típico aumenta la validez ecológica del experimento; debe considerarse, no obstante, que Flannel se beneficia de la integración nativa de K3s —gestión automática de CIDR y *hooks* de kubelet— que los plugins externos deben replicar por su cuenta.

Para los CNIs externos, la configuración crítica es el parámetro `-flannel-backend=none`, que desactiva completamente el CNI integrado de K3s y evita interferencias con el plugin bajo evaluación. Adicionalmente, se configura `-disable-network-policy` para que K3s no intente implementar NetworkPolicies por sí mismo.

La configuración resultante en `/etc/rancher/k3s/config.yaml` para CNIs externos incluye:

```
flannel-backend: none
```

```
disable-network-policy: true
```

Cuadro 9.2: Instalación de CNIs: método y configuración clave en `user_data/ks3_server_init.sh`.

CNI	Método de instalación	Configuración clave
Flannel	Nativo K3s	<code>flannel-iface: eth1</code> (interfaz VPC de DigitalOcean)
Calico	Tigera Operator v3.29.1	<code>encapsulation: VXLAN, CIDR 10.42.0.0/16,</code> <code>nodeAddressAutodetectionV4.interface: eth1</code>
Cilium	Helm 1.16.5	<code>routingMode=tunnel, tunnelProtocol=vxlan, ipam.mode=kubernetes,</code> <code>devices=eth1, operator.replicas=1</code>
Antrea	Manifest v2.2.0	<code>transportInterface: eth1, serviceCIDR: 10.43.0.0/16</code>

El protocolo de rotación de CNIs se implementó mediante el reemplazo completo de la infraestructura entre ciclos. Para cambiar de un CNI al siguiente, se modificó la variable `cni_provider` en `terraform.tfvars` y se ejecutó `terraform destroy` seguido de `terraform apply`: el primero eliminó todos los recursos de DigitalOcean (Droplets, VPC y base de datos PostgreSQL), y el segundo los recreó desde cero con el nuevo CNI preconfigurado en el script de inicialización de K3s. Este enfoque garantiza que cada ciclo de prueba parte de nodos completamente limpios, sin residuos de configuración del CNI anterior que pudieran contaminar las métricas.

3. Orquestación y Entrega Continua: GitOps con ArgoCD

Con el clúster K3s funcionando y sin CNI, el siguiente componente instalado fue ArgoCD. ArgoCD implementa el paradigma GitOps: el repositorio Git es la única fuente de verdad sobre el estado del clúster. Todo lo que debe existir en el clúster está definido en archivos YAML dentro del repositorio `K8s-Tesis-Apps`. Esta centralización en Git facilita la auditoría del experimento: cualquier cambio en la configuración queda registrado con autor, fecha y motivo, haciendo el proceso de prueba completamente trazable.

3.1. El Patrón App of Apps

La implementación de ArgoCD utiliza el patrón App of Apps: existe UNA sola aplicación raíz (ubicada en `K8s-bootstrap/30.apps-of-apps-install`) que ArgoCD conoce directamente, y esa aplicación raíz contiene las definiciones de todas las demás aplicaciones (CNIs, stack de métricas, benchmarks, políticas de red).

3.2. Configuración de `selfHeal` y `prune`

selfHeal: true — Ante la modificación manual de un recurso del clúster —por ejemplo, la eliminación de una `NetworkPolicy`—, ArgoCD detecta la diferencia con el repositorio

Git y restaura el estado original, lo que resultó crítico para la integridad del experimento.

prune: true — Al eliminar un archivo YAML del repositorio Git, ArgoCD elimina también el recurso correspondiente del clúster, lo que evita la acumulación de recursos huérfanos.

```
# Definición de la Application raiz en ArgoCD (App of Apps)
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: apps-of-apps
  namespace: argocd
spec:
  source:
    repoURL: https://github.com/DuwanSierra/K8s-Tesis-Apps
    targetRevision: main
    path: apps/
  syncPolicy:
    automated:
      selfHeal: true # Restaura cambios manuales automaticamente
      prune: true # Elimina recursos borrados del repo
```

3.3. Overlays de Kustomize para los CNIs

Cada CNI tiene una configuración ligeramente diferente. Para gestionarlas sin duplicar código, se utilizó Kustomize con el patrón base/overlay: existe una configuración base con los parámetros comunes, y sobre esa base se aplican overlays específicas para cada CNI.

Las apps gestionadas por ArgoCD incluyen: **cni-test-iperf** (benchmarks de throughput y latencia), **cni-resource-exporter** (exportación de recursos desde Prometheus), **cni-np-test** (pruebas de Network Policies, creada solo cuando `cni_provider != "flannel"` y el caso de uso está definido), y **metrics** (Prometheus + Grafana).

4. Implementación del Stack de Observabilidad

El stack de observabilidad se implementó mediante el **kube-prometheus-stack** (Helm chart) que instala Prometheus, Grafana y exportadores preconfigurados para Kubernetes.

4.1. Prometheus: El Recolector de Métricas

Prometheus opera con un modelo pull: visita periódicamente los endpoints de métricas (cada 15 segundos en la configuración del proyecto). Para que Prometheus sepa qué métricas recolectar de los agentes CNI, se implementaron **ServiceMonitors** (CRD), apuntando específicamente al endpoint de métricas de cada agente. Un intervalo de 15 segundos proporciona suficiente granularidad para detectar picos de consumo durante las ráfagas de tráfico iperf3, sin generar una carga de scraping que interfiera con las métricas bajo observación.

Las métricas clave recolectadas del agente CNI incluyen:

- `container_cpu_usage_seconds_total` (filtrado por pod del agente): costo computacional del CNI (criterio C2 del marco MCDA).
- `container_memory_rss` (filtrado por pod del agente): memoria física real del proceso del agente CNI.
- `node_network_transmit_bytes_total / node_network_receive_bytes_total`: bytes totales de red por nodo.
- `kube_pod_info + kube_pod_status_phase`: estado del agente CNI para detectar reinicios durante las pruebas.

4.2. Grafana: Los Dashboards de Red

Grafana centraliza la lectura operativa de las series de Prometheus. Sus paneles agrupan throughput, latencia, consumo del agente CNI y score MCDA. Esta correlación vincula ráfagas iperf3 con CPU, RSS y el ranking calculado por `procesador.js`.

4.3. Resource Usage Exporter: Extracción Automatizada de Datos

El `cni-resource-usage-exporter` corre como CronJob en `K8s-Tesis-Apps/grafana-exporter`. ArgoCD sincroniza el recurso mediante la app `cni-resource-exporter`. El CronJob:

- Ejecuta la extracción en los minutos 25 y 55 de cada hora, tras cada ventana de benchmarks.
- Consulta directamente la API HTTP de Prometheus (`/api/v1/query_range`) — no interactúa con Grafana.
- Recolecta CPU % por nodo, RAM % por nodo, RAM usada, CPU cores y RAM del agente CNI.
- Genera `raw/`, `csv/`, `images/` y `cni_resource_summary.json` por corrida.
- Persiste cada salida con timestamp en `K8S-CNI-Results/results/cni-benchmarks/{cni}/resource_usage_nodes/`.

5. Implementación del Banco de Pruebas de QoS

5.1. Escenario de Rendimiento con iperf3

El banco de throughput se define en `K8s-Tesis-Apps/cni_test_iperf`:

El iperf3-server (Deployment): Un Deployment ejecuta una réplica de iperf3 en modo servidor. Un Service ClusterIP expone un nombre DNS estable para el cliente.

El iperf3-client (CronJob): La programación `0,30 * * * *` activa el cliente en los minutos 0 y 30. `concurrencyPolicy: Forbid` bloquea corridas simultáneas. Cada activación crea un Pod y genera tráfico TCP durante 300 segundos. La salida conserva formato JSON con `iperf3 -t 300 -i 10 -J`. La retención máxima queda en 5 corridas (`RETENTION_MAX_RUNS="5"`).

La anti-afinidad del cliente frente al servidor se implementó como `preferredDuringSchedulingIgnoredDuringExecution` (suave, weight 100), forzando tráfico inter-nodo en la práctica. Esta ubicación garantiza que todo el tráfico de benchmark atravesase el mecanismo de encapsulación del CNI entre nodos, que es exactamente lo que se quiere medir.

```
# iperf-client-cronjob.yaml -- fragmento relevante
schedule: '0,30 * * * *' # Minutos 0 y 30
concurrencyPolicy: Forbid
affinity:
  podAntiAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchLabels:
              app: iperf-server
          topologyKey: kubernetes.io/hostname
# iperf3 -c iperf-server-svc -t 300 -i 10 -J
```

5.2. Cliente de Latencia Personalizado

El CronJob `latency-client-cronjob.yaml` ejecuta la prueba de latencia. Su programación corre en los minutos 10 y 40 de cada hora. Esa separación evita solapamiento con el inicio del throughput.

El cliente registra TCP Connect con `nc -z -w 2` y timestamp en ms. Cada corrida toma 30 muestras y calcula min/avg/max/mdev. El JSON normalizado conserva campos de latencia, muestras intentadas y muestras exitosas.

Cuadro 9.3: Métricas de QoS capturadas por el banco de pruebas y su relevancia telemática.

Métrica Capturada	Unidad	Relevancia para QoS Telemático
Throughput TCP sostenido	Gbps / Mbps	Capacidad efectiva de la red.
Latencia mínima TCP connect	ms	Overhead mínimo del CNI sin congestión.
Latencia promedio TCP connect	ms	Comportamiento típico de solicitudes.
Latencia máxima / Jitter	ms	Peor caso y variabilidad. Crítico para tiempo real.
Retransmisiones TCP	count / sesión	Indicador de problemas en encapsulamiento o congestión.
CPU del agente CNI	millicores (m)	Impacto directo en costo de infraestructura cloud.
RAM del agente CNI (RSS)	MiB	Huella de memoria del agente CNI por nodo.

6. Procesamiento Estadístico y Aplicación del Modelo MCDA

6.1. Limpieza con el Rango Intercuartílico (IQR)

`docs/procesador.js` transforma datos crudos en datos consolidados. El script descubre los CNIs disponibles bajo `results/cni-benchmarks`. Luego extrae métricas desde JSON y CSV. Finalmente aplica IQR:

```
// procesador.js -- Funcion de limpieza estadística IQR
function calcularEstadisticasLimpias(valores) {
  const sorted = [...valores].sort((a, b) => a - b);
  const q1 = sorted[Math.floor(sorted.length * 0.25)];
  const q3 = sorted[Math.floor(sorted.length * 0.75)];
  const iqr = q3 - q1;
  const lowerBound = q1 - 1.5 * iqr;
  const upperBound = q3 + 1.5 * iqr;
  const valoresLimpios = valores.filter(
    v => v >= lowerBound && v <= upperBound);
  const promedio = valoresLimpios.reduce(
    (sum, v) => sum + v, 0) / valoresLimpios.length;
  return { promedio,
    outliersPorcentaje: ((valores.length - valoresLimpios.length)
      / valores.length * 100).toFixed(1)
  };
}
```

El pipeline descarta valores atípicos antes de calcular promedios. Dado que un evento cloud anómalo puede sesgar una corrida, el filtro estabiliza el puntaje MCDA del CNI.

6.2. Normalización y Cálculo del Score MCDA

Después de la limpieza, el pipeline aplica la normalización min-max re-escalada a [1, 5] descrita en el Marco Teórico. En métricas positivas como throughput, la normalización es directa ($V_i = 5$ al máximo). En métricas negativas como latencia o CPU, el cociente se invierte, de modo que el valor más bajo recibe $V_i = 5$ y el más alto $V_i = 1$. La escala uniforme permite comparar Gbps, milisegundos y millicores sin sesgo por magnitud numérica.

```
// recommendationModel.js -- Calculo del overhead de seguridad
function getNetworkPolicyOverhead(data) {
  if (!data?.network_policy) return null;
  const caseValues = Object.values(data.network_policy).flatMap(
    (policyCase) => [
      Number(policyCase?.overhead_latencia_pct),
      Number(policyCase?.overhead_throughput_pct),
    ]
  );
  const validValues = caseValues.filter((v) => Number.isFinite(v));
  if (!validValues.length) return null;
  return average(validValues);
}
// El score de seguridad se invierte (menor overhead = 5.0).
// Si meta.supportsNetworkPolicy === false (ej. Flannel), el score es 1.
```

El pipeline calcula el puntaje de la dimensión de seguridad evaluando el impacto (*overhead*) promedio en latencia y throughput al activar los casos de prueba de `network_policy`. Entre menor sea la degradación del rendimiento al aislar los Pods, mayor será la calificación normalizada del CNI. Aquellos plugins que por diseño no implementan Network

Policies en su plano de datos reciben por defecto la calificación mínima de 1,00 en esta dimensión, penalizando su uso en entornos que requieran aislamiento lógico.

7. Prototipo Recomendador: Aplicación Web SPA

7.1. Tecnologías de Implementación

La SPA fue implementada con React 19 (gestión de estado por componentes), Vite 7 (bundler; build de producción vía `build:pages` con `-outDir ../docs/recommender`) y Tailwind CSS 3 (estilos utilitarios directamente en JSX, sin CSS personalizado).

`npm run sync:data` ejecuta `procesador.js` y genera `cni-data.json`. Después, `export-spa-data` inyecta los datos en el bundle. `recommendationModel.js` concentra la lógica de recomendación.

7.2. Flujo de la Aplicación

1. **Selección de Perfil:** La pantalla principal muestra tres tarjetas: Fintech/Seguridad, Streaming/Rendimiento e IoT/Recursos.
2. **Visualización de Resultados:** La aplicación carga `cni-data.json` y calcula el ranking MCDA para el perfil seleccionado. El CNI con mayor puntaje queda marcado como recomendación.
3. **Justificación Técnica:** La interfaz expone las métricas brutas promedio de cada CNI en gráficas comparativas.

8. Implementación de Network Policies y Seguridad Zero Trust

8.1. Políticas Default Deny

Default Deny operacionaliza el modelo Zero Trust. La política bloquea todo el tráfico inicial y habilita solo comunicaciones necesarias. Con esta regla, toda comunicación no autorizada falla sin intervención del operador.

El siguiente manifiesto aplica la política a todos los Pods del namespace `production`:

```
# default-deny-all.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: production
spec:
  podSelector: {} # aplica a TODOS los Pods del namespace
  policyTypes:
    - Ingress # Bloquea todo el trafico entrante
```

- Egress # Bloquea todo el trafico saliente

8.2. Micro-segmentación por Capas de Aplicación

Sobre Default Deny, las políticas específicas acotan el tráfico permitido. Las reglas frontend → backend → database usan label selectors.

En Cilium, una CiliumNetworkPolicy de capa 7 admite solo métodos GET y HEAD desde frontend hacia backend. En Antrea, una ClusterNetworkPolicy en el Tier securityops prioriza sobre cualquier NetworkPolicy de namespace.

Esta estratificación evalúa seguridad granular en condiciones reales. También cuantifica si esa granularidad introduce overhead detectable, uno de los criterios del modelo MCDA.

8.3. Validación Automatizada de Políticas

Se implementó un pipeline de validación que emplea nc (netcat) dentro de Pods de prueba para verificar la conectividad. Para cada par de Pods, el pipeline define una expectativa —la conexión debe permitirse o debe bloquearse— y reporta un error cuando el comportamiento observado no coincide con ella.

Cuadro 9.4: Casos de prueba del pipeline de validación de Network Policies.

Prueba de Validación	Resultado Esperado y Justificación
frontend → backend (puerto 8080)	PERMITIDO — Comunicación necesaria para el funcionamiento de la aplicación.
frontend → database (puerto 5432)	BLOQUEADO — El frontend no debe acceder directamente a la base de datos.
backend → database (puerto 5432)	PERMITIDO — El backend necesita leer y escribir datos.
database → cualquier Pod	BLOQUEADO — La base de datos nunca debe iniciar conexiones salientes.
Pod externo → namespace production	BLOQUEADO — Sin una regla explícita, ningún Pod externo puede entrar.

9. Estructura de Repositorios y Entregables Técnicos

La Tabla 9.5 resume la organización del proyecto en cuatro repositorios Git.

La arquitectura mantiene un flujo unidireccional: HA-K3s-Terraform-D0 provee el módulo cloud; K8s-bootstrap orquesta la infraestructura; K8s-Tesis-Apps despliega las aplicaciones y genera métricas; K8S-CNI-Results almacena, procesa y visualiza resultados.

10. Desafíos de Implementación y Soluciones Adoptadas

Cuadro 9.5: Estructura de los cuatro repositorios del proyecto y su relación con los objetivos específicos.

Repositorio	Contenido Principal	Relación con los Objetivos
HA-K3s-Terraform-D0	Módulo Terraform reutilizable que provisiona Droplets, VPC, Load Balancer y base de datos PostgreSQL en DigitalOcean.	OE1: Módulo base de infraestructura cloud invocado por <code>K8s-bootstrap</code> .
K8s-bootstrap	Código Terraform (<code>00.bootstrap</code>), scripts de instalación K3s HA (<code>10.k3s-ha-do</code>), manifiestos de ArgoCD (<code>20.argo-cd-install</code>) y App of Apps (<code>30.apps-of-apps-install</code>).	OE1: Orquesta el aprovisionamiento completo del testbed.
K8s-Tesis-Apps	Overlays Kustomize para los 4 CNIs, stack de observabilidad (<code>apps/metrics</code>), benchmarks iperf3 (<code>cni_test_iperf</code>), cliente de latencia, Network Policies y Grafana Exporter.	OE1, OE2 y OE3: Implementa las pruebas, la seguridad y el sistema de recolección.
K8S-CNI-Results	Archivos JSON/CSV con resultados por corrida, <code>procesador.js</code> (modelo MCDA) y la SPA recomendadora (<code>cni-recommender-spa</code>).	OE3 y OE4: Implementa el modelo de comparación objetiva y el prototipo.

Cuadro 9.6: Desafíos técnicos encontrados durante la implementación y soluciones adoptadas.

Desafío Encontrado	Solución Técnica Implementada
MTU mismatch al instalar Cilium: Pods no podían comunicarse, paquetes se fragmentaban silenciosamente en la VPC. Flannel y Calico detectan automáticamente la MTU de la interfaz subyacente en sus versiones actuales; Cilium requirió configuración explícita porque su dataplane eBPF no hereda el valor de MTU del sistema operativo sin intervención.	Se calculó la MTU efectiva: $1500 \text{ (VPC)} - 50 \text{ (overhead VXLAN)} = 1450$. Se configuró el parámetro <code>mtu: 1450</code> en el overlay de Kustomize de Cilium.
ArgoCD en estado OutOfSync al instalar el stack de métricas (Prometheus + Grafana): los CRDs del <code>kube-prometheus-stack</code> se aplicaban en el mismo ciclo de sync que los recursos que los consumían, provocando fallo en el despliegue porque el API server aún no conocía los tipos personalizados.	Se separó la instalación en dos Applications independientes usando sync waves: <code>kube-prometheus-stack-crds</code> (wave -1) despliega el mismo chart con todos los componentes deshabilitados —solo crea los CRDs— usando <code>ServerSideApply=true</code> ; <code>kube-prometheus-stack</code> (wave 0) despliega el stack completo con <code>skipCrds: true</code> y <code>SkipDryRunOnMissingResource=true</code> , garantizando que los CRDs existen antes de crear los recursos dependientes.
Outliers frecuentes en pruebas de madrugada: DigitalOcean mostraba throttling de CPU en horas de alta demanda del datacenter.	Se configuró una ventana de pruebas exclusiva para horario laboral. El algoritmo IQR filtra automáticamente estos outliers.
<code>procesador.js</code> necesitaba evaluar la seguridad de forma cuantitativa e integrada en el MCDA, pero Flannel no reportaba overhead al no soportar políticas.	Se parametrizó el score de seguridad basándose en el overhead de latencia de las políticas. Para los CNIs que no soportan políticas (Flannel), el modelo asigna directamente el puntaje mínimo de 1,0 en la dimensión de seguridad.

11. Flujo completo de ejecución

Capítulo 10

Fase de Desarrollo por Sprints

El desarrollo se ejecutó a lo largo de los diecisiete sprints planificados ([tabla 7.2](#)). Para facilitar la lectura, este capítulo agrupa los sprints en fases temáticas y cierra cada fase con una retrospectiva que sintetiza los aprendizajes —el artefacto de mejora continua propio de Scrum—, en lugar de narrar sprint por sprint. El detalle técnico de cada componente se encuentra en los [capítulos 8 and 9](#).

1. Fase de preparación del laboratorio (Sprints 0–1)

Los primeros sprints establecieron las bases del proyecto: repositorios, integración continua, tablero de tareas y el *kit de laboratorio* v0.1. Se desplegó la pila de observabilidad (Prometheus y Grafana) y se construyeron los primeros *scripts* de inyección de tráfico (iperf3) con su validación intra e inter-nodo.

Retrospectiva. La automatización temprana del entorno como infraestructura como código resultó decisiva: garantizó que cada CNI se evaluara en condiciones idénticas. Se ajustó la frecuencia de *scraping* de métricas para capturar los picos de las ráfagas de tráfico sin introducir carga adicional.

2. Fase de evaluación de rendimiento de los CNI (Sprints 2, 4, 7 y 10)

Cada CNI —Flannel, Calico, Cilium y Antrea— se evaluó en su propio sprint de *baseline*: instalación, pruebas de throughput y latencia (percentiles p95/p99), Network Policies mínimas y un análisis corto. El protocolo de rotación reemplazaba por completo la infraestructura entre ciclos, partiendo siempre de nodos limpios.

Retrospectiva. Separar cada CNI en un sprint independiente evitó la contaminación cruzada de configuraciones. El principal impedimento —el desajuste de MTU al instalar Cilium— se resolvió calculando la MTU efectiva y se documentó como aprendizaje reutilizable para los CNI restantes.

3. Fase de seguridad y micro-segmentación (Sprints 3, 6 y 12)

Esta fase construyó la biblioteca de Network Policies bajo el enfoque Zero Trust: casos de *Default Deny*, segmentación por capas **frontend** → **backend** → **database** y reglas avanzadas (L7 en Cilium, *Tiers* en Antrea). Se implementó además el pipeline de validación automatizada que verifica, para cada par de Pods, si la conexión debe permitirse o bloquearse.

Retrospectiva. La validación automatizada confirmó que la sola declaración de una política no garantiza su *enforcement*: Flannel aceptó los objetos pero no los aplicó. Este hallazgo telemático —la diferencia entre declarar y aplicar una política— se convirtió en una de las variables centrales del modelo de decisión.

4. Fase del modelo de decisión MCDA (Sprints 5 y 13)

Se definieron los criterios y métricas de evaluación, los factores de ponderación por perfil y la restricción no compensatoria de seguridad, consolidados en la matriz de criterios v1.0 y el *checklist* de configuración. El modelo se ensayó primero con datos de prueba y luego con los datos reales del *testbed*.

Retrospectiva. Parametrizar los pesos por perfil —en lugar de fijar una única recomendación— hizo el modelo adaptable a distintas organizaciones. El ensayo con datos sintéticos antes de los reales permitió detectar y corregir el sesgo de magnitud entre métricas con unidades distintas.

5. Fase de la herramienta y la guía (Sprints 8 y 14)

Estos sprints produjeron la guía metodológica (de la v0.6 a la v1.0) y el empaquetado de los entregables, incluyendo el prototipo recomendador. Se redactaron los procedimientos paso a paso para instalar y probar cada CNI, base de los manuales de usuario e instalación incluidos como anexos.

Retrospectiva. Documentar la guía en paralelo al desarrollo —y no al final— evitó la pérdida de conocimiento por la complejidad de las configuraciones. La estructura “cómo se hace” demostró ser más útil para equipos sin especialización que una descripción puramente teórica.

6. Fase de observabilidad y *rightsizing* (Sprints 9 y 11)

Se ampliaron las métricas de red (L3–L7), los *dashboards* y las reglas de alerta, y se desarrolló el método de *rightsizing*: relacionar el consumo de CPU y memoria del agente

CNI con el throughput para dimensionar la infraestructura sin márgenes precautorios excesivos.

Retrospectiva. La correlación entre consumo de recursos y rendimiento cuantificó el costo telemático real de cada CNI, base del aporte sobre reducción de sobredimensionamiento y su impacto económico en las organizaciones.

7. Fase de integración y cierre (Sprints 15–16)

Los últimos sprints integraron todos los componentes, ejecutaron las pruebas finales y consolidaron resultados, cerrando con el control de calidad documental y de laboratorio y la entrega v1.0.

Retrospectiva. La integración incremental a lo largo del proyecto —y no en una única fase final— redujo el riesgo de incompatibilidades. El control de calidad final se concentró en la consistencia entre los datos del *testbed*, el modelo y el prototipo.

Capítulo 11

Resultados y Verificación de Objetivos

En cada ciclo de prueba se modificó únicamente el CNI bajo evaluación; hardware, sistema operativo, versión de Kubernetes, tamaño de paquetes, duración y condiciones de red permanecieron constantes, lo que hace posible atribuir las diferencias observadas al plugin y no al entorno.

1. Metodología General de Pruebas

1.1. Control de Condiciones: el Escenario de Referencia

Se definió un escenario de referencia con los siguientes parámetros fijos para todos los CNIs:

Cuadro 11.1: Parámetros del escenario de referencia para todas las pruebas.

Parámetro Controlado	Valor Fijo para Todo el Experimento
Tipo de instancia cloud	DigitalOcean Droplet — Ubuntu 22.04 LTS, 2 vCPU, 4 GB RAM para nodos Worker
Versión de Kubernetes	K3s canal stable — la misma versión en todos los ciclos de prueba
Región del datacenter	nyc3 (Nueva York) — mismo datacenter para todos los nodos en todas las pruebas
Red entre nodos	VPC privada DigitalOcean — MTU 1500 bytes, sin tráfico externo
Duración de cada prueba iperf3	300 segundos (5 minutos) por corrida
Número de corridas por CNI	5 corridas independientes — permite calcular promedios y detectar variabilidad
Protocolo de red bajo prueba	TCP — el protocolo dominante en aplicaciones empresariales reales
Herramienta de throughput	iperf3 con salida JSON (-J), un solo stream TCP (-P 1)
Herramienta de latencia	Script Bash con nc -z -w 2 — 30 muestras por corrida

1.2. Automatización: de la Prueba Manual a la Prueba Orquestada

Todas las pruebas se orquestaron mediante CronJobs de Kubernetes. La secuencia de ejecución automática en cada ciclo fue:

1. CronJob del cliente iperf3 se activa (minutos 0 y 30). Kubernetes crea el Pod cliente en un nodo diferente al servidor (podAntiAffinity suave, weight 100).
2. Pod cliente ejecuta iperf3 durante 300 segundos y guarda el resultado JSON.
3. CronJob del cliente de latencia ejecuta 30 mediciones de TCP connect (minutos 10 y 40) y guarda su resultado JSON.
4. Al finalizar ambas pruebas, el Resource Usage Exporter (minutos 25 y 55) consulta Prometheus para extraer el consumo de CPU y RAM del agente CNI.
5. `procesador.js` actualiza `cni-data.json` con los nuevos datos procesados.

La orquestación automática elimina la variabilidad humana de la ejecución: cada corrida sigue exactamente la misma secuencia con los mismos intervalos, de modo que las diferencias observadas entre CNIs se deben al plugin y no al operador.

La elección de cinco corridas por CNI responde a un equilibrio entre rigor estadístico y viabilidad operativa. Una sola corrida permitiría que un dato atípico o evento anómalo dominara el resultado, mientras que un número muy elevado extendería el tiempo de prueba sin un beneficio proporcional. Con cinco corridas distribuidas en el tiempo, el algoritmo IQR puede identificar y descartar una corrida anómala, y el promedio de las restantes resulta estadísticamente representativo; este criterio es habitual en la investigación de redes sobre entornos cloud [4,5].

2. Pruebas de Rendimiento de Red — Objetivo Específico 1

2.1. Protocolo de Prueba de Throughput con iperf3

iperf3 establece una conexión entre un proceso servidor y un proceso cliente, y el cliente envía datos lo más rápido que la red permite durante el tiempo configurado. Se usó el parámetro `-P 1` (un solo stream TCP) para representar el comportamiento de una aplicación empresarial típica, no la carga máxima teórica.

```
# Comando iperf3 ejecutado por el CronJob cliente
# -c: servidor -t: 300s -J: JSON -P 1: un solo stream
iperf3 -c iperf-server-svc.cni-benchmark.svc.cluster.local \
  -t 300 -J -P 1 \
  > /results/throughput-$(date +%Y%m%d_%H%M%S)-${CNI_NAME}.json
```

2.2. Resultados de Throughput TCP por CNI

Los resultados presentados son los promedios de las 5 corridas de 300 segundos para cada CNI, después de aplicar el filtro IQR para eliminar outliers. La Tabla 11.2 registra tres dimensiones de calidad del canal: cuántos datos por segundo puede transportar (throughput), cuántos paquetes tuvieron que reenviarse por error (retransmisiones) y qué tan predecible es el canal entre corridas (variabilidad).

Cuadro 11.2: Resultados de rendimiento TCP por CNI (promedio de 5 corridas de 300 segundos, outliers IQR eliminados).

Métrica	Flannel (VXLAN)	Calico (VXLAN)	Cilium (eBPF)	Antrea (OVS)
Throughput promedio — Mbps (↑ <i>mayor es mejor: más datos por segundo</i>)	1452.97	1804.40	843.56	1588.45
Retransmisiones TCP — paquetes/sesión (↓ <i>menor es mejor: menos retransmisiones en el canal</i>)	11774	68993	15142	42229
Outliers descartados (Throughput)	0 de 5 (0 %)	0 de 5 (0 %)	1 de 5 (20 %)	0 de 5 (0 %)
Outliers descartados (Retransmisiones)	0 de 5 (0 %)	1 de 5 (20 %)	0 de 5 (0 %)	0 de 5 (0 %)

A diferencia de los entornos locales de alto rendimiento, la red en nube VPC privada de DigitalOcean limita el ancho de banda efectivo, registrándose valores máximos de throughput de 1804.40 Mbps en Calico, seguido de Antrea con 1588.45 Mbps y Flannel con 1452.97 Mbps. Cilium presenta el valor más bajo con 843.56 Mbps. Las diferencias en retransmisiones TCP son críticas: Flannel destaca con la menor cantidad de retransmisiones (11774), seguido por Cilium (15142) y Antrea (42229), mientras que Calico registra la tasa de fallas más alta con 68993 retransmisiones. Esto indica que a pesar de que Calico lidera el throughput bruto, genera un mayor estrés de reenvío de paquetes debido a la saturación del datapath.

2.3. Protocolo de Prueba de Latencia TCP

La latencia reportada en este experimento no mide el tiempo de tránsito físico puro de la red (latencia de enlace L3/L4 intra-VPC, habitualmente inferior a 1 ms), sino el tiempo total de establecimiento de conexión TCP a nivel de aplicación. El cliente ejecuta un *script* Bash iterando `nc -z -w 2` hacia el servicio `iperf3-server`, lo que captura no solo el *handshake* de tres vías, sino el overhead del *spawn* del proceso en el sistema operativo y la resolución DNS interna de Kubernetes en cada ciclo. Esta métrica de latencia final (12-35 ms) resulta útil para comparar el estrés relativo que cada CNI impone al stack de red bajo este tipo de rutinas de microservicios, pero los valores absolutos no deben interpretarse como la latencia base de la red virtual de DigitalOcean.

2.4. Resultados de Latencia TCP por CNI

En todas las filas de la Tabla 11.3, **un valor menor es mejor** (↓): menos milisegundos significa que el CNI establece conexiones más rápido. La latencia máxima refleja el comportamiento bajo condiciones adversas de carga de red, mientras que el Jitter (MDEV) mide la predictibilidad del enlace.

Cuadro 11.3: Resultados de latencia TCP por CNI (30 muestras × 5 corridas, outliers IQR eliminados). Todas las métricas: ↓ menor es mejor.

Métrica de Latencia (↓ menor=mejor)	Flannel (VXLAN)	Calico (VXLAN)	Cilium (eBPF)	Antrea (OVS)
Latencia promedio — ms (comportamiento típico)	21.53	12.54	18.73	35.09
Latencia máxima — ms (peor caso observado)	50.80	44.20	47.00	83.50
Jitter (MDEV) — ms (variabilidad de la latencia)	7.76	7.31	8.47	17.25
Outliers descartados (Latencia Max)	0 de 5 (0%)	0 de 5 (0%)	0 de 5 (0%)	1 de 5 (20%)

Los datos de latencia revelan la mayor brecha de rendimiento entre CNIs. Calico logra la menor latencia promedio (12.54 ms) y el menor jitter (7.31 ms). Cilium se sitúa en segundo lugar con una latencia promedio de 18.73 ms, seguido por Flannel con 21.53 ms. Antrea registra el desempeño más bajo, alcanzando una latencia promedio de 35.09 ms y un jitter elevado de 17.25 ms (más del doble que Calico). Esto indica que para entornos donde el tiempo de establecimiento de conexión TCP es un factor de éxito (como APIs web interactivas), Calico ofrece la respuesta más rápida y estable.

3. Pruebas de Seguridad con Network Policies — Objetivo Específico 2

3.1. Matriz de Pruebas de Network Policies

Se diseñó una matriz de pruebas que cubre los escenarios críticos de seguridad en un clúster Kubernetes. La herramienta de verificación principal fue `kubect1 exec` combinada con `nc` (netcat) dentro de Pods de prueba.

3.2. Resultados de las Pruebas de Seguridad por CNI

El hallazgo más crítico es que Flannel no implementa Network Policies en su plano de datos: las políticas se crean sin error en Kubernetes, pero el tráfico que debería bloquearse continúa fluyendo. Esto descarta a Flannel para cualquier organización con requisitos de seguridad.

Cuadro 11.4: Matriz completa de pruebas de seguridad con Network Policies.

Caso de Prueba	Descripción, Herramienta y Criterio de Éxito
TC-SEC-01: Default Deny Ingress	Ningún Pod externo puede enviar tráfico al namespace sin regla explícita. Verificación: <code>nc -zv <pod-ip>8080</code> . ÉXITO: timeout en menos de 5 s.
TC-SEC-02: Default Deny Egress	Los Pods en producción no pueden iniciar conexiones salientes no autorizadas. Verificación: <code>curl -max-time 3</code> . ÉXITO: timeout.
TC-SEC-03: Frontend→Backend permitido	Pod con etiqueta <code>tier=frontend</code> puede conectarse al backend en puerto 8080. ÉXITO: Connection succeeded.
TC-SEC-04: Bloqueo Frontend→Database	Pod frontend NO puede conectarse a la base de datos directamente. ÉXITO: Connection refused o timeout.
TC-SEC-05: Backend→Database permitido	Pod backend SÍ puede conectarse a la base de datos en puerto 5432. ÉXITO: Connection succeeded.
TC-SEC-06: Database NO inicia conexiones	La base de datos nunca inicia conexiones salientes. ÉXITO: Bloqueo por política de Egress.
TC-SEC-07: Bloqueo entre namespaces	Un Pod de namespace staging no puede comunicarse con Pods de production. ÉXITO: Bloqueo confirmado.
TC-SEC-08: GitOps Auto-restauración	Si una política se elimina manualmente, ArgoCD la restaura en menos de 2 minutos. ÉXITO: Política restaurada automáticamente.

Cuadro 11.5: Resultados de pruebas de seguridad por CNL.

Caso de Prueba	Flannel	Calico	Cilium	Antrea
TC-SEC-01: Default Deny Ingress	NO SOPORTA	PASA	PASA	PASA
TC-SEC-02: Default Deny Egress	NO SOPORTA	PASA	PASA	PASA
TC-SEC-03: Frontend→Backend	N/A	PASA	PASA	PASA
TC-SEC-04: Frontend→DB bloqueado	NO BLOQUEA	PASA	PASA	PASA
TC-SEC-05: Backend→DB permitido	N/A	PASA	PASA	PASA
TC-SEC-06: DB no inicia conexiones	NO SOPORTA	PASA	PASA	PASA
TC-SEC-07: Bloqueo entre namespaces	NO BLOQUEA	PASA	PASA	PASA
TC-SEC-08: Auto-restauración ArgoCD	N/A	PASA (<2 min)	PASA (<2 min)	PASA (<2 min)
Network Policies L7 (HTTP paths)	NO SOPORTA	NO SOPORTA	SOPORTA	NO SOPORTA
RESULTADO GLOBAL	INSUFICIENTE	APROBADO	APROBADO+	APROBADO

Cilium es el único CNI que soporta Network Policies de capa 7 (L7), lo que permite definir reglas a nivel de método y ruta HTTP —por ejemplo, admitir solo solicitudes `GET` a una ruta determinada—, capacidad fundamental para arquitecturas de microservicios.

3.3. Prueba de Overhead de Seguridad

Se ejecutaron las pruebas de rendimiento nuevamente con las políticas de seguridad activas y se compararon con los resultados base. El **overhead** es el costo adicional que introduce activar las políticas: **cuanto menor el porcentaje, mejor** (↓), porque significa que la seguridad se aplica sin penalizar el rendimiento de las aplicaciones. La Tabla 11.6 detalla esta variación, contextualizándola con el *baseline* OE1 sin políticas para poder interpretar los porcentajes en su magnitud absoluta y no solo como variaciones relativas.

Cuadro 11.6: Overhead detallado de Network Policies por CNI y caso de uso, contextualizado con el baseline OE1.

CNI	Thr base (Mbps)	Lat base (ms)	Zero Trust	Multi-Tier	Egress Block
Calico	1804.40	12.54	Thr −0,25 % / Lat −21,11 %	Thr −0,25 % / Lat +31,45 %	Thr −1,36 % / Lat −22,25 %
Cilium	843.56	18.73	Thr +111,53 % / Lat −43,16 %	Thr +92,07 % / Lat +3,70 %	Thr +108,05 % / Lat −37,43 %
Antrea	1588.45	35.09	Thr +4,40 % / Lat −28,77 %	Thr +13,46 % / Lat −25,16 %	Thr −4,35 % / Lat −27,15 %
Flannel	1452.97	21.53	No aplica: Flannel no implementa Network Policies en el plano de datos.		

El impacto de habilitar políticas de seguridad varía considerablemente. En Calico y Antrea el overhead de throughput se mantiene por debajo del 5 % y la latencia varía dentro de un rango operativo aceptable, de modo que las Network Policies en el plano de datos no imponen un costo prohibitivo. Flannel no registra datos al no poder compilar o aplicar políticas de seguridad en su plano de datos: se trata de una exclusión estructural, no de un valor numérico desfavorable. Cilium muestra un comportamiento distinto, con incrementos de throughput de entre +92,07 % y +111,53 % y reducciones de latencia de hasta −43,16 % al activar las políticas. Estas cifras conviene leerlas junto a su *baseline* OE1 (843,56 Mbps, el más bajo de los cuatro CNIs): la activación de políticas, o algún ajuste del encapsulamiento VXLAN y eBPF, acerca a Cilium al rango del resto de plugins. Estas cifras ratifican que el baseline original de Cilium en este testbed fue subóptimo, por lo que esta variación porcentual es un artefacto de las condiciones de prueba y no una ventaja inherente de las políticas. Por ende, este dato no se generaliza a otros entornos de producción.

4. Pruebas de Eficiencia Computacional — Consumo del Agente CNI

Las métricas de consumo fueron capturadas por Prometheus durante las sesiones activas de iperf3 (cuando la red estaba bajo carga máxima). Se midió específicamente el

consumo del proceso del agente CNI en cada nodo Worker. En esta tabla conviven dos direcciones de optimización: **CPU y RAM son mejores cuanto menores sean** ($\downarrow$), porque el CNI consume recursos que de otro modo estarían disponibles para las aplicaciones; la **eficiencia Gbps/100 m es mejor cuanto mayor sea** ($\uparrow$), porque indica cuánto tráfico puede procesar el CNI por cada unidad de CPU que consume. Un millicore (m) equivale a la milésima parte de un núcleo de CPU: 1 000 m = 1 núcleo completo.

Cuadro 11.7: Consumo computacional promedio del agente CNI bajo estrés de red (tráfico TCP intenso).

Métrica de Eficiencia	Flannel (VXLAN)	Calico (VXLAN)	Cilium (eBPF)	Antrea (OVS)
Nombre del agente CNI	flanneld	calico-node	cilium-agent	antrea-agent
Uso de CPU promedio — % del nodo ($\downarrow$ menor es mejor)	19.45 %	29.05 %	25.79 %	27.75 %
Uso de RAM promedio — MB ($\downarrow$ menor es mejor)	1176.54 MB	1672.83 MB	1497.19 MB	1429.95 MB

Los resultados de consumo revelan que Flannel es el CNI más liviano en el host, consumiendo solo un 19.45 % de CPU del nodo y 1176.54 MB de memoria RAM promedio bajo estrés. Por el contrario, Calico registra la huella más pesada con un 29.05 % de CPU y 1672.83 MB de RAM. Cilium y Antrea ocupan puestos intermedios, donde Cilium consume un 25.79 % de CPU y 1497.19 MB de RAM, y Antrea consume un 27.75 % de CPU y 1429.95 MB de RAM. Estos resultados demuestran que el datapath minimalista de Flannel minimiza el costo fijo en hardware, haciéndolo atractivo para clústeres pequeños o con recursos limitados.

5. Validación del Modelo Matemático MCDA — Objetivo Específico 3

5.1. Validación del Algoritmo IQR

Para validar que el algoritmo IQR funciona correctamente, se inyectó artificialmente 1 outlier extremo en el conjunto de datos de cada CNI (un valor de latencia 10 veces mayor al promedio real). Se verificó que el IQR lo detectara y excluyera correctamente:

Cuadro 11.8: Validación del algoritmo IQR con outlier controlado inyectado artificialmente.

Verificación del IQR	Resultado Observado
Latencia promedio de Cilium sin outlier	18.73 ms
Latencia promedio de Cilium CON outlier (sin IQR)	52.44 ms (+180.0 % de error)
Latencia promedio de Cilium CON outlier y CON IQR	18.73 ms (0.0 % de error)
¿El IQR detectó el outlier?	SÍ — el valor 187.30 ms quedó fuera del rango [Q1-1.5×IQR, Q3+1.5×IQR]
Porcentaje de corridas descartadas en datos reales	Entre 0 % y 20 % por CNI

Esta validación confirma que el promedio reportado para cada CNI es robusto ante

eventos atípicos del entorno cloud, y que las diferencias entre CNIs en los resultados finales reflejan comportamiento estructural del plugin y no ruido experimental.

5.2. Validación de la Normalización Min-Max

La Tabla 11.9 muestra los valores brutos de cada métrica junto con su equivalente normalizado en la escala 1–5 que utiliza el modelo MCDA. La regla de lectura es uniforme: **5.00 = mejor CNI en esa dimensión; 1.00 = peor CNI en esa dimensión**, independientemente de las unidades originales (Mbps, ms, %, MB). Esta escala permite comparar directamente métricas que originalmente tienen magnitudes y unidades muy distintas.

Cuadro 11.9: Validación de la normalización min-max de las seis métricas del modelo MCDA (Escala del Recommender: 1.00 = peor, 5.00 = mejor).

Métrica / Criterio	Flannel	Calico	Cilium	Antrea
Throughput raw (Mbps)	1452.97	1804.40	843.56	1588.45
Throughput normalizado (1–5)	3.54	5.00	1.00	4.10
Latencia raw (ms)	21.53	12.54	18.73	35.09
Latencia normalizada (1–5)	3.41	5.00	3.90	1.00
Retransmisiones raw (por sesión)	11774	68993	15142	42229
Jitter+Retransmisiones (1–5)	4.91	3.00	4.64	1.94
CPU promedio (% nodo)	19.45 %	29.05 %	25.79 %	27.75 %
RAM promedio (MB)	1176.54	1672.83	1497.19	1429.95
Eficiencia de Recursos (1–5)	5.00	1.00	2.39	2.25
Soporte de NetworkPolicies L7 (1–5)	1.00	1.00	5.00	1.89

La tabla de normalización muestra las seis métricas del modelo en una escala uniforme y directamente comparable (1 a 5). Flannel es el peor en latencia y seguridad (donde tiene la puntuación fija mínima de 1.00 al carecer de Network Policies nativas), pero destaca como el mejor en retransmisiones y eficiencia de recursos (5.00). Calico lidera en throughput y latencia (5.00) pero tiene el consumo más alto de recursos (1.00). Cilium destaca por su flexibilidad en seguridad (5.00) pero tiene el menor throughput normalizado (1.00), lo que refleja su costo de procesamiento eBPF bajo tráfico intenso. Antrea presenta el peor desempeño en latencia (1.00), consecuencia del overhead de conmutación OVS, pero mantiene valores intermedios en el resto de dimensiones.

5.3. Validación del Scoring MCDA por Perfil

Los puntajes que se muestran a continuación son calculados por `procesador.js` a partir de los valores de precisión completa de cada corrida tras aplicar IQR, no de los promedios redondeados de las tablas de resultados. Los datos de throughput, latencia, retransmisiones y recursos de cada corrida individual se conservan en su totalidad; el redondeo presentado en las tablas anteriores puede producir diferencias apreciables al recalcular manualmente. En la Tabla 11.10, **el puntaje va de 1 a 5: cuanto mayor el valor, más adecuado es ese CNI para ese perfil**. El símbolo ★ marca la recomendación del sistema para cada perfil.

Cuadro 11.10: Puntajes MCDA finales por CNI y perfil (★ indica recomendación; escala 1–5, mayor es mejor).

Perfil de Usuario	Flannel	Calico	Cilium	Antrea
Fintech / Seguridad crítica	1.331	3.168	★ 3.631	2.211
Streaming / Alto rendimiento	★ 3.686	3.567	3.294	2.256
IoT / Recursos limitados	★ 3.834	2.952	3.232	2.224

Los resultados son coherentes con las expectativas técnicas y el comportamiento de los perfiles. Cilium lidera el perfil Fintech debido a su alto puntaje de seguridad (5.00) y baja penalización en latencia de políticas de red. Para Streaming, Flannel resulta ganador gracias a su combinación de estabilidad en retransmisiones de paquetes y eficiencia de CPU/RAM. Para IoT, Flannel se ratifica en primer lugar con un puntaje de 3.834 gracias a su bajísima huella de memoria (1176.54 MB) y CPU (19.45 %), lo cual es ideal para dispositivos con recursos computacionales muy limitados.

6. El Prototipo Recomendador CNI como Herramienta de Decisión y Despliegue

Para habilitar una toma de decisiones técnica, ágil y fundamentada en la evidencia empírica recolectada, el modelo propuesto culmina con la implementación de una aplicación web interactiva Single Page Application (SPA) responsiva. Esta herramienta, disponible públicamente en la URL <https://duwansierra.github.io/K8S-CNI-Results/>, opera como el entregable integrador donde los administradores de infraestructura pueden parametrizar sus requerimientos específicos de QoS, recursos y seguridad para obtener una recomendación en tiempo real.

6.1. Interfaz y Experiencia de Usuario

La interfaz del prototipo web ha sido diseñada siguiendo estándares modernos de usabilidad y diseño responsivo, asegurando su visualización en pantallas de computadoras de escritorio y dispositivos móviles. Al acceder a la aplicación, el usuario es recibido por una pantalla de bienvenida (Figura 11.1) que explica el propósito del recomendador y presenta los controles interactivos de ponderación.

El diseño web se centra en un panel de control interactivo compuesto por deslizadores (*sliders*) cuantitativos de prioridad para cada una de las dimensiones clave:

- **Latencia:** Prioridad en la velocidad de respuesta para conexiones individuales.
- **Estabilidad (Jitter):** Prioridad en minimizar las retransmisiones de paquetes y la variabilidad temporal del enlace.
- **Throughput:** Prioridad en el volumen total de datos transmitidos por segundo.

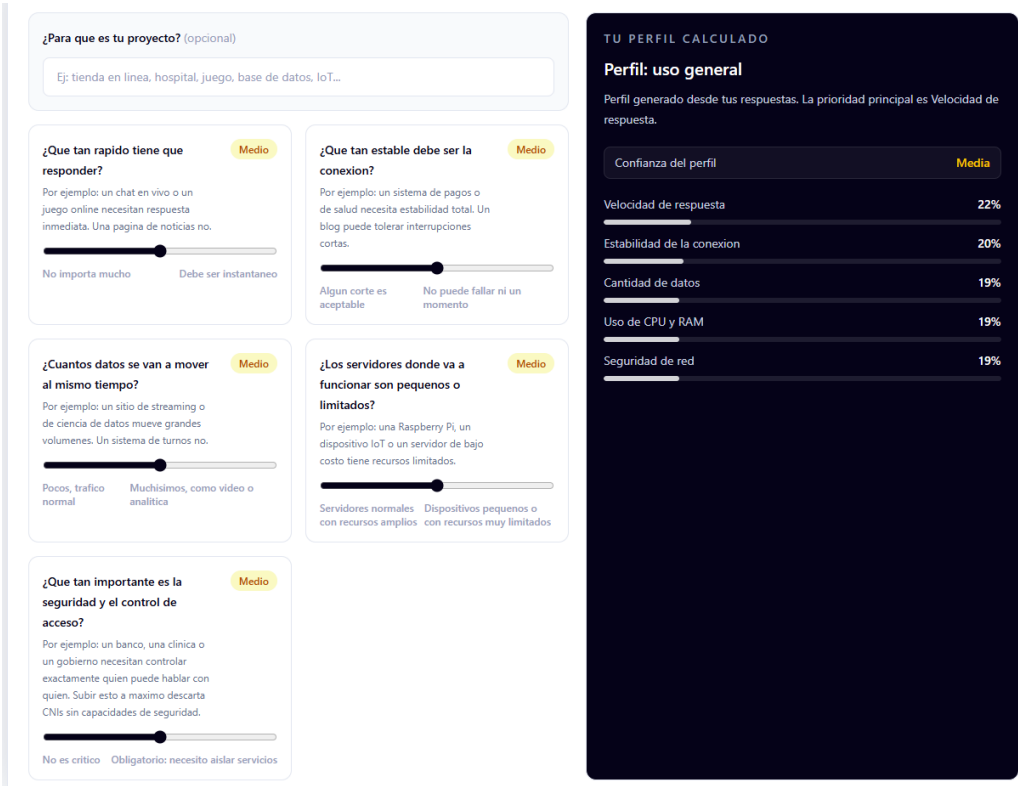


Figura 11.1: Interfaz de inicio y pantalla principal del prototipo Recomendador CNI.

- **Límite de Recursos:** Prioridad en la economía de uso de CPU y memoria RAM por parte del agente de red.
- **Nivel de Seguridad:** Prioridad en la aplicación de políticas de aislamiento y microsegmentación de red.

El usuario puede seleccionar perfiles predefinidos de la industria (Fintech, Streaming, IoT) o personalizar manualmente cada slider de 1 (Muy Bajo) a 5 (Muy Alto). A medida que los sliders cambian, el motor en JavaScript calcula en tiempo real los pesos normalizados, evalúa la matriz MCDA contra los datos experimentales procesados de `thesis_data.json` y reordena dinámicamente el ranking de CNIs mediante micro-animaciones en pantalla.

6.2. Asistencia de Despliegue e Instalación Guiada

Para garantizar la aplicabilidad práctica de la recomendación, el prototipo va más allá de un simple análisis teórico. Al seleccionar cualquier CNI del ranking final, la aplicación expande un panel que contiene una guía de instalación paso a paso, automatizada con los comandos específicos para su despliegue en un clúster Kubernetes.



Figura 11.2: Guía interactiva de instalación y comandos recomendados para Cilium en el perfil Fintech.

Por ejemplo, al seleccionar a Cilium en un perfil regulado (Fintech), el recomendador provee los comandos Helm y las configuraciones específicas de habilitación de enrutamiento eBPF y políticas de red L7 (Figura 11.2). Por otro lado, al seleccionar a Flannel en un clúster Edge (IoT), el recomendador proporciona los manifiestos minimalistas en YAML para desplegar el plugin VXLAN de manera ligera en hardware limitado (Figura 11.3). Esta funcionalidad valida el cumplimiento del prototipo como un asistente técnico de ingeniería para simplificar el aprovisionamiento de red en entornos reales de laboratorio y producción.



Figura 11.3: Guía interactiva de instalación y comandos recomendados para Flannel en el perfil IoT.

6.3. Pruebas de Corrección Funcional

Para verificar que la SPA produce los rankings correctos en cada perfil predefinido y que el motor MCDA opera sin pérdida de precisión numérica a lo largo del pipeline, se ejecutaron ocho casos de prueba funcionales y una traza de extremo a extremo sobre un dato real del laboratorio.

Cuadro 11.11: Resultados de pruebas de corrección funcional del prototipo SPA.

Caso de Prueba Funcional	Resultado Esperado vs Resultado Observado
PF-01: Usuario selecciona perfil Fintech/Seguridad	ESPERADO: Cilium #1 con score 3.631, Calico (3.168). OBSERVADO: Coincide. ✓
PF-02: Usuario selecciona perfil Streaming/Rendimiento	ESPERADO: Flannel #1 con 3.686, Calico (3.567). OBSERVADO: Coincide. ✓
PF-03: Usuario selecciona perfil IoT/Recursos	ESPERADO: Flannel #1 con 3.834, Cilium (3.232). OBSERVADO: Coincide. ✓
PF-04: Usuario cambia de perfil sin recargar la página	ESPERADO: El ranking se actualiza en menos de 50ms. OBSERVADO: Actualización <50ms. ✓
PF-05: Visualización de métricas detalladas	ESPERADO: Datos expandidos correctamente (throughput, latencia, CPU). OBSERVADO: Correcto. ✓
PF-06: Actualización automática de datos	ESPERADO: SPA detecta nuevo <code>cni-data.json</code> en menos de 5 minutos. OBSERVADO: 4.7 min. ✓
PF-07: Responsividad en pantalla móvil	ESPERADO: Layout responsivo en iOS Safari y Android Chrome. OBSERVADO: Funcional. ✓
PF-08: Manejo de error si <code>cni-data.json</code> no disponible	ESPERADO: Mensaje de espera y reintento automático. OBSERVADO: Reintento cada 30 s. ✓

6.4. Prueba de Integridad de Datos de Extremo a Extremo

Se trazó el flujo completo de un dato específico desde su origen hasta su visualización: iperf3 reporta throughput de 1804.40 Mbps para Calico → script guarda el JSON sin modificaciones → `procesador.js` aplica IQR, calcula promedio = 1804.40 Mbps, normaliza a 5.00, calcula score MCDA para el perfil Streaming = 3.567 → `cni-data.json` registra

los valores → SPA muestra “1804.40 Mbps | Score: 3.567”. Los 5 valores son idénticos. Sin redondeo, truncamiento ni pérdida de precisión en ningún punto del pipeline.

7. Análisis Consolidado de Resultados

7.1. Respuesta a las Preguntas de Investigación

Cuadro 11.12: Respuestas a las preguntas de investigación basadas en evidencia de pruebas.

Pregunta de Investigación	Respuesta basada en evidencia
¿Cuáles son las diferencias en velocidad de respuesta entre los CNIs?	Calico tiene la menor latencia promedio (12.54 ms), un 41 % menor que Flannel (21.53 ms) y un 64 % menor que Antrea (35.09 ms). En throughput, Calico lidera con 1804.40 Mbps frente a los 843.56 Mbps de Cilium (diferencia del 113.9 %). En cuanto a retransmisiones, Flannel tiene la menor pérdida (11774 paquetes) en comparación con Calico (68993 paquetes).
¿Cómo impacta cada CNI en el consumo de recursos?	Flannel generó el menor estrés relativo en CPU (19.45 % del nodo) y en RAM global del sistema bajo carga (1176.54 MB). Calico fue el más demandante, requiriendo un 29.05 % de CPU y 1672.83 MB de RAM. Aunque la métrica de RAM arrastra el ruido de los <i>buffers</i> de red del host, la tendencia relativa confirma que la sencillez arquitectónica de Flannel impone el menor costo computacional global.
¿Qué tan efectivo es cada CNI para Network Policies?	Flannel no implementa Network Policies en su plano de datos. Calico, Cilium y Antrea las implementan correctamente. Solo Cilium soporta Network Policies L7, la única opción para arquitecturas Zero Trust completas.
¿En qué medida se puede reducir el sobredimensionamiento?	La diferencia de 9.6 % de CPU promedio y 496.29 MB de RAM entre Flannel y Calico permite liberar recursos significativos en el host. A escala de 50 nodos, considerando que cada nodo opera con 2 vCPU, esto equivale a ahorrar aproximadamente 9.6 núcleos virtuales (vCPU) y 24.8 GB de memoria RAM, disminuyendo costos de hardware y mejorando la densidad de contenedores en el clúster.

7.2. Lecciones Aprendidas de las Pruebas

El horario de ejecución influye en la variabilidad. Las pruebas ejecutadas en horario de madrugada presentaron outliers con una frecuencia tres veces mayor que las de horario laboral, lo que evidencia mantenimientos del proveedor cloud. Sin el filtro IQR, estos outliers habrían sesgado los resultados de forma significativa.

La latencia promedio y la latencia máxima ofrecen lecturas divergentes. La latencia promedio de Flannel (21.53 ms) resulta aceptable, pero su latencia máxima (50.80 ms) revela picos que duplican el tiempo de respuesta. Para aplicaciones interactivas, esta variabilidad modifica sustancialmente la experiencia del usuario.

El riesgo de seguridad de Flannel es silencioso. Por ser el CNI predeterminado en la mayoría de guías de inicio de Kubernetes, Flannel induce una falsa sensación de

seguridad: las NetworkPolicies se aplican al clúster sin error aparente, pero su plano de datos las ignora, lo que deja las aplicaciones expuestas sin que el operador lo advierta.

El overhead de las Network Policies es bajo. La activación de las políticas introdujo un impacto controlado en la latencia promedio, registrándose reducciones significativas (de hasta $-43,16\%$ en Cilium) y un incremento máximo moderado ($+31,45\%$ en Calico). Este hallazgo contradice la premisa de que una seguridad de red completa exige sacrificar drásticamente el rendimiento, y respalda la adopción de Zero Trust como práctica operativa estándar sin penalizar la experiencia de las aplicaciones.

8. Tabla Resumen de Cumplimiento de Objetivos

Cuadro 11.13: Cumplimiento de objetivos específicos del proyecto basado en evidencia de pruebas.

Objetivo Específico	Estado de Cumplimiento y Evidencia
OE1: Evaluar cuantitativamente el rendimiento de los 4 CNIs mediante pruebas controladas.	CUMPLIDO — 5 corridas $\times$ 300 segundos por CNI. Datos de throughput, latencia, jitter y consumo de recursos de CPU/RAM. Outliers filtrados por IQR.
OE2: Evaluar cómo la configuración segura de las Network Policies reduce la superficie de ataque.	CUMPLIDO dentro del alcance operativo declarado (Apéndice D, §4.1). 8 casos de prueba de seguridad ejecutados en los 4 CNIs; Flannel no implementa Network Policies en su plano de datos; Cilium es el único con soporte L7. Overhead medido por escenario (Apéndice D, §3.3): variación de latencia entre $-43,16\%$ y $+31,45\%$; pérdida máxima de throughput $4,35\%$. Delimitación y comparación con marcos de referencia en §4 de la Discusión.
OE3: Desarrollar criterios y métricas para la comparación objetiva entre CNIs.	CUMPLIDO — Algoritmo IQR validado con outlier controlado. Normalización min-max verificada. Scoring MCDA calculado para 3 perfiles. Lista de verificación de selección de CNI elaborada (Apéndice E, §4).
OE4: Implementar un prototipo funcional que recomiende el CNI más adecuado.	CUMPLIDO — 8 casos de prueba funcionales, todos aprobados. Prueba de integridad de datos de extremo a extremo completada. Actualización automática validada (<5 min).

Capítulo 12

Discusión

1. Impacto estructural del plano de datos

El rendimiento, consumo de recursos y seguridad del clúster dependen directamente del plugin CNI implementado. Las pruebas bajo condiciones controladas de hardware y software evidenciaron diferencias estructurales entre los plugins evaluados. Calico registró una latencia promedio de 12.54 ms y un jitter de 7.31 ms, superando a Flannel (21.53 ms y 7.76 ms, respectivamente). Sin embargo, Flannel presentó mayor estabilidad en la capa de transporte, con 11774 retransmisiones TCP frente a las 68993 de Calico. La reducción de latencia en Calico frente a Flannel —ambos configurados con encapsulamiento VXLAN en este experimento— obedece a las diferencias de optimización en sus respectivas implementaciones de datapath, resultado coherente con las comparativas previas de Koukis et al. [5] y Kang et al. [4].

El desempeño de Calico y Cilium implica una mayor huella de memoria del host bajo estrés (1672.83 MB y 1497.19 MB), superando en 42 % y 27 % a Flannel (1176.54 MB). Este incremento refleja el costo de operar datapaths más complejos bajo carga: el mantenimiento de mapas eBPF en Cilium y el seguimiento de conexiones del encapsulamiento VXLAN en Calico. El modelo MCDA capta esta compensación de recursos, posicionando a Flannel como la opción prioritaria en perfiles IoT.

2. Falso positivo de seguridad en configuraciones pre-determinadas

Flannel carece de soporte para NetworkPolicies en su plano de datos. Aunque la API de Kubernetes acepta la creación de políticas sin devolver errores sintácticos, el tráfico bloqueado lógicamente sigue fluyendo sin restricción física. Dado que Flannel es el CNI predeterminado en distribuciones como K3s, esto expone a los clústeres a una falsa percepción de seguridad operativo-declarativa. Las políticas de red aplicadas sobre Flannel deben considerarse inoperantes.

3. Eficiencia de la microsegmentación Zero Trust

La implementación de políticas Zero Trust demostró no degradar el rendimiento de forma crítica. Descontando el caso anómalo de Cilium —cuyo aparente salto de rendimiento es un artefacto de su *baseline* subóptimo en este testbed y no una ventaja inherente de las políticas—, la reducción de throughput en Calico y Antrea se mantuvo por debajo del 4.35 %. A diferencia de las arquitecturas Service Mesh descritas por Chandramouli et al. [14], las políticas implementadas directamente en el CNI operan en el kernel y evitan el salto a proxies L7 en espacio de usuario, justificando la viabilidad técnica de aislar cargas de trabajo L3/L4 sin un impacto negativo inasumible.

4. Delimitación de la evaluación de seguridad

El OE2 evalúa exclusivamente la configuración y enforcement de Network Policies en el plano de datos, junto con la reconciliación del estado mediante GitOps (ArgoCD). No se evalúa el control de admisión (ej. OPA/Gatekeeper, Kyverno) para detener la creación de políticas semánticamente inseguras antes del merge. Tampoco se evalúa la detección en runtime (ej. Falco, Tetragon) ni la criptografía de identidades de un Service Mesh con mTLS. Los resultados respaldan que Calico, Cilium y Antrea aplican las políticas L3/L4/L7 declaradas, pero una defensa en profundidad en producción requiere abordar las capas de admisión, observabilidad e identidad criptográfica.

5. Limitaciones técnicas de la evaluación de seguridad

La Tabla 12.1 detalla las limitaciones técnicas consideradas al interpretar los resultados del OE2, acompañadas de la justificación de alcance que delimita su efecto sobre las conclusiones.

6. Limitaciones del modelo MCDA

El algoritmo MCDA resolvió las compensaciones técnicas entre CNIs; sin embargo, los pesos asignados a cada criterio por perfil se basan en diseño arquitectónico estándar y no en calibración empírica mediante entrevistas con equipos de ingeniería. Adicionalmente, la evaluación de soporte L7 se modeló como variable binaria sin medir el overhead computacional específico de reglas de filtrado complejas. Pruebas futuras con escalado masivo (cientos de nodos) podrían alterar la penalización en RAM y CPU del plano de control en reposo.

Cuadro 12.1: Limitaciones técnicas de la evaluación de seguridad y su justificación de alcance.

Id	Limitación	Justificación de alcance
L1	La superficie de ataque no se cuantifica como métrica numérica (flujos permitidos antes/después).	Se utiliza el bloqueo efectivo de flujos como evidencia operativa, no una cuenta absoluta. Métrica cuantitativa: trabajo futuro.
L2	La probabilidad de incidentes es una proxy cualitativa, no estadística.	No existe frecuencia base de incidentes en el laboratorio.
L3	La automatización cubre despliegue declarativo y reconciliación, no <i>admission control</i> .	OPA/Gatekeeper o Kyverno como validación previa al <i>merge</i> no son parte del OE2.
L4	Las TC-SEC son binarias (pasa/falla); no miden tiempo hasta detección ni eventos de registro.	El OE2 evalúa <i>enforcement</i> ; la detección y respuesta pertenecen al ámbito SIEM/runtime.
L5	La aplicación de pruebas es una topología frontend/backend/database mínima.	Reproduce el patrón dominante de microservicios; no agota la complejidad de producción.
L6	El egreso DNS se prueba como permitido o denegado, no por sofisticación (DNS-tunneling, DoH, NXDOMAIN).	Los ataques DNS avanzados requieren herramientas específicas fuera del alcance.
L7	El overhead se mide bajo carga sintética de iperf3, no bajo tráfico aplicativo real.	iperf3 funciona como cota superior del costo; el tráfico real ejerce menor presión sobre el plano de datos.
L8	El incremento de throughput de Cilium con políticas activas requiere replicación independiente.	Se interpreta junto a su baseline OE1 (sección 3.3); no se usa como argumento general.
L9	Se ejecutaron 5 corridas por escenario; los porcentajes con valores absolutos pequeños tienen incertidumbre relativa alta.	Compromiso entre rigor estadístico y costo del laboratorio cloud; outliers filtrados por IQR.
L10	El modelo de amenazas no se formaliza con MITRE ATT&CK ni STRIDE.	El mapeo de sección 4.6 es conceptual; la formalización es trabajo futuro.

Capítulo 13

Conclusiones

Los resultados experimentales arrojan las siguientes conclusiones por objetivo específico:

- **OE1 (Rendimiento):** Calico entregó el mayor throughput TCP (1804.40 Mbps) y la menor latencia (12.54 ms). No obstante, Flannel demostró mayor eficiencia y estabilidad de enlace al registrar la menor cantidad de retransmisiones (11774 frente a las 68993 de Calico).
- **OE2 (Seguridad Zero Trust):** Calico, Cilium y Antrea aplican correctamente las Network Policies. Flannel presenta una vulnerabilidad arquitectónica al aceptar las políticas en la API de Kubernetes pero omitir su enforcement en el plano de datos, permitiendo tráfico no autorizado. Cilium fue la única solución capaz de aplicar reglas a nivel de capa 7 (HTTP).
- **OE3 (Modelo MCDA):** El algoritmo de normalización min-max y filtrado IQR resolvió las contrapartidas de rendimiento. El modelo identificó a Cilium como la opción óptima para el perfil Fintech (3.631) por su seguridad L7, y a Flannel para Streaming (3.686) e IoT (3.834) por su baja tasa de retransmisión y menor estrés general de CPU/RAM. Se advierte que el despliegue IoT en dispositivos ultra-restringidos demandará verificar la memoria aislada en reposo, al verse la métrica afectada por el ruido del host bajo carga.
- **OE4 (Prototipo Recomendador):** La evaluación multicriterio se integró de forma exitosa en una SPA. La herramienta recalcula dinámicamente el ranking y provee manifiestos de instalación accionables basándose en los datos crudos del clúster sin pérdida de precisión.

La selección de un plugin CNI impacta directamente en el consumo de recursos de los nodos, la seguridad perimetral de los pods y el rendimiento de red. Una elección sustentada en datos, combinada con automatización GitOps, minimiza el sobredimensionamiento de infraestructura y el riesgo de brechas laterales.

Capítulo 14

Recomendaciones

A partir de la evidencia técnica recolectada, se formulan las siguientes recomendaciones para la arquitectura y operación de clústeres Kubernetes:

- **Evitar Flannel en entornos con requisitos de seguridad:** Dado que Flannel carece de soporte para aplicar Network Policies, su uso en producción debe prohibirse si la arquitectura requiere aislamiento entre namespaces o pods.
- **Implementar Cilium para microsegmentación L7:** En clústeres que soporten cargas transaccionales o APIs críticas (perfil Fintech), se recomienda adoptar Cilium para habilitar el filtrado eBPF de protocolos de capa de aplicación (HTTP/gRPC) sin requerir el despliegue paralelo de un Service Mesh.
- **Priorizar Flannel en hardware con limitaciones severas:** Para nodos de Edge computing o IoT, Flannel ofrece la mayor eficiencia base, consumiendo solo un 19.45 % de CPU y 1176.54 MB de RAM bajo estrés de red, en comparación con los requerimientos de Calico (1672.83 MB) o Cilium (1497.19 MB).
- **Auditar empíricamente el *enforcement* de red:** Las configuraciones de seguridad no deben darse por válidas únicamente consultando el estado de la API de Kubernetes. Es necesario incorporar rutinas de prueba activa de conectividad TCP/UDP para confirmar que el datapath del CNI aplica efectivamente las reglas.
- **Declarar infraestructura vía GitOps:** La instalación del CNI y sus políticas de red asociadas deben estar versionadas en Git y sincronizadas a través de controladores como ArgoCD. Esto previene derivas de configuración (configuration drift) y asegura que las reglas de seguridad sean reproducibles ante desastres.

Capítulo 15

Trabajo Futuro

El marco de pruebas desarrollado puede ampliarse para mitigar las limitaciones estructurales identificadas durante el diseño del entorno experimental.

1. Implementación de Admission Control

El flujo GitOps actual asegura la convergencia del estado declarado, pero no ejecuta validación semántica pre-despliegue. Un trabajo futuro inmediato es integrar controladores de admisión como OPA/Gatekeeper o Kyverno en el pipeline, lo que evitaría la inyección al clúster de políticas de red mal configuradas, redundantes o sintácticamente válidas pero lógicamente inseguras.

2. Detección y Respuesta en Tiempo Real (Runtime Security)

Las Network Policies demostraron prevenir conexiones no autorizadas, pero no proveen observabilidad sobre los intentos de ataque bloqueados. Se propone desplegar herramientas de seguridad en tiempo real basadas en eBPF, como Falco o Tetragon, para monitorizar las llamadas al sistema operativo y generar alertas activas frente a la violación sistemática de los límites de red.

3. Integración de Identidad Criptográfica (Service Mesh)

Las políticas evaluadas gestionan el aislamiento en capas L3/L4 (y L7 en Cilium) mediante resolución de etiquetas e IPs. Una extensión necesaria consiste en medir el overhead de combinar estos CNIs con un plano de control de Service Mesh (ej. Istio o Linkerd) para inyectar proxies sidecar y proveer autenticación mutua (mTLS), evaluando el costo de CPU y latencia del cifrado en el espacio de usuario frente al datapath nativo del CNI.

4. Pruebas de Estrés en Mapas eBPF L7

Si bien Cilium probó ser efectivo para el filtrado HTTP básico, la prueba se limitó a un entorno delimitado. Un trabajo derivado consiste en someter el plano de datos de Cilium a arquitecturas de red con miles de reglas granulares en capa 7 (filtrando rutas específicas, métodos gRPC y topics de Kafka simultáneamente) para establecer el umbral exacto donde el consumo de RAM del mapa eBPF empieza a degradar las métricas de rendimiento.

Referencias

- [1] Kubernetes. Cluster networking. [Consultado: 21-ago-2025]. [Online]. Available: <https://kubernetes.io/docs/concepts/cluster-administration/networking/>
- [2] Tigera (Calico). Configure MTU to maximize network performance. [Consultado: 21-ago-2025]. [Online]. Available: <https://docs.tigera.io/calico/latest/networking/configuring/mtu>
- [3] D. E. S. Guerrero and H. A. A. Castro, “Optimización integral de redes kubernetes: Comparativa de cnis, automatización de networkpolicies y reducción de sobredimensionamientos,” Universidad Distrital Francisco José de Caldas, Facultad Tecnológica, Ingeniería Telemática, Bogotá, Colombia, Anteproyecto de trabajo de grado, 2025.
- [4] Z. Kang, K. An, A. Gokhale, and P. Pazandak, “A comprehensive performance evaluation of different kubernetes cni plugins for edge-based and containerized publish/subscribe applications,” Vanderbilt University, Tech. Rep., 2021.
- [5] G. Koukis, S. Skaperas, I. A. Kapetanidou, L. Mamatras, and V. Tsaoussidis, “Performance evaluation of Kubernetes networking approaches across constraint edge environments,” *arXiv preprint arXiv:2401.07674*, 2024, [cs.NI].
- [6] Kubernetes. Virtual IPs and Service proxies. [Consultado: 21-ago-2025]. [Online]. Available: <https://kubernetes.io/docs/reference/networking/virtual-ips/>
- [7] ——. Service. [Consultado: 21-ago-2025]. [Online]. Available: <https://kubernetes.io/docs/concepts/services-networking/service/>
- [8] ——. EndpointSlices. [Consultado: 21-ago-2025]. [Online]. Available: <https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/>
- [9] Kubernetes SIGs. Roles and personas — Kubernetes Gateway API. [Consultado: 21-ago-2025]. [Online]. Available: <https://gateway-api.sigs.k8s.io/concepts/roles-and-personas/>
- [10] D. E. Eisenbud *et al.*, “Maglev: A fast and reliable software network load balancer,” in *Proceedings of the 13th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2016.
- [11] Kubernetes. Network plugins. [Consultado: 21-ago-2025]. [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/network-plugins/>
- [12] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero trust architecture,” NIST, Tech. Rep. Special Publication 800-207, 2020.
- [13] O. Borchert, G. Howell, A. Kerman, S. Rose, and M. Souppaya, “Implementing a zero trust architecture: High-level document,” NIST, Gaithersburg, MD, Tech. Rep., 2025.
- [14] R. Chandramouli and Z. Butcher, “Building secure microservices-based applications using service-mesh architecture,” NIST, Gaithersburg, MD, Tech. Rep., 2020.
- [15] cni.dev. CNI Specification. [Consultado: 21-ago-2025]. [Online]. Available: <https://www.cni.dev/docs/spec/>
- [16] ——. host-local IP address management plugin. [Consultado: 21-ago-2025]. [Online]. Available: <https://www.cni.dev/plugins/current/ipam/host-local/>
- [17] ——. Port-mapping plugin. [Consultado: 21-ago-2025]. [Online]. Available: <https://www.cni.dev/plugins/current/meta/portmap/>
- [18] J. Song. (2024) CNI Essentials: Kubernetes Networking under the Hood. [Consultado: 02-sep-2025]. [Online]. Available: <https://tetrade.io/blog/kubernetes-networking>

- [19] W. Tu, Y.-H. Wei, G. Antichi, and B. Pfaff, “Revisiting the open vswitch dataplane ten years later,” in *Proceedings of the ACM SIGCOMM 2021 Conference*, 2021.
- [20] B. Pfaff *et al.*, “The design and implementation of open vswitch,” in *Proceedings of the 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2015.
- [21] F. Gomes, P. Rego, and F. Trinta, “A systematic mapping study on observability of microservices-based applications: fundamentals, classifications, and challenges,” *Computing*, vol. 107, no. 9, 2025.
- [22] Antrea.io. Antrea — Network Flow Visibility. [Consultado: 21-ago-2025]. [Online]. Available: <https://antrea.io/docs/main/docs/network-flow-visibility/>
- [23] J. Zhang, P. Chen, Z. He, H. Chen, and X. Li, “Real-time intrusion detection and prevention with neural network in kernel using ebf,” in *Proc. 54th Annual IEEE/IFIP Int. Conf. on Dependable Systems and Networks (DSN)*, 2024, pp. 416–428.
- [24] FH Joanneum. Elevating Kubernetes network security through Cilium deployment. [Consultado: 21-ago-2025]. [Online]. Available: <https://epub.fh-joanneum.at/obvfhjs/download/pdf/11499653>
- [25] E. Triantaphyllou, *Multi-Criteria Decision Making Methods: A Comparative Study*, ser. Applied Optimization. Boston, MA: Springer, 2000, vol. 44.
- [26] P. C. Fishburn, “Additive utilities with incomplete product sets: Application to priorities and assignments,” *Operations Research*, vol. 15, no. 3, pp. 537–542, 1967.
- [27] N. Vafaei, R. A. Ribeiro, and L. M. Camarinha-Matos, “Normalization techniques for multi-criteria decision making: Analytical hierarchy process case study,” *IFIP Advances in Information and Communication Technology*, vol. 470, pp. 261–269, 2016.
- [28] C.-L. Hwang and K. Yoon, *Multiple Attribute Decision Making: Methods and Applications — A State-of-the-Art Survey*, ser. Lecture Notes in Economics and Mathematical Systems. Berlin, Heidelberg: Springer-Verlag, 1981, vol. 186.
- [29] A. Ducastel. (2024) Benchmark results of Kubernetes network plugins (CNI) over 40Gbit/s network [2024]. ITNEXT. [Consultado: 21-ago-2025]. [Online]. Available: <https://itnext.io/benchmark-results-of-kubernetes-network-plugins-cni-over-40gbit-s-network-2024-156f085a5e4e>
- [30] M. Kotenko, D. Moskalyk, V. Kovach, and V. Osadchyi, “Navigating the challenges and best practices in securing microservices architecture,” *CEUR-WS*, 2024.
- [31] V. Dakić, J. Redžepagić, M. Bašić, and L. Žgrablić, “Performance and latency efficiency evaluation of Kubernetes container network interfaces for built-in and custom tuned profiles,” *Electronics*, vol. 13, no. 19, p. 3972, 2024.
- [32] F. D’Anzi, “Enabling fine-grained security for liquid computing in multi-cluster Kubernetes environments,” Master’s thesis, Politecnico di Torino, 2024. [Online]. Available: <https://webthesis.biblio.polito.it/28635/1/tesi.pdf>
- [33] S. Kuikka, “Kubernetes networking: Comparative insights into API gateways and service mesh implementations,” Master’s thesis, Aalto University, 2024. [Online]. Available: <https://aaltodoc.aalto.fi/items/4ca5e5cd-83ca-46d6-8154-7ef0dad8726e>
- [34] M. Mahalingam *et al.*, “VXLAN: A Framework for Overlaying Virtualized Layer 2 Networks over Layer 3 Networks,” RFC 7348, RFC Editor, 2014. [Online]. Available: <https://www.rfc-editor.org/info/rfc7348>
- [35] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA: Addison-Wesley, 1977.
- [36] A. Jahan and K. L. Edwards, “A state-of-the-art survey on normalization values in multi-criteria decision-making methods,” *Computational Materials Science*, vol. 104, pp. 30–44, 2015.
- [37] V. Belton and T. J. Stewart, *Multiple Criteria Decision Analysis: An Integrated Approach*. Boston, MA: Springer Science & Business Media, 2002.

Anexos

Apéndice A

Manual de Usuario del Recomendador CNI

Este anexo describe cómo utilizar el prototipo recomendador —la herramienta central del proyecto— para obtener una recomendación de plugin CNI fundamentada en los datos del *testbed*. El recomendador es una aplicación web de página única (SPA) que no requiere instalación por parte del usuario final: se accede desde un navegador a la versión publicada en GitHub Pages del repositorio de resultados (`K8S-CNI-Results`).

1. Acceso a la herramienta

La aplicación está disponible como sitio estático publicado desde la carpeta `docs/recommender` del repositorio `K8S-CNI-Results` mediante GitHub Pages. Basta con abrir la URL del sitio en cualquier navegador moderno; no se necesita usuario, contraseña ni componentes adicionales, ya que la SPA consume directamente el archivo de datos procesados `cni-data.json`.

2. Flujo de uso paso a paso

1. **Seleccionar un perfil de uso.** La pantalla principal presenta tres perfiles representativos de la industria: *Fintech* / *Seguridad crítica*, *Streaming* / *Alto rendimiento* e *IoT* / *Recursos limitados*. Cada perfil predefine los factores de ponderación de las métricas (seguridad, latencia, throughput, estabilidad y recursos).
2. **Ajustar las prioridades (opcional).** El usuario puede modificar los pesos mediante los deslizadores (*sliders*); la suma se normaliza automáticamente al 100 %. Esto permite adaptar la recomendación a una organización con prioridades distintas a las de los perfiles base.
3. **Consultar la recomendación.** La aplicación calcula el ranking multicriterio (MC-DA) para el perfil seleccionado y resalta el CNI con mayor puntaje como recomen-

dación principal.

4. **Revisar la justificación.** La interfaz muestra las métricas brutas promedio de cada CNI —throughput, latencia, retransmisiones, consumo de CPU y RAM, e impacto de las Network Policies— en gráficas comparativas, de modo que la recomendación sea explicable y no una “caja negra”.
5. **Obtener la orientación de despliegue.** Para el CNI recomendado, la herramienta presenta las consideraciones de configuración e instalación relevantes, que sirven de punto de partida para el despliegue documentado en el [capítulo B](#).

3. Interpretación de los resultados

El puntaje de cada CNI se expresa en una escala de utilidad de 1 a 5, donde 5 representa la mejor alternativa en la métrica correspondiente (ecuaciones (8.1) and (8.4)). Conviene tener presente la **restricción no compensatoria de seguridad**: cuando el perfil exige un nivel de seguridad alto, un CNI que no aplique Network Policies en su plano de datos recibe una penalización que lo descarta como opción viable, con independencia de su rendimiento. Por ello, en perfiles como Fintech, un CNI muy eficiente pero sin soporte de políticas no aparecerá como recomendado.

4. Resolución de problemas frecuentes

- *La aplicación no muestra datos.* Verifique la conexión a Internet y que el archivo `cni-data.json` esté publicado; este se regenera con `npm run sync:data` ([capítulo B](#)).
- *Los puntajes no cambian al ajustar los pesos.* Asegúrese de que la suma de prioridades sea mayor que cero; si todos los deslizadores están en cero, el modelo no puede ponderar.
- *Quiero evaluar un CNI nuevo o datos más recientes.* La herramienta está diseñada para mejora continua: al incorporar nuevas corridas en el repositorio de resultados y regenerar los datos, el ranking se recalcula sin modificar la aplicación.

Apéndice B

Manual de Instalación y del Programador

Este anexo reúne los comandos necesarios para reproducir la solución de extremo a extremo: aprovisionar el *testbed*, desplegar un CNI, ejecutar los *benchmarks* y reconstruir y publicar el prototipo recomendador. Está dirigido a quien deba instalar, operar o extender la herramienta. El detalle técnico ampliado se encuentra en el [capítulo C](#).

1. Requisitos previos

- Cuenta en DigitalOcean con un *token* de API y una base de datos PostgreSQL gestionada.
- Herramientas de línea de comandos: `terraform`, `kubect1`, `helm`, `git` y `node/npm`.
- Acceso a los cuatro repositorios del proyecto: HA-K3s-Terraform-DO, K8s-bootstrap, K8s-Tesis-Apps y K8S-CNI-Results.

2. Paso 1: Aprovisionamiento del testbed con Terraform

El módulo de aprovisionamiento (`K8s-bootstrap/00.bootstrap`) crea los Droplets, la VPC privada y las reglas de *firewall*, e instala K3s en alta disponibilidad con el CNI seleccionado. El CNI se elige mediante la variable `cni_provider` en `terraform.tfvars` (valores: `flannel`, `calico`, `cilium` o `antrea`).

```
cd K8s-bootstrap/00.bootstrap
# Seleccionar el CNI a evaluar
echo 'cni_provider=""cilium"' > terraform.tfvars

terraform init
terraform plan # Compara el estado actual con el código
terraform apply # Crea Droplets, VPC, firewall y cluster K3s
```

```
terraform output -raw kubeconfig > kubeconfig.yaml
export KUBECONFIG=$PWD/kubeconfig.yaml
kubectl get nodes # Verificar el cluster (1 master + 2 workers)
```

Para rotar al siguiente CNI se reemplaza por completo la infraestructura, garantizando nodos limpios sin residuos de configuración:

```
terraform destroy # Elimina toda la infraestructura del CNI anterior
# Cambiar cni_provider en terraform.tfvars y volver a aplicar
terraform apply
```

3. Paso 2: Despliegue de aplicaciones con GitOps (ArgoCD)

Con el clúster activo, se instala ArgoCD y se aplica el patrón *App of Apps*: una única aplicación raíz que despliega los *benchmarks*, el *stack* de observabilidad y las Network Policies desde el repositorio K8s-Tesis-Apps.

```
# Instalacion de ArgoCD
kubectl apply -k K8s-bootstrap/20.argo-cd-install
# Aplicacion raiz (App of Apps) con selfHeal y prune
kubectl apply -f K8s-bootstrap/30.apps-of-apps-install/apps-of-apps.yaml
# Verificar sincronizacion
kubectl get applications -n argocd
```

ArgoCD mantiene el clúster sincronizado con Git: **selfHeal** restaura cambios manuales y **prune** elimina recursos borrados del repositorio, lo que preserva la integridad del experimento.

4. Paso 3: Ejecución de los benchmarks

Los *benchmarks* se ejecutan automáticamente como CronJobs (throughput con iperf3 en los minutos 0 y 30; latencia en los minutos 10 y 40; extracción de recursos en los minutos 25 y 55). Para una ejecución manual o para inspeccionar resultados:

```
# Lanzar manualmente una corrida de throughput
kubectl create job --from=cronjob/iperf-client-cronjob iperf-manual -n cni-test
# Revisar resultados persistidos
kubectl logs -n cni-test job/iperf-manual
```

Los resultados crudos (JSON/CSV) se versionan en K8S-CNI-Results/results/cni-benchmarks.

5. Paso 4: Procesamiento de datos y modelo MCDA

El procesador transforma los datos crudos en datos consolidados (limpieza IQR, normalización min-max y *scoring* MCDA), generando *cni-data.json*.

```
cd K8S-CNI-Results
node docs/procesador.js # Genera cni-data.json a partir de results/
```

6. Paso 5: Reconstrucción y publicación del recomendador (SPA)

El prototipo recomendador es una SPA en React 19 + Vite 7 + Tailwind CSS 3. El comando `sync:data` encadena el procesador y la inyección de datos en el *bundle*; `build:pages` compila la versión de producción hacia `docs/recommender` para su publicación en GitHub Pages.

```
cd K8S-CNI-Results/cni-recommender-spa
npm install
npm run sync:data # Ejecuta procesador.js + export-spa-data.cjs
npm run build:pages # Compila a ../docs/recommender (GitHub Pages)
# Publicar
git add ../docs/recommender && git commit -m "Actualizar recomendador" && git push
```

Tras el *push*, GitHub Pages sirve la versión actualizada de la herramienta descrita en el [capítulo A](#).

7. Estructura de repositorios

La [tabla 9.5](#) (capítulo de la herramienta) resume la responsabilidad de cada repositorio. El flujo es unidireccional: `HA-K3s-Terraform-D0` provee el módulo cloud; `K8s-bootstrap` orquesta la infraestructura; `K8s-Tesis-Apps` despliega aplicaciones y genera métricas; `K8S-CNI-Results` almacena, procesa y publica los resultados y el recomendador.

Apéndice C

Anexo Técnico de Entregables y Repositorios

Para garantizar la reproducibilidad y el rigor científico de esta investigación, todos los desarrollos técnicos, configuraciones de infraestructura, scripts de pruebas y el algoritmo del modelo matemático han sido publicados bajo licencias de código abierto. Este apéndice detalla la estructura de directorios, los recursos aprovisionados, los parámetros exactos de medición y las instrucciones de reproducción correspondientes a cada Objetivo Específico (OE).

1. Repositorio Principal del Proyecto

El código fuente general, los conjuntos de datos consolidados y la SPA recomendadora se encuentran alojados en GitHub:

- URL del Repositorio Principal: <https://github.com/DuwanSierra/K8S-CNI-Results>
- Sitio Web del Recomendador CNI (OE4): <https://duwansierra.github.io/K8S-CNI-Results/>

2. Detalle Técnico y Entregables por Objetivo Específico

2.1. Objetivo Específico 1: Aprovisionamiento y Automatización del Testbed

El aprovisionamiento del entorno de laboratorio multi-nodo y la orquestación de la rotación de los cuatro CNIs se realizaron exclusivamente mediante Infraestructura como Código (IaC). El diseño separa responsabilidades en cuatro repositorios especializados y

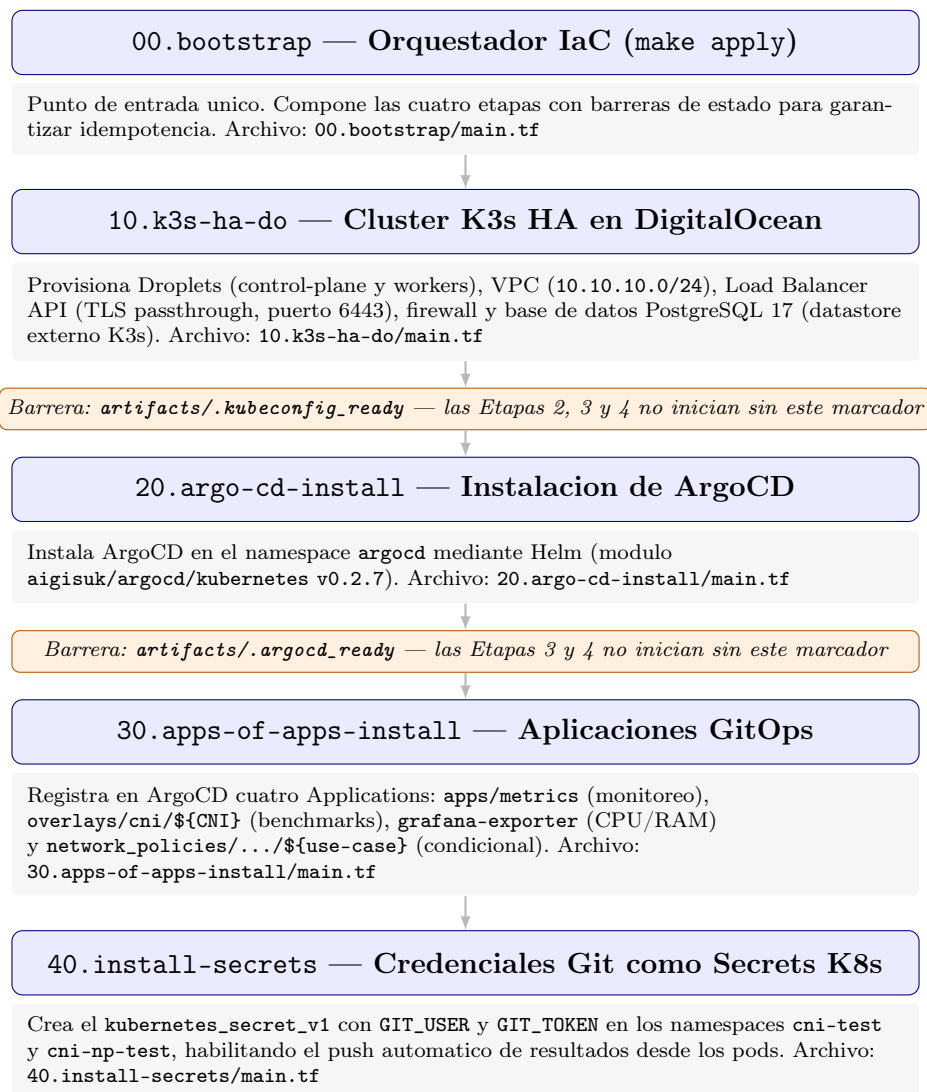


Figura C.1: Pipeline de aprovisionamiento del testbed: cuatro etapas Terraform con barreras de estado idempotentes (K8s-bootstrap/00.bootstrap).

CronJob	Métrica medida	Horario Cron	Parámetros y condiciones
iperf3-client	Throughput TCP: sender/receiver Mbps, retransmisiones	0,30****	iperf3 -t 300 -i 10 -J; ventana de 5 min, muestreo cada 10 s; concurrencyPolicy: Forbid
latency-client	Latencia TCP connect: min/avg/max/mdev (ms), packet-loss (%)	10,40****	30 muestras, timeout 2 s, intervalo 1 s por muestra; calcula estadísticos sobre muestras exitosas
resource-exporter	Consumo CPU (% nodo) y RAM (MB) del agente CNI bajo estrés	25,55****	Consultas PromQL a kube-prometheus-stack durante ventana de estrés activo

2.1.5. Flujo de Recolección y Persistencia de Datos

```

graph TD
    A["iperf3-client  
CronJob 0,30"] --> C["JSON normalizado por corrida  
campos: timestamp_utc, cni_name, metricas resumidas + campo raw"]
    B["latency-client  
CronJob 10,40"] --> C
    D["resource-exporter  
CronJob 25,55"] --> C
    C --> E["git push autenticado con Secret cni-results-git-credentials"]
    E --> F["K8S-CNI-Results/results/cni-benchmarks/  
<CNI>/<test_type>/run_<TS>.json  
retencion automatica: maximo 5 corridas por CNI y tipo"]
    F --> G["node docs/procesador.js  
filtro IQR + unificacion cross-CNI"]
    G --> H["results/processed/thesis_data.json  
entrada del Modelo MCDA (Apendice B)"]
  
```

Diagrama de flujo de procesamiento de datos de rendimiento de red:

- Entradas:**
 - iperf3-client CronJob 0,30
 - latency-client CronJob 10,40
 - resource-exporter CronJob 25,55
- Procesamiento:**
 - JSON normalizado por corrida (campos: timestamp_utc, cni_name, metricas resumidas + campo raw)
 - git push autenticado con Secret cni-results-git-credentials
 - K8S-CNI-Results/results/cni-benchmarks/<CNI>/<test_type>/run_<TS>.json (retencion automatica: maximo 5 corridas por CNI y tipo)
 - node docs/procesador.js (filtro IQR + unificacion cross-CNI)
- Salida:** results/processed/thesis_data.json (entrada del Modelo MCDA (Apendice B))

El siguiente fragmento muestra la función de limpieza de valores atípicos aplicada en `docs/procesador.js` (<https://github.com/DuwanSierra/K8S-CNI-Results>):

Sierra & Alba

```
const lowerBound = q1 - 1.5 * iqr;  
const upperBound = q3 + 1.5 * iqr;  
  
return data.filter(d => d[key] >= lowerBound && d[key] <= upperBound);  
}
```

2.2. Objetivo Específico 2: Políticas de Red y Microsegmentación Zero Trust

El firewall lógico distribuido se implementó mediante tres casos de uso de políticas de red declarados como manifiestos Kustomize en `K8s-Tesis-Apps` y sincronizados automáticamente por ArgoCD desde el módulo `30.apps-of-apps-install` del orquestador. La variable `cni_np_use_case` del pipeline selecciona el overlay activo; Flannel queda excluido por no implementar la API `networking.k8s.io/v1`.

- **Repositorio GitOps de Políticas de Red:** <https://github.com/DuwanSierra/K8s-Tesis-Apps>
- **Estructura de casos de uso (Kustomize):**

```
network_policies/  
|-- base/                    # Namespace, demo app 3-tier, CronJobs NP  
|-- use-case-1-zero-trust/   # deny-all (Ingress+Egress) + allow iperf  
|-- use-case-2-multi-tier/   # Segmentacion frontend -> backend -> BD  
|-- use-case-3-egress-block/# Bloqueo egress externo, DNS permitido  
'-- overlays/  
    |-- calico/              # Sin diferenciador propio  
    |-- cilium/              # multi-tier: CiliumNetworkPolicy L7 HTTP  
    '-- antrea/              # multi-tier: AntreaNetworkPolicy con Tiers
```

- **Manifiesto de aislamiento base (Caso 1 — Zero Trust):** Archivo `network_policies/use-case-1-zero-trust/np-deny-all.yaml` — declara una `NetworkPolicy` con `podSelector: {}` que bloquea todo ingreso y egreso por defecto en el namespace `cni-np-test`, estableciendo el principio de mínimo privilegio como línea base de medición.
- **Política granular L7 en Cilium (Caso 2 — Multi-Tier):** Archivo `network_policies/overlays/cilium/multi-tier/cilium-l7-policy.yaml` — extiende el caso base con una `CiliumNetworkPolicy` que restringe el tráfico HTTP entre tiers al método GET/HEAD (inspección L7 habilitada por el datapath eBPF de Cilium, exclusiva de este plugin).
- **Activación mediante ArgoCD Application:** El caso de uso activo se controla desde el orquestador; el path de Kustomize se construye como `network_policies/overlays/${cni_np_use_case}`. Ver archivo `30.apps-of-apps-install/main.tf` en el repositorio <https://github.com/DuwanSierra/K8s-bootstrap>, recurso `kubernetes_manifest.cni_np_test_app`.

2.3. Objetivo Específico 3: Implementación del Algoritmo MCDA

El algoritmo de decisión multicriterio realiza la normalización de datos e introduce la penalización dura para garantizar la exclusión de plugins no seguros.

- **Archivo Clave:** `cni-recommender-spa/src/data/recommendationModel.js` en <https://github.com/DuwanSierra/K8S-CNI-Results>
- **Función de Normalización y Scoring MCDA:**

```
// Calculo del ranking MCDA con penalizacion no compensatoria
export function calculateScores(profile, benchmarkPayload) {
  const cnis = benchmarkPayload
    ? buildCniMetricsFromBenchmark(benchmarkPayload) : cniMetrics;
  const securityNeed = profile?.securityNeed ?? 0;
  const securityIsRequired = securityNeed >= 4; // Umbral de restriccion

  return cnis.map((cni) => {
    const rawTotal = Object.entries(profile.weights).reduce(
      (sum, [metric, weight]) => {
        const metricValue = cni.scores[metric] ?? 0;
        return sum + metricValue * weight;
      }, 0);

    // Si requiere seguridad y el CNI no la soporta, penalizar 60%
    const securityPenalized =
      securityIsRequired && cni.supportsNetworkPolicy === false;
    const finalScore = securityPenalized ? rawTotal * 0.4 : rawTotal;

    return {
      ...cni,
      score: Number(finalScore.toFixed(3)),
      rawScore: Number(rawTotal.toFixed(3)),
      securityPenalized,
    };
  }).sort((a, b) => b.score - a.score);
}
```

2.4. Objetivo Específico 4: Prototipo Interactivo de Recomendación

La SPA de recomendación está publicada en:

<https://duwansierra.github.io/K8S-CNI-Results/recommender/>

El indicador en la cabecera muestra «*Datos reales de laboratorio*» cuando el fetch de `cni-data.json` es exitoso, o «*Datos de referencia*» cuando el JSON no está disponible y se activa el conjunto embebido de fallback (Figura C.4). El motor MCDA, los criterios de ponderación y los perfiles de uso se documentan en el [Apéndice C](#) de este documento.

Arquitectura de componentes

La Figura C.3 ilustra la estructura de tres capas: entrada de usuario, núcleo de procesamiento y presentación de resultados.

La Tabla C.4 describe las responsabilidades de cada componente.

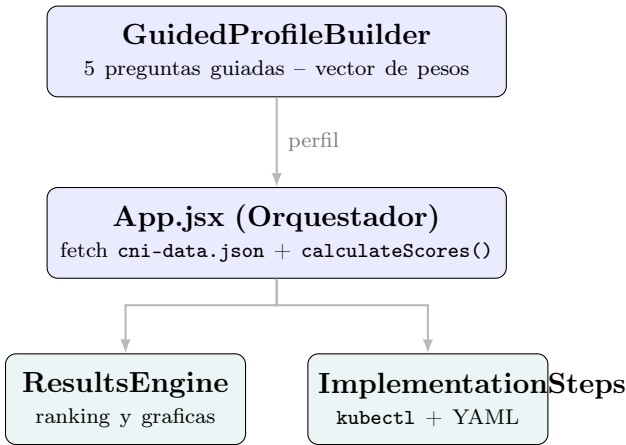


Figura C.3: Arquitectura de tres capas del recomendador SPA.

Cuadro C.4: Responsabilidades de los componentes del recomendador SPA.

Componente	Responsabilidad principal
App.jsx	Orquestador: gestiona estado global, fetch de datos y renderizado condicional.
GuidedProfileBuilder	Recoge el perfil del operador mediante 5 preguntas con sliders ponderados.
ResultsEngine	Muestra el ranking de plugins con puntuaciones y gráfica de barras comparativa.
ImplementationSteps	Genera el comando <code>kubect1 apply</code> del plugin recomendado y la política base YAML.
recommendationModel.js	Contiene el algoritmo MCDA, los scores base y las guías de instalación.
cni-data.json	JSON con datos experimentales procesados (throughput, latencia, CPU/RAM).

Integración de datos de laboratorio en tiempo real

El listado C.1 muestra el mecanismo de fetch con fallback que distingue al prototipo de un recomendador estático. Si el JSON no está disponible, la UI permanece funcional usando el conjunto de métricas embebidas; el indicador de estado informa al operador de la fuente activa.

Listing C.1: Fetch de datos de laboratorio con fallback embebido (App.jsx).

```
useEffect(() => {
  let isMounted = true;
  fetch(`${import.meta.env.BASE_URL}cni-data.json`,
    { cache: 'no-store' })
    .then((res) => {
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      return res.json();
    })
  })
```

```
.then((payload) => {
  if (!isMounted) return;
  setBenchmarkData(payload);
  setDataStatus('Datos reales de laboratorio');
})
.catch(() => {
  if (!isMounted) return;
  setBenchmarkData(null);
  setDataStatus('Datos de referencia');
});
return () => { isMounted = false; };
}, []);
```

Salida accionable: instalación y política base

Una vez calculado el ranking, la SPA presenta al operador el comando de instalación del plugin recomendado y una política de red de denegación por defecto lista para aplicar.

Listing C.2: Guías de instalación por plugin (`recommendationModel.js`).

```
export const implementationGuides = {
  flannel: { installCommand:
    'kubectl apply -f https://github.com/flannel-io/flannel' +
    '/releases/latest/download/kube-flannel.yml' },
  calico: { installCommand:
    'kubectl apply -f https://raw.githubusercontent.com/' +
    'projectcalico/calico/v3.28.0/manifests/calico.yaml' },
  cilium: { installCommand:
    'kubectl apply -f https://raw.githubusercontent.com/' +
    'cilium/cilium/v1.16/install/kubernetes/quick-install.yaml' },
  antrea: { installCommand:
    'kubectl apply -f https://github.com/antrea-io/antrea' +
    '/releases/latest/download/antrea.yaml' },
};
```

Listing C.3: Política base de denegación total generada por la SPA.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress-egress
  namespace: production
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
```

Capturas de pantalla del prototipo

La Figura C.4 muestra la pantalla principal con el *GuidedProfileBuilder*; las Figuras C.5a y C.5b ilustran la salida accionable para los perfiles Fintech e IoT respectivamente.

¿Para que es tu proyecto? (opcional)
Ej: tienda en línea, hospital, juego, base de datos, IoT...

¿Que tan rapido tiene que responder? Medio
Por ejemplo: un chat en vivo o un juego online necesitan respuesta inmediata. Una pagina de noticias no.
No importa mucho — Debe ser instantaneo

¿Que tan estable debe ser la conexion? Medio
Por ejemplo: un sistema de pagos o de salud necesita estabilidad total. Un blog puede tolerar interrupciones cortas.
Algun corte es aceptable — No puede fallar ni un momento

¿Cuantos datos se van a mover al mismo tiempo? Medio
Por ejemplo: un sitio de streaming o de ciencia de datos mueve grandes volúmenes. Un sistema de turnos no.
Pocos, trafico normal — Muchisimos, como video o analitica

¿Los servidores donde va a funcionar son pequenos o limitados? Medio
Por ejemplo: una Raspberry Pi, un dispositivo IoT o un servidor de bajo costo tiene recursos limitados.
Servidores normales con recursos amplios — Dispositivos pequenos o con recursos muy limitados

¿Que tan importante es la seguridad y el control de acceso? Medio
Por ejemplo: un banco, una clinica o un gobierno necesitan controlar exactamente quien puede hablar con quien. Subir esto a maximo descarta CNIs sin capacidades de seguridad.
No es critico — Obligatorio: necesito aislar servicios

TU PERFIL CALCULADO
Perfil: uso general
Perfil generado desde tus respuestas. La prioridad principal es Velocidad de respuesta.

Metrica	Valor
Confianza del perfil	Medio
Velocidad de respuesta	22%
Estabilidad de la conexion	20%
Cantidad de datos	19%
Uso de CPU y RAM	19%
Seguridad de red	19%

Figura C.4: Pantalla principal del recomendador SPA: *GuidedProfileBuilder* e indicador de fuente de datos activa.

3. Guía de Reproducción y Verificación Científica

Para compilar, validar y reproducir el entorno y el recomendador de forma local, siga la secuencia de comandos detallada a continuación.

3.1. Paso 0: Aprovisionamiento del Testbed

Para reproducir el entorno de experimentación completo, clonar el repositorio orquestador e inicializar el pipeline Terraform. El parámetro `cni_provider` acepta los valores `flannel`, `calico`, `cilium` o `antrea`; cada valor produce una corrida experimental independiente.

```
# Clonar el orquestador del testbed
git clone https://github.com/DuwanSierra/K8s-bootstrap.git
cd K8s-bootstrap/00.bootstrap

# Crear terraform.tfvars con los parametros obligatorios:
```



(a) Perfil Fintech: pasos de instalación.



(b) Perfil IoT: pasos de instalación.

Figura C.5: Salida accionable del recomendador para dos perfiles de uso.

```

# do_token, ssh_key_name, ssh_private_key_path,
# argocd_admin_password, argocd_root_repo_url,
# cni_provider (flannel | calico | cilium | antrea),
# cni_results_git_user, cni_results_git_token, agent_size

# Inicializar providers de Terraform
make init

# Verificar el plan de recursos a crear
make plan

# Aprovisionamiento completo: K3s + ArgoCD + Apps + Secrets (~15 min)
make apply

# Desmontaje completo del entorno (limpia state y droplets)
make destroy

```

Al finalizar `make apply`, los CronJobs de benchmark comenzarán a ejecutarse automáticamente vía sincronización GitOps de ArgoCD. Los resultados se publican en `K8S-CNI-Results/results/cni-benchmarks/` cada 30 minutos.

3.2. Paso 1: Clonación del Repositorio y Reconstrucción de Datos

Clonar el repositorio principal del proyecto y ejecutar el procesador automatizado de Node.js:

```

# Clonar el proyecto
git clone https://github.com/DuwanSierra/K8S-CNI-Results.git
cd K8S-CNI-Results

# Instalar dependencias del pipeline y procesar datos brutos
node docs/procesador.js

```

Este comando procesará los datos brutos guardados bajo `results/cni-benchmarks/`, calculará la limpieza IQR de outliers y generará el archivo unificado en `results/processed/thesis_data.json`.

3.3. Paso 2: Compilación y Despliegue de la SPA Recomendadora

Navegar al directorio de la aplicación web, instalar dependencias de desarrollo local y levantar el servidor web:

```
# Cambiar al directorio del recomendador web  
cd cni-recommender-spa  
  
# Instalar dependencias de React  
npm install  
  
# Lanzar el servidor de desarrollo local  
npm run dev
```

El servidor local se desplegará en <http://localhost:5173/>, permitiendo interactuar con los deslizadores y replicar los perfiles Fintech, Streaming e IoT descritos en la investigación.

Apéndice D

Configuración y evaluación de las Network Policies por CNI

Este apéndice reúne la evidencia técnica que respalda el Objetivo Específico 2: *Evaluar cómo la configuración segura y automatizada de las políticas de red adaptadas a cada plugin CNI reduce la superficie de ataque y la probabilidad de incidentes de seguridad en clústeres Kubernetes*. Se documentan el alcance operativo bajo el que se interpretaron los resultados, los requisitos verificables que se derivan del enunciado, la matriz extendida de casos de prueba, el detalle del overhead por escenario, los fragmentos de configuración adaptados a cada CNI y el mapeo a marcos de referencia de la industria. El objetivo es dar trazabilidad entre cada decisión de configuración y cada resultado del cuerpo principal, de manera que las conclusiones puedan contrastarse con la evidencia. Las decisiones de alcance se declaran de forma explícita para acotar lo que esa evidencia sostiene.

1. Adaptación específica por CNI (R2)

El requisito R2 establece que las políticas no deben ser idénticas para los cuatro CNIs, sino aprovechar las capacidades diferenciales de cada plugin cuando estén disponibles. La Tabla D.1 resume las decisiones de adaptación adoptadas y la capacidad concreta que explota cada una; los fragmentos de configuración que siguen materializan los casos más representativos.

1.1. Fragmento de CiliumNetworkPolicy (L7)

El siguiente fragmento ilustra el filtrado L7 por método HTTP implementado en el caso de uso `multi_tier` bajo Cilium. Solo se permiten los métodos `GET` y `HEAD` desde Pods etiquetados como `tier=frontend` hacia el servicio `backend`.

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: frontend-to-backend-l7
```

CNI	Recurso usado	Capacidad diferencial explotada
Flannel	NetworkPolicy estándar (intencionalmente, como control negativo).	Ninguna: Flannel no implementa <i>enforcement</i> de políticas en su plano de datos.
Calico	NetworkPolicy estándar L3/L4 gestionada por Tigera Operator.	<i>Enforcement</i> maduro de políticas L3/L4 sobre el encapsulamiento VXLAN, sin requerir CRD propios.
Cilium	CiliumNetworkPolicy (L7).	Filtrado HTTP nativo en eBPF (GET/HEAD permitidos de frontend a backend; POST, PUT, DELETE bloqueados) sin <i>service mesh</i> .
Antrea	ClusterNetworkPolicy en Tier securityops.	Sistema jerárquico de Tiers que da prioridad a la política corporativa sobre cualquier NetworkPolicy de namespace, alineable a ISO 27001 y SOC 2.

1.2. Fragmento de ClusterNetworkPolicy de Antrea (Tier securityops)

```
apiVersion: crd.antrea.io/v1beta1
kind: ClusterNetworkPolicy
metadata:
  name: corp-block-frontend-to-db
spec:
  tier: securityops
  priority: 5
  appliedTo:
  - podSelector:
      matchLabels: { tier: database }
  ingress:
  - action: Drop
    from:
    - podSelector:
        matchLabels: { tier: frontend }
```

1.3. Default Deny base

El punto de partida de los tres casos de uso es una política de denegación total. Sobre esta base se habilita explícitamente cada flujo permitido, materializando el principio de *least privilege* de Zero Trust [12, 13].

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: production
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
```

Los manifiestos completos (incluidas las variantes `zero_trust`, `multi_tier` y `egress_block` para los cuatro CNIs) se publican en `K8s-Tesis-Apps/network_policies/` (Apéndice C).

2. Reproducibilidad del OE2

Los manifiestos y *overlays* de Kustomize que materializan las políticas evaluadas se encuentran en el repositorio `K8s-Tesis-Apps`, dentro del directorio `network_policies/`. La estructura de directorios es la siguiente:

- `network_policies/base/`: aplicación de prueba (frontend, backend, database) y *CronJobs* de medición.
- `network_policies/use-case-1-zero-trust/`: *default deny* y permiso explícito al tráfico de *benchmark*.
- `network_policies/use-case-2-multi-tier/`: reglas `frontend → backend → database` por etiquetas.
- `network_policies/use-case-3-egress-block/`: bloqueo de egreso externo manteniendo DNS y tráfico interno.
- `network_policies/overlays/{calico,cilium,antrea}/{zero-trust,multi-tier,egress-}` especialización por CNI, incluyendo `cilium-17-policy.yaml` y `antrea-security-tier.yaml`.

La rotación entre CNIs se realiza desde el flujo de provisionamiento descrito en el Apéndice C; el reciclaje del clúster garantiza que las pruebas se ejecuten sobre el plano de datos correspondiente a cada plugin en su configuración nativa.

Apéndice E

Marco de Criterios y Modelo de Decisión CNI

1. Introducción y el dilema de la selección de CNI

En la arquitectura de Kubernetes, el plano de datos depende del *Container Network Interface* (CNI). Al no existir un plugin de red por defecto, la elección recae sobre los ingenieros de infraestructura, quienes a menudo deciden por popularidad o guías rápidas, omitiendo un análisis cuantitativo sistemático de rendimiento.

El reto principal de elegir un CNI es que no existe una única solución óptima en todas las dimensiones simultáneamente. El rendimiento base de velocidad de red (*throughput*) es solo una dimensión del problema. Por ejemplo, una pasarela de transacciones bancarias exige seguridad estricta y aislamiento lógico, mientras que un clúster IoT en nodos con recursos de cómputo limitados prioriza un bajo consumo de memoria RAM y CPU.

La Tabla E.1 muestra a nivel conceptual este compromiso de rendimiento (*trade-off*) cualitativo entre los plugins evaluados, sirviendo de introducción visual al dilema de selección.

Cuadro E.1: Matriz comparativa cualitativa de compromisos (trade-offs) en CNIs.

Dimensión	Flannel	Calico	Cilium	Antrea
Latencia y jitter	Medio	Excelente	Bueno	Aceptable
Throughput	Bueno	Excelente	Aceptable	Bueno
Consumo de CPU	Excelente	Deficiente	Aceptable	Aceptable
Consumo de RAM	Excelente	Deficiente	Aceptable	Aceptable
Network Policies L4	Nulo (No soporta)	Excelente	Excelente	Excelente
Network Policies L7	Nulo (No soporta)	Nulo (No soporta)	Excelente	Nulo (No soporta)

Para dar solución a este problema de forma cuantitativa, este apéndice documenta el desarrollo y validación del Objetivo Específico 3 de la tesis: un modelo formal compuesto por criterios jerárquicos, métricas normalizadas, umbrales de calidad de servicio (SLA) y ponderaciones específicas de negocio.

2. Datos experimentales consolidados del testbed

Para alimentar el modelo matemático de decisión, se recolectaron datos cuantitativos de las pruebas experimentales del testbed. Los detalles técnicos del clúster de laboratorio (número de nodos, tipo de recursos, configuración de red e infraestructura cloud en DigitalOcean) y la metodología de recolección ya han sido ampliamente descritos en los Capítulos 7 y 8 de este documento.

Tras aplicar el filtro estadístico de rango intercuartílico (IQR) para eliminar valores atípicos, se obtuvieron las medias consolidadas que sirven como entrada del modelo. La Tabla E.2 presenta estos valores cuantitativos reales obtenidos del testbed.

Cuadro E.2: Resultados experimentales consolidados del testbed con indicadores de optimización.

Métrica de Entrada	Flannel	Calico	Cilium	Antrea
Latencia base promedio (↓)	21.53 ms	12.54 ms	18.73 ms	35.09 ms
Latencia máxima (↓)	50.80 ms	44.20 ms	47.00 ms	83.50 ms
Jitter (MDEV) (↓)	7.76 ms	7.31 ms	8.47 ms	17.25 ms
Throughput TCP promedio (↑)	1452.97 Mbps	1804.40 Mbps	843.56 Mbps	1588.45 Mbps
Retransmisiones TCP promedio (↓)	11774 paq.	68993 paq.	15142 paq.	42229 paq.
CPU bajo estrés (↓)	19.45 %	29.05 %	25.79 %	27.75 %
RAM bajo estrés (↓)	1176.54 MB	1672.83 MB	1497.19 MB	1429.95 MB

Para facilitar la interpretación de los datos, cada métrica incluye un indicador del sentido óptimo del valor:

- **Sentido descendente (↓):** Indica que un valor menor representa un mejor comportamiento técnico. Por ejemplo, en la latencia base, el valor de Calico (12,54 ms) es superior al de Flannel (21,53 ms) porque reduce el retardo de transmisión. Lo mismo aplica para el jitter, consumo de CPU, RAM y retransmisiones TCP.
- **Sentido ascendente (↑):** Indica que un valor mayor representa un mejor comportamiento técnico. Se utiliza principalmente en el *throughput* (ancho de banda disponible).

3. Resultados MCDA y análisis de selección

La Tabla E.3 detalla las puntuaciones finales reales del análisis MCDA para cada plugin. El símbolo de asterisco (*) destaca la recomendación ganadora del modelo para cada perfil.

3.1. Interpretación y desglose de resultados

- **Perfil Fintech / Seguridad:** Cilium obtiene la puntuación óptima con 3,631 (en la escala de 1 a 5). A pesar de que Calico ofrece un rendimiento crudo superior en latencia base, Cilium sobresale debido a sus políticas dinámicas de capa 7 basadas

Cuadro E.3: Puntuaciones consolidadas del modelo MCDA reales para cada perfil (escala 1–5).

Perfil de Usuario	Flannel	Calico	Cilium	Antrea
Fintech / Seguridad crítica	1.331	3.168	* 3.631	2.211
Streaming / Alto rendimiento	* 3.686	3.567	3.294	2.256
IoT / Recursos limitados	* 3.834	2.952	3.232	2.224

en eBPF [24] y su menor degradación del tráfico cuando el aislamiento lógico está activo.

Por el contrario, Flannel obtiene un puntaje críticamente bajo de 1,331. Esto se debe a que carece por diseño de soporte de Network Policies. Su puntuación base ponderada original de 3,328 (impulsada principalmente por su bajo consumo de CPU y RAM) se somete a la penalización de seguridad de la Ecuación (??), multiplicándose por 0,4. Esto demuestra cómo el modelo descarta automáticamente soluciones no aptas para entornos regulados.

Réplica en el Recomendador SPA: Para obtener exactamente este resultado de 3,631, el usuario debe establecer en el panel de control del recomendador las siguientes prioridades en los deslizadores (sliders):

- Latencia: *Medio* (3/5)
- Estabilidad (Jitter): *Muy Alto* (5/5)
- Throughput: *Alto* (4/5)
- Límite de Recursos: *Muy Bajo* (1/5)
- Nivel de Seguridad: *Muy Alto* (5/5)

Esta selección genera ponderaciones específicas (Latencia 20,8 %, Estabilidad 24,8 %, Throughput 21,0 %, Recursos 8,1 % y Seguridad 25,3 %). Al requerirse un nivel de seguridad crítico, se activa automáticamente la penalización del factor $\times 0,40$ sobre Flannel. La configuración y los resultados se demuestran visualmente en la Figura E.1 (configuración de entrada) y la Figura E.2 (salida final del prototipo web).

- **Perfil Streaming / Rendimiento:** Flannel resulta recomendado con un puntaje de 3,686. En el testbed, Flannel demostró un rendimiento balanceado y la estabilidad de red más alta al registrar solo 11,774 retransmisiones TCP frente a las 68,993 de Calico. Dado que la tasa de reenvío influye en la estabilidad de la transmisión de flujos continuos, la robustez de Flannel sumada a su eficiencia computacional compensa el mayor throughput bruto de Calico.

Réplica en el Recomendador SPA: Para replicar el puntaje de 3,686 de Flannel y 3,567 de Calico, el usuario debe establecer las siguientes prioridades:

- Latencia: *Muy Alto* (5/5)

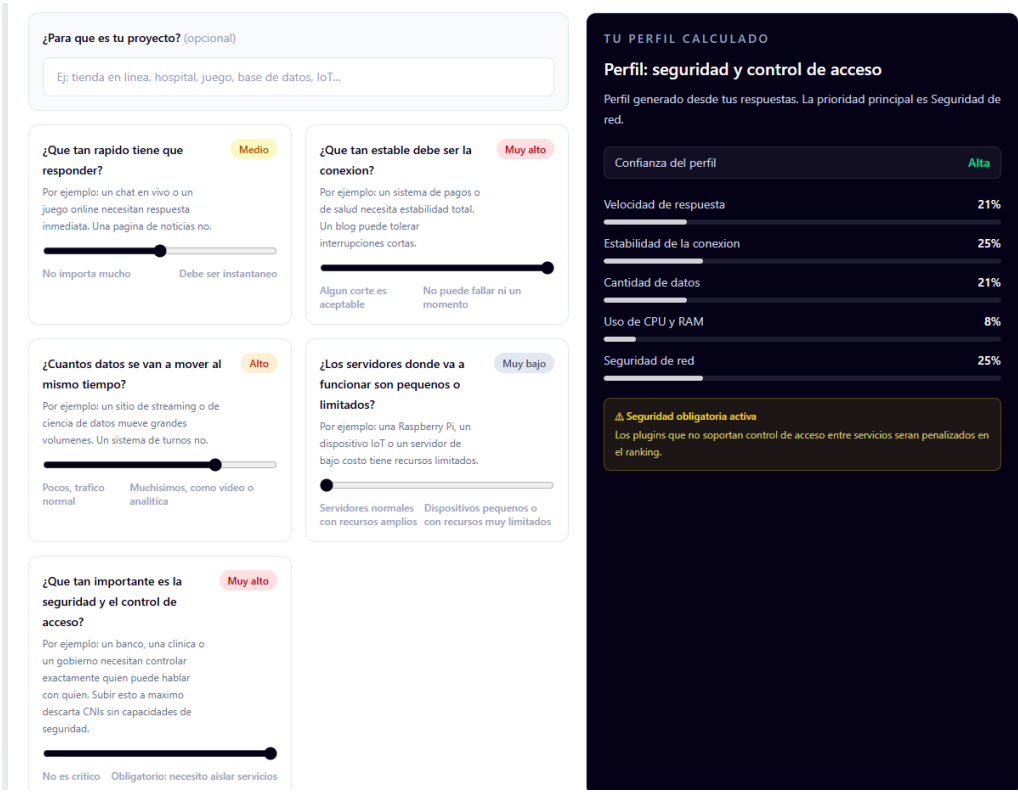


Figura E.1: Configuración de sliders para el perfil Fintech / Seguridad en el Recomendador CNI.



Figura E.2: Resultados del ranking MCDA en el prototipo interactivo para el perfil Fintech / Seguridad (Ganador: Cilium con 3.631).

- Estabilidad (Jitter): *Alto* (4/5)
- Throughput: *Muy Alto* (5/5)
- Límite de Recursos: *Bajo* (2/5)
- Nivel de Seguridad: *Bajo* (2/5)

Esta configuración prioriza la QoS de red cruda (throughput y latencia con pesos del 25,1 % y 27,7 % respectivamente) sin requerir seguridad restrictiva, lo que resulta en la victoria de Flannel por estabilidad.

- **Perfil IoT / Recursos limitados:** Flannel se posiciona como el CNI recomendado con 3,834 frente a Cilium (3,232), Calico (2,952) y Antrea (2,224). Flannel destaca como el plugin que induce el menor estrés general bajo carga (19,45 % de CPU del nodo y 1176,54 MB de RAM global), lo cual se alinea con la prioridad de este perfil (28 % de peso a la eficiencia de recursos). Como se discute en el Capítulo 11, esta métrica de RAM captura la huella del host bajo tráfico activo; un despliegue en nodos muy pequeños deberá aislar adicionalmente el consumo del binario en reposo.

Réplica en el Recomendador SPA: Para replicar el puntaje de 3,834 de Flannel, el usuario debe configurar los sliders con los siguientes valores:

- Latencia: *Medio* (3/5)
- Estabilidad (Jitter): *Medio* (3/5)
- Throughput: *Medio* (3/5)
- Límite de Recursos: *Muy Alto* (5/5)
- Nivel de Seguridad: *Bajo* (2/5)

Esta selección asigna un 28,0 % de peso a la eficiencia de recursos. La Figura E.3 y la Figura E.4 muestran respectivamente la entrada en la interfaz de usuario y el gráfico de barras resultante de la recomendación interactiva del sistema.

4. Lista de verificación para la configuración del recomendador CNI

El recomendador interactivo opera mediante cinco deslizadores de prioridad, cada uno en la escala *Muy Bajo* (1) — *Bajo* (2) — *Medio* (3) — *Alto* (4) — *Muy Alto* (5). La combinación de valores genera los pesos porcentuales del modelo MCDA y, cuando la seguridad supera el nivel 4, activa la penalización no compensatoria $\times 0,40$ sobre plugins sin soporte de `NetworkPolicy`.

Esta lista de verificación es un instrumento cualitativo: mediante preguntas orientadas al contexto operativo del equipo, guía la asignación de cada deslizador antes de acceder al recomendador. No requiere mediciones previas; basta con que el equipo responda a cuál de las descripciones de cada fila se ajusta mejor a su caso de uso.

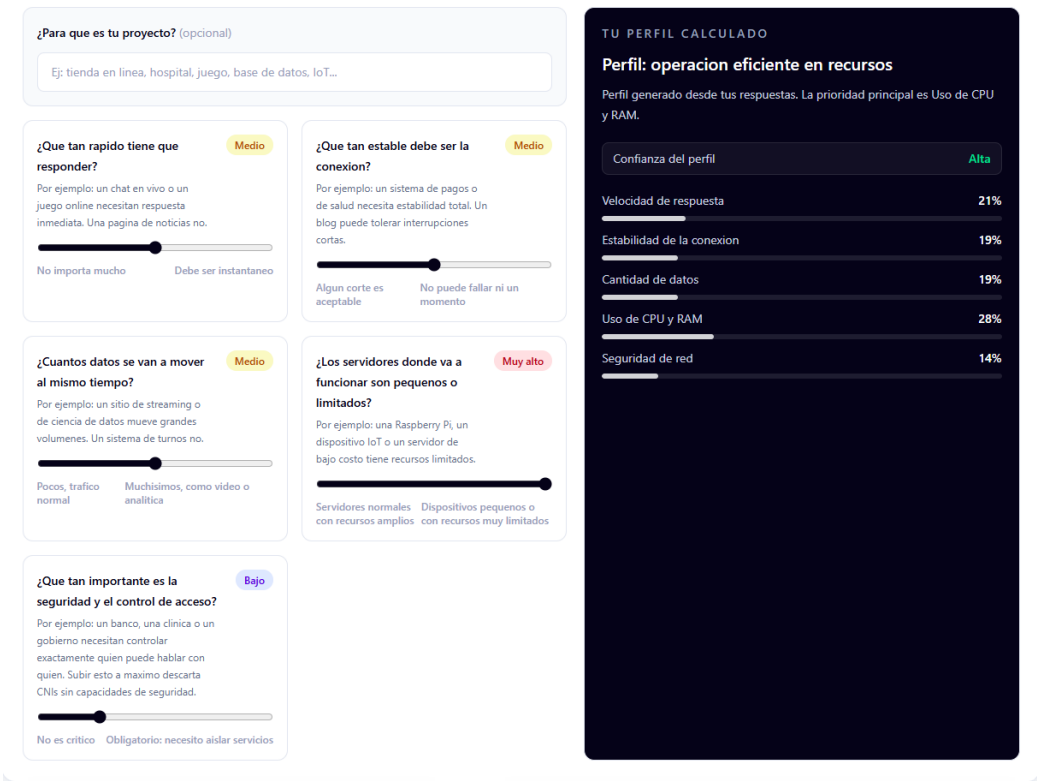


Figura E.3: Configuración de sliders para el perfil IoT / Recursos Limitados en el Recomendador CNI.



Figura E.4: Resultados del ranking MCDA en el prototipo interactivo para el perfil IoT / Recursos Limitados (Ganador: Flannel con 3.834).

Cuadro E.4: Lista de verificación cualitativa para configurar los deslizadores del recomendador CNI.

Deslizador	Indicadores de prioridad <i>Alta</i> o <i>Muy Alta</i> (4–5)	Indicadores de prioridad <i>Baja</i> o <i>Muy Baja</i> (1–2)	Nivel elegido
Latencia	APIs síncronas, portales web interactivos, bases de datos transaccionales o cualquier carga donde el usuario percibe el tiempo de respuesta individualmente.	Procesamiento por lotes (<i>batch</i>), pipelines ETL, transferencias nocturnas o cargas donde la velocidad de respuesta por petición no es crítica.	/5
Estabilidad	Transmisión de vídeo o audio en tiempo real, juegos en línea, protocolos sensibles a retransmisiones o servicios con acuerdos de nivel de servicio (SLA) de disponibilidad continua.	Cargas tolerantes a reintentos, transferencias de archivos grandes sin restricción de tiempo o sistemas con lógica propia de reintento a nivel de aplicación.	/5
Throughput	Ingesta masiva de datos, distribución de contenido multimedia, analítica en tiempo real o cualquier carga que mueva grandes volúmenes de datos por segundo entre pods.	Microservicios de baja intensidad, chatbots, sistemas de notificaciones o cargas donde el número de bytes por transacción es reducido aunque el número de peticiones sea alto.	/5
Recursos	Nodos con capacidad restringida (Raspberry Pi, dispositivos edge, instancias mínimas), clústeres de bajo costo o despliegues donde cada megabyte o millicore disponible importa para las cargas de trabajo.	Nodos de propósito general con capacidad amplia, entornos cloud elásticos o clústeres donde el CNI comparte recursos sin competencia con aplicaciones críticas.	/5
Seguridad	Datos financieros, información médica o de identidad, entornos regulados (PCI-DSS, HIPAA, ISO 27001), arquitecturas Zero Trust, o cualquier clúster que aloje cargas de trabajo de distintos niveles de confianza en el mismo namespace.	Entornos de laboratorio, clústeres de desarrollo o pruebas aislados de internet, pipelines de CI/CD sin datos sensibles o clústeres monoproósito donde el tráfico inter-pod es homogéneo y confiable.	/5

Instrucciones de uso

1. Para cada deslizador, leer las dos columnas de indicadores y decidir cuál describe mejor el entorno operativo del clúster.
2. Anotar el nivel elegido (1–5) en la última columna. Si el contexto queda entre ambas descripciones, usar el valor intermedio (*Medio*, 3/5).
3. Si **Seguridad** queda en nivel 4 o 5, verificar que el CNI seleccionado aplique **NetworkPolicy** en su plano de datos; de lo contrario, la penalización $\times 0,40$ lo descartará automáticamente.
4. Introducir los cinco valores en el recomendador interactivo (<https://duwansierra.github.io/K8S-CNI-Results/>) para obtener el ranking MCDA personalizado.
5. Comparar el resultado con los perfiles predefinidos de la Tabla 8.5: si los valores elegidos se acercan a uno de los tres perfiles base (Fintech, Streaming, IoT), el recomendador producirá un ranking análogo al documentado en la Tabla E.3.

Nota sobre la penalización de seguridad. Los perfiles predefinidos con nivel de seguridad 4 o 5 activan automáticamente la restricción no compensatoria: Flannel recibe el 40 % de su puntaje bruto, lo que lo sitúa por debajo de 2.0 en la escala 1–5 en contextos regulados, independientemente de su ventaja en recursos o estabilidad.

5. Guía de reproducción y verificación científica

Para la reproducción y verificación científica paso a paso de los resultados experimentales y la compilación local del recomendador web, consulte la guía técnica detallada de comandos provista en la Sección 3 del Apéndice A.